



Programa Profesional en Full Stack Developer

Módulo 5. Desarrollo de Aplicaciones en Node y Express JS

Índice

Tema 1. Introducción a NodeJS

- 1.1. Introducción y objetivos
- 1.2. Cómo funciona NodeJS
- 1.3. Características principales de NodeJS
- 1.4. Instalación de NodeJS
- 1.5. Creación y uso de módulos
- 1.6. Dependencias, package.json y node_modules
- 1.7. Referencias bibliográficas

Tema 2. Node Core

- 2.1. Introducción y objetivos
- 2.2. El objeto global de NodeJS
- 2.3. Control del proceso de una aplicación
- 2.4. Funciones de timing
- 2.5. Usos del módulo readline
- 2.6. Gestión de eventos con EventEmitter
- 2.7. Ejecución de procesos externos

Tema 3. File System

- 3.1. Introducción y objetivos
- 3.2. Lectura de directorios
- 3.3. Lectura y escritura de ficheros
- 3.4. Operaciones más comunes sobre ficheros
- 3.5. Manejo de Streams

Tema 4. Http

- 4.1. Introducción y objetivos
- 4.2. Creación básica de peticiones
- 4.3. Creación de servidores básicos
- 4.4. Respondiendo a peticiones POST
- 4.5. Respondiendo con JSON

Tema 5. ExpressJS

- 5.1. Introducción y objetivos
- 5.2. Características de Express
- 5.3. Arquitectura de aplicación Express
- 5.4. Gestión de rutas
- 5.5. Creación de middlewares
- 5.6. Respuestas disponibles

Tema 6. Integración de bases de datos MySQL

- 6.1. Introducción y objetivos
- 6.2. Conexión a la base de datos
- 6.3. Creación de modelos e integración de queries
- 6.4. CRUD sobre MySQL para la creación de una API

Tema 7. Integración de bases de datos Mongo DB

- 7.1. Introducción y objetivos
- 7.2. Conexión a la base de datos con Mongoose
- 7.3 Creación de esquemas
- 7.4 Métodos CRUD de acceso a la base de datos

Tema 8. Template Engines

- 8.1. Introducción y objetivos
- 8.2. Gestión de plantillas en Express
- 8.3. Características básicas de PUG
- 8.4. Características básicas de HBS

Tema 9. Sesiones y autenticación

- 9.1. Introducción y objetivos
- 9.2. Configuración y manejo de sesiones con Express
- 9.3. Configuración y manejo de cookies con Express
- 9.4. Registro API Rest. Encriptación de password
- 9.5. Login API Rest
- 9.6. Middleware de autenticación

Tema 10. Sockets

- 10.1. Introducción y objetivos
- 10.2. ¿Qué son los websockets?
- 10.3. Ejemplo básico de websocket
- 10.4. Uso de la librería Socket.io

Tema 1. Introducción a NodeJS

1.1. Introducción y objetivos

A partir del módulo que arrancamos con este tema vamos a empezar a descubrir cómo podemos generar aplicaciones web complejas tomando como base para el desarrollo NodeJS, uno de los motores de ejecución de código Javascript más potentes del mercado.

Tomando como base todos los conceptos adquiridos en el desarrollo Javascript y extrayendo información y datos de las diferentes bases de datos seremos capaces de generar aplicaciones que resuelvan cualquier tipo de proyecto.

En este primer tema sentaremos las bases de qué es NodeJS y cómo lo vamos a utilizar a nuestro favor a la hora de generar nuestros desarrollos. En nuestro caso particular vamos a establecer muchos de los conceptos orientados al mundo del desarrollo web, pero tenemos que entender que NodeJS se trata de una herramienta que va más allá de la simple creación de aplicaciones o páginas web.

La aparición de NodeJS marcó un hito en la historia del desarrollo web y en este tema vamos a tratar de desgranar el por qué.

Tema 1. Introducción a NodeJS

1.2. Cómo funciona NodeJS

¿Qué es NodeJS?

En muchas ocasiones nos enfrentamos a definiciones incorrectas de lo qué es NodeJS y es por eso por lo que debemos tener muy claro a qué nos enfrentamos para poder seleccionar las herramientas correctas de cara a nuestros desarrollos.

NodeJS es un entorno en tiempo de ejecución multiplataforma, de código abierto, orientado a la creación de aplicaciones en el servidor y basado en el lenguaje Javascript.

Podemos sacar unas cuantas conclusiones de esta definición y sobre todo analizar por qué NodeJS ha marcado un antes y un después en el desarrollo de aplicaciones web modernas.

Hasta la aparición de NodeJS, si necesitábamos lanzar cualquier tipo de script basado en Javascript, dependíamos de los navegadores web. Cada uno de ellos, dependiendo de la empresa a la que pertenezcan, disponen de un motor de ejecución Javascript que se encarga de interpretar el código.

Todas las creaciones que se realizaban a partir de Javascript eran para completar desarrollos web sobre el navegador. Actuaba como un lenguaje de apoyo para realizar modificaciones sobre el HTML que interpreta el navegador.

NodeJS rompe con estas pautas ya que es el primer motor de ejecución Javascript que **nos da la posibilidad de extraer el código del navegador y ejecutarlo dentro de un servidor**, es decir, independientemente dentro de un ordenador. Aunque su tarea principal es la creación de servidores (como trataremos más adelante), podemos encontrar aplicaciones desarrolladas con NodeJS en múltiples ámbitos como relojes, placas arduino, raspberry pi...

Tema 1. Introducción a NodeJS

Con este nuevo entorno donde poder crear aplicaciones las posibilidades que nos ofrece NodeJS son infinitas. Apoyándonos en las miles de librerías existentes podemos tratar el manejo de ficheros, la comunicación en tiempo real entre diferentes máquinas, la extracción de datos a través de internet, sin dejar de lado la creación de servidores para la gestión de peticiones web.

NodeJS está basado en el motor V8 de Javascript perteneciente a Google. Se trata del motor de ejecución que usa por defecto Google Chrome, está escrito en C++ y es de código abierto, por lo cual podríamos entrar en detalle dentro del código para poder afinar mucho más nuestros desarrollos.

No es obligatorio conocerlo en profundidad ya que para el desarrollador web es simplemente una herramienta con la cual conseguiremos alguno de nuestros objetivos. Si nuestro trabajo va orientado a sacar el mayor rendimiento de nuestros desarrollos, en ese caso sí sería una buena idea investigar cómo funciona por dentro.

Continúa tu exposición incluyendo los apartados que necesites. Recuerda que cuanto más amplio sea el índice, más fácil será para el alumno encontrar la información en el aula virtual.

Creación de servidores con NodeJS

En el mundo de la creación de servidores siempre se han impuesto todos aquellos creados a partir de Apache. Es más, la mayoría de los servidores que nos podemos encontrar hoy en día dentro de internet están basados en esta tecnología. **¿Cómo cambia la gestión de servidores si usamos NodeJS?**

El trabajo fundamental de cualquier servidor que desarrollemos es el de **recibir peticiones desde el cliente (aplicación web, aplicación móvil, chatbot...) y procesar la respuesta**. Cada tecnología lleva esta tarea a cabo de una manera

Tema 1. Introducción a NodeJS

diferente y es por eso por lo que, dependiendo el objetivo que necesitemos alcanzar, escogeremos una u otra.

Una de las limitaciones que encontramos dentro de los servidores creados con Apache es su falta de rendimiento cuando se incrementan el número de peticiones. Cada petición entrante en el servidor es tratada por un proceso independiente, es decir, si tenemos millones de peticiones entrando a la vez en un servidor Apache, tendremos millones de proyectos funcionando a la vez dentro de una máquina.

Esta acción acarrea las mismas consecuencias que tendría en un ordenador convencional, si nosotros lanzamos muchas aplicaciones a la vez, el rendimiento del ordenador se ve afectado y tenemos que esperar a que algunas aplicaciones se cierren para continuar con su uso normal.

La solución para un problema de este tipo es la adquisición de más servidores para poder hacer frente a todos los procesos que se arrancan al mismo tiempo.

¿Cómo trata de resolver este problema NodeJS?

Uno de los conceptos con los que más vamos a convivir cuando se trata de NodeJS es **la asincronía**. Casi siempre debemos pensar de manera asíncrona a la hora de desarrollar las tareas que vamos a tener en nuestros servidores.

En nuestros servidores disponemos de **un único proceso en ejecución** que se encarga de recibir la petición y la coloca dentro de un **pool de threads** creado a partir de la librería **libuv**.

Dentro de este pool las diferentes tareas se van resolviendo de manera asíncrona, es decir, no bloqueamos ninguna de ellas. Cuando se realiza el trabajo determinado por la tarea ésta se libera y es de nuevo el proceso de ejecución el encargado de recoger la respuesta y enviarla de vuelta al cliente.

Tema 1. Introducción a NodeJS

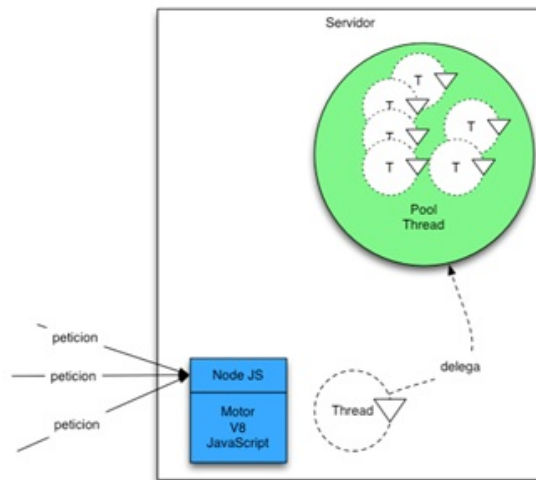


Figura 1. NodeJS Pool Thread. Fuente: Álvarez, 2014.

La mayor parte del código que nos vamos a encontrar durante nuestro aprendizaje de las herramientas más comunes de NodeJS dispondrá de **callbacks o promesas** para poder gestionar esa asincronicidad que hacen de NodeJS la herramienta perfecta para desarrollos en los que necesitemos mucha velocidad y alto rendimiento.

Ejemplo práctico

Vamos a analizar esta asincronicidad a partir de un ejemplo en el que ejecutaremos una sencilla función de la librería **File System** de NodeJS.

```
console.log("inicio");
fs = require ('fs');
fs.readFile('fichero.txt', 'utf8', function(err, datos){
  if (err){
    return console.log(err);
  }
  console.log(datos);
});
console.log("final");
```

Para poder ejecutar este código sin tener instalado NodeJS, puedes hacerlo a través

Tema 1. Introducción a NodeJS

de este enlace: <https://replit.com/@mariogironm/PrimerEjemplo-1#index.js>

En el ejemplo podemos observar cómo existen 3 líneas en las que estamos lanzando la sentencia de **console.log**. Si estuviésemos trabajando en un entorno de programación normal la lógica nos diría que la ejecución debería ser secuencial.

En este ejemplo vemos cómo lo trata NodeJS. Primero se ejecutará el log de inicio y posteriormente la función **readFile**, encargada de hacer la lectura del fichero. Esta lectura del fichero es una acción totalmente asíncrona que no bloquea la ejecución de las líneas que tendríamos a continuación. Es por eso por lo que, antes de obtener el resultado de dicha lectura, observaremos la ejecución del log final.

Una vez se termine la recopilación de datos del fichero, la ejecución de nuestra aplicación devolverá el control a la función **callback** que encontramos como segundo parámetro de la llamada a **readFile**.

La mayoría de los métodos con los que vamos a trabajar disponen de esta serie de herramientas para evitar bloqueos dentro de nuestras aplicaciones creadas con NodeJS. Es tarea del desarrollador buscar las mejores alternativas para desarrollar un código de calidad y totalmente escalable.

Tema 1. Introducción a NodeJS

1.3. Características principales de NodeJS

Las principales características en las cuales se basan todos los desarrollos que realicemos con NodeJS son las siguientes:

- ▶ Utiliza un **modelo de no-bloqueo orientado a eventos**. Esto provoca que nuestros desarrollos sean muy ágiles y eficientes. Destacan todas aquellas aplicaciones creadas para el intercambio de información en tiempo real.
- ▶ **Se programa del lado del servidor**. Es por ello por lo que disponemos de muchas más herramientas de las que tenemos cuando hacemos desarrollos en el navegador con Javascript. No hay que olvidar que ahora estamos programando dentro de un servidor por lo que vamos a tener acceso completo a todos los elementos propios del sistema operativo con el que trabajemos.
- ▶ **NodeJS implementa todos los protocolos de comunicación más habituales** (http, https, ws...). El abanico de posibilidades a la hora de crear aplicaciones crece de manera exponencial y no necesariamente nos tenemos que centrar en la creación de servidores.
- ▶ Es muy importante el trabajo que hace NodeJS sobre las acciones concurrentes que se ejecutan en nuestros desarrollos. La asincronicidad que se le presupone nos permite ejecutar múltiples tareas a la vez sin que existan demasiados tiempos de espera o bloqueos entre ellas.
- ▶ Mediante el uso de **callbacks o promesas** obtenemos esa ayuda necesaria para el desarrollador para manejar todos estos ámbitos asíncronos.
- ▶ Para evitar problemas de **código piramidal** se implementan soluciones que nos ayudan a sustituir en ciertos momentos críticos el uso de callbacks por otras herramientas equivalentes.

Tema 1. Introducción a NodeJS

1.4. Instalación de NodeJS

La forma oficial de instalación de la herramienta es la más rápida y eficiente. Para poder llevar a cabo debemos entrar en la página oficial de descarga:

<https://nodejs.org/en/download/>

The screenshot shows the Node.js download page with two main tabs: **LTS** (Recommended For Most Users) and **Current** (Latest Features). Under the LTS tab, there are three main categories: **Windows Installer** (node-v16.13.2-x64.msi), **macOS Installer** (node-v16.13.2.pkg), and **Source Code** (node-v16.13.2.tar.gz). Below these, there are links for **Windows Installer (.msi)**, **Windows Binary (.zip)**, **macOS Installer (.pkg)**, **macOS Binary (.tar.gz)**, **Linux Binaries (x64)**, **Linux Binaries (ARM)**, and **Source Code**. To the right, there are tables for different architectures: 32-bit, 64-bit, 64-bit / ARM64, 64-bit, ARMv7, and ARMv8. Under the 'Additional Platforms' section, there are links for **Docker Image** (Official Node.js Docker Image), **Linux on Power LE Systems** (64-bit), **Linux on System z** (64-bit), and **AIX on Power Systems** (64-bit).

Architecture	Download Link
32-bit	node-v16.13.2-x86.msi
64-bit	node-v16.13.2-x64.msi
64-bit / ARM64	node-v16.13.2-arm64.msi
64-bit	node-v16.13.2-x64.msi
ARMv7	node-v16.13.2-armv7.tar.gz
ARMv8	node-v16.13.2-armv8.tar.gz

Platform	Download Link
Docker Image	Official Node.js Docker Image
Linux on Power LE Systems	node-v16.13.2-ppc64le.tar.gz
Linux on System z	node-v16.13.2-s390x.tar.gz
AIX on Power Systems	node-v16.13.2-aix.tar.gz

Figura 2. Opciones de instalación de NodeJS. Fuente: elaboración propia.

De todas las opciones para nuestro sistema operativo y la versión **LTS** seleccionamos la que mejor nos convenga. Se descargará y ejecutará un instalador estándar el cual nos pedirá una serie de confirmaciones para proceder a la instalación.

Para comprobar que todo ha terminado de manera correcta podemos abrir una instancia del terminal dentro de nuestro ordenador y lanzar el comando.

node -version

Si el resultado de esta ejecución nos devuelve información sobre la versión que

Tema 1. Introducción a NodeJS

hemos instalado previamente quiere decir que hemos completado la instalación de NodeJS correctamente en nuestro ordenador.

A partir de ahora, si necesitamos ejecutar cualquier tipo de archivo Javascript, lo podemos hacer con NodeJS de la siguiente manera.

node nombreFichero.js

Tema 1. Introducción a NodeJS

1.5. Creación y uso de módulos

Mediante el uso de módulos vamos a poder organizar nuestro código de una manera más eficiente y además nos permitirá poder hacer modificaciones y escalar las diferentes funcionalidades sin demasiada dificultad.

Una de las buenas prácticas que debemos seguir a la hora de programar para NodeJS es la de **evitar ficheros con miles de líneas de código**. La modificación de este tipo de ficheros o el desarrollo de pruebas para los mismos se convierten en algo muy tedioso y complicado.

Cuanto más separemos las funcionalidades en distintos ficheros más cómodas se van a volver las modificaciones sobre nuestros proyectos.

En NodeJS disponemos de la función **require()** la cual nos permite importar ciertos bloques de código para compartir funcionalidades entre los diferentes ficheros de nuestra aplicación. Lo vemos a través de un ejemplo.

Definimos el fichero **operaciones.js**:

```
function suma(a, b) {  
  return a + b;  
}  
function resta(a, b) {  
  return a - b;  
}  
module.exports = {  
  suma: suma,  
  resta: resta  
}
```

Dentro de este fichero estamos creando dos funciones, **suma y resta** con cálculos muy sencillos.

En la parte final del fichero y a través de la sentencia **module.exports** generamos un

Tema 1. Introducción a NodeJS

objeto donde vamos a asignar a cada una de las claves el valor que necesitemos exportar y por tanto que quede accesible desde el exterior. En este caso exportamos las dos funciones generadas.

Si creamos un fichero **index.js** podemos observar cómo hacer uso de esta técnica:

```
const operaciones = require('./operaciones');  
console.log(operaciones.suma(2, 6));  
console.log(operaciones.rest(6, 2));
```

Lo único que debemos tener en cuenta es que, en este segundo fichero, utilizaremos las mismas claves que hemos exportado en el primero. Esas claves llevan asociadas cada una de las funciones (suma y resta) y por lo tanto las ejecutamos como tal.

De esta manera nuestro código queda mucho más limpio y podemos separar diferentes funcionalidades lo cual va a mejorar nuestros desarrollos de cara a un futuro.

De la misma manera podemos exportar cualquier tipo de dato y solo objetos complejos:

mensaje.js

```
module.exports = 'Hola NodeJS';
```

index.js

```
const msg = require('./mensaje');  
console.log(msg);
```

Incluso podemos exportar únicamente una función:

log.js

```
exports.log = (msg) => {  
  console.log(msg);  
}
```

Tema 1. Introducción a NodeJS

index.js

```
const log = require('./log');  
log('Esto es una prueba')
```


Tema 1. Introducción a NodeJS

1.6. Dependencias, package.json y node_modules

Uno de los puntos importantes del uso de NodeJS es la cantidad de librerías generadas por la comunidad que podemos utilizar en nuestros propios desarrollos. A este tipo de librerías las denominamos dependencias y debemos instalarlas en cada uno de los proyectos donde las vayamos a utilizar.

Para poder instalar y utilizar estas dependencias, el primer paso es la generación de un directorio donde colocaremos todo nuestro código. Dentro del terminal, ejecutamos:

```
mkdir proyectoprueba
```

```
cd proyectoprueba
```

Una vez tengamos el directorio generado podemos crear el fichero **package.json** donde tendremos toda la información del proyecto así como las dependencias del mismo. Dentro del terminal lanzamos el siguiente comando:

```
npm init -y
```

Si analizamos el resultado veremos que se ha generado un fichero llamado **package.json** el cual tendrá el siguiente aspecto:

```
{
  "name": "proyectoprueba",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Tema 1. Introducción a NodeJS

Este fichero es muy importante para poder llevar un control de toda la información referida al proyecto que estamos llevando a cabo. Cualquier de las propiedades que tenemos dentro del fichero son modificables en función de nuestras necesidades.

Si necesitamos instalar algún tipo de dependencia debemos dejar un registro de esta dentro de este fichero para marcar que el proyecto necesita una serie de librerías para su correcto funcionamiento.

En este ejemplo vamos a instalar la librería **dayjs**, la cual nos permite trabajar con diferentes tipos de fechas:

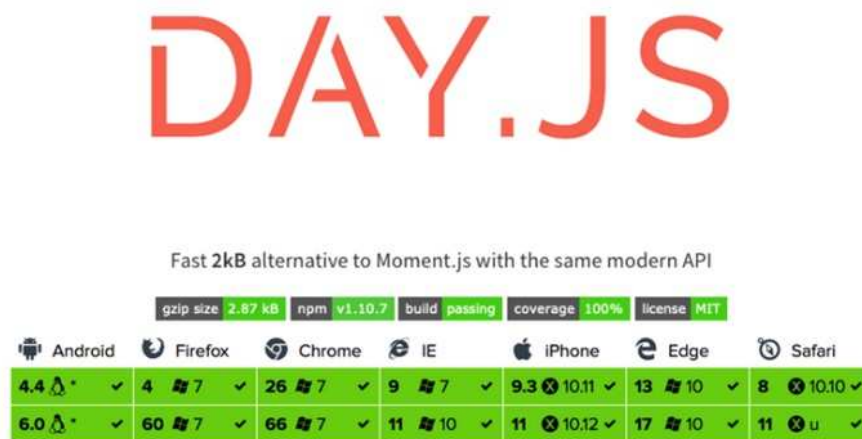


Figura 3. Compatibilidad de librería DayJS. Fuente: <https://www.npmjs.com/package/dayjs>

Para proceder a su instalación y su posterior uso, ejecutamos dentro del terminal el comando genérico de instalación:

npm install dayjs

Este comando siempre es el mismo independientemente de la librería, pero, casi siempre, viene especificado en la propia documentación de la librería.

Una vez descargado el código necesario para el buen funcionamiento de la librería debemos prestar atención a dos aspectos:

Tema 1. Introducción a NodeJS

- ▶ Dentro del fichero **package.json** podemos observar una nueva sección llamada **dependencies** donde encontraremos la referencia a la librería instalada y su versión.

```
"dependencies": {  
  "dayjs": "^1.10.7"  
}
```

- ▶ En la raíz del proyecto se habrá generado una carpeta **node_modules** con el código correspondiente a la librería instalada, así como de las librerías que tiene dayjs como dependencias. Esta descarga es automática y es npm la herramienta encargada de hacerla.

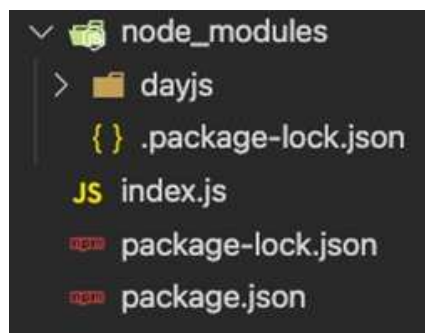


Figura 4. Organización del proyecto. Fuente: elaboración propia.

A partir de aquí ya tenemos el código de la librería disponible en nuestro proyecto y podemos utilizarlo dentro de cualquier fichero como el siguiente:

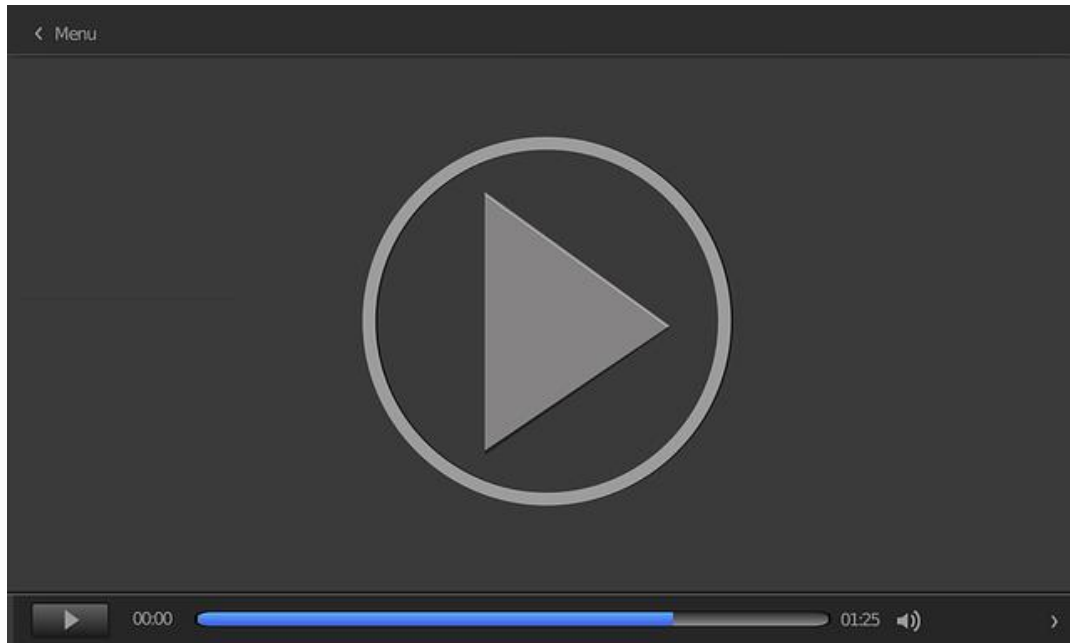
index.js

```
const dayjs = require('dayjs');  
console.log(dayjs().format('D/M/YY'));
```

Si ejecutamos el fichero anterior observamos el resultado del uso de la librería.

Tema 1. Introducción a NodeJS

En el vídeo **Creación de proyecto npm** se muestran las acciones básicas para la creación y posterior instalación de dependencias dentro de un proyecto basado en NodeJS.

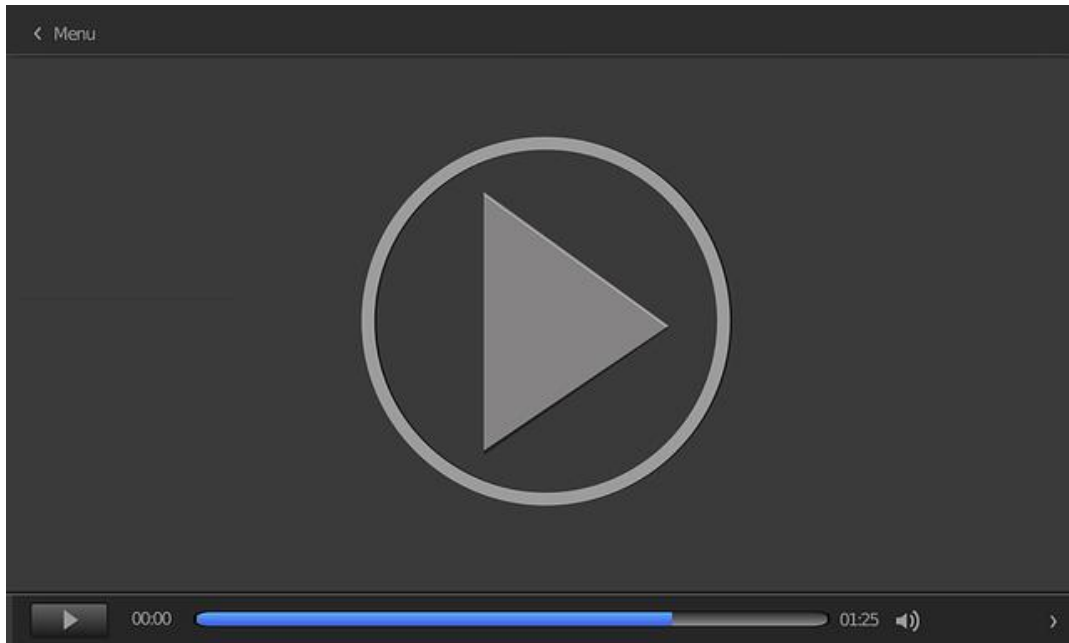


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=3ef6343d-6144-45a0-a27e-ae2d016aa1a1>

Tema 1. Introducción a NodeJS

En el vídeo **Creación de módulos** se muestran ejemplos de creación de módulos dentro de NodeJS para poder organizar de la manera más eficiente nuestras aplicaciones.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=413bea5f-588c-4df5-a9bb-ae2d01722cb3>

Tema 1. Introducción a NodeJS

1.7. Referencias bibliográficas

Álvarez, C. (9 de julio de 2014). *NodeJS Pool Thread* [infografía]
<https://www.genbeta.com/desarrollo/como-funciona-node-js>

2.1. Introducción y objetivos

Uno de los principios fundamentales de NodeJS es el uso de librerías externas como ya hemos visto en el tema anterior. Pero también disponemos de una serie de librerías que vienen incluidas con la propia instalación de la herramienta.

Las librerías que podemos utilizar sin realizar ningún tipo de instalación extra suelen contener todas las acciones que podemos llevar a cabo desde la máquina donde tengamos instalada la aplicación que estemos desarrollando. Por ejemplo, acciones básicas contra el sistema operativo, lectura y escritura de ficheros, conexión con diferentes sistemas externos...

En este tema vamos a analizar las librerías más básicas de NodeJS y además intentaremos que este aprendizaje nos sirva como primera piedra de toque de cara a coger experiencia a la hora de enfrentarnos a este tipo de funciones.

2.2. El objeto global de NodeJS

Existe un namespace global donde podemos encontrar muchos de los métodos que utilizamos habitualmente dentro de nuestros desarrollos con NodeJS. Se trata de un grupo de variables y métodos que estarían accesibles sin necesidad de ser importados desde cualquier fichero de nuestra aplicación.

Un ejemplo de propiedad dentro de este espacio es **console**, mediante la cual podemos acceder al método **log** para poder mostrar ciertos literales por consola. Las dos sentencias del siguiente código serían totalmente equivalentes.

```
global.console.log('Hola Mundo');  
console.log('Hola Mundo');
```

De todas maneras, en este aspecto, no es necesario hacer referencia al espacio global y nuestra opción, tal y como hacemos cuando estamos trabajando con Javascript nativo.

En cada una de las aplicaciones que generemos se va a crear un ámbito global dentro del cual podríamos incluir nuestros propios valores y poder acceder a ellos desde cualquier punto de nuestro código. De todas maneras, esta forma de actuar no es demasiado recomendable y ya analizaremos otras técnicas para poder compartir información a lo largo de nuestras aplicaciones.

Todas las novedades que se van incluyendo dentro del lenguaje Javascript en sus nuevas versiones están disponibles dentro de NodeJS y se pueden utilizar de manera nativa.

Es decir, podemos utilizar template literals, arrow functions, promesas, clases y objetos y todo lo nuevo que vaya apareciendo con los años sin necesidad de importar ningún tipo de librería externa.

En la creación de proyectos más complejos existen dos variables que nos pueden resultar especialmente útiles cuando necesitamos trabajar con rutas:

```
console.log(__dirname);  
console.log(__filename);
```

dirname

Nos devuelve la **ruta completa** al directorio donde estemos ejecutando la aplicación de NodeJS.

filename

Nos devuelve la ruta completa al fichero donde estemos trabajando.

Estas dos variables globales nos permiten interactuar con rutas más complejas independientemente del ordenador donde hayamos instalado nuestra aplicación e independientemente de la ruta seleccionada para su instalación.

Otro de los métodos importantes dentro de este espacio global es **require**. Como ya hemos visto en temas anteriores, este método nos permite incluir cualquier tipo de fichero o librería dentro del proyecto en el que estemos trabajando.

En el siguiente ejemplo incluimos la librería path para poder extraer el nombre del fichero a partir de su ruta genérica.

```
var path = require('path');  
console.log(path.basename(__filename));
```

Desde la propia documentación de NodeJS Podemos recuperar el resto de los valores o métodos a los cuales puedes acceder desde este espacio global.

2.3. Control del proceso de una aplicación

Existen muchas ocasiones en las que, dentro de nuestro propio código, vamos a necesitar interactuar con la ejecución actual. Podemos llevar a cabo acciones que nos permitan obtener información sobre el proceso activo, sobre el entorno donde se está ejecutando, tener una comunicación activa con el entorno o simplemente dar por terminado el proceso.

Para todo ello podemos utilizar el módulo `process` y todos los métodos asociados.

Una de las funcionalidades básicas a las cuales podemos acceder gracias a este módulo es a los diferentes valores que se incluyen desde la línea de comandos al lanzar la aplicación con NodeJS. Estos valores se almacenan en una propiedad llamada **`argv` (argument variables)**.

Podemos hacer la prueba lanzando el siguiente script:

```
console.log(process.argv);  
// El resultado es algo parecido a esto:  
[  
  '/Users/mario/.nvm/versions/node/v14.17.4/bin/node',  
  '/Users/mario/Documents/nodejs/proyectoprueba/index.js'  
]
```

Recuperamos un array. La primera posición de ese array nos indica la ruta donde tenemos instalado NodeJS. La segunda posición del array incluye la ruta completa hasta el fichero que estamos ejecutando.

Lo interesante en este caso sería incluir más información en la ejecución de nuestro script. Podemos lanzarlo de la siguiente manera:

`node index --user Luis --saludo Hola`

Mediante una pequeña función que busque los elementos dentro de los argumentos del proceso podríamos acceder a sus valores:

```
function grab(flag) {
  const index = process.argv.indexOf(flag);
  return (index === -1) ? null : process.argv[index + 1];
}
const saludo = grab('--saludo');
const user = grab('--user')
if (!user || !saludo) {
  console.log("No funciona");
} else {
  console.log(`El usuario es ${user} y tu saludo es ${saludo}`)
}
```

Mediante este tipo de acciones podríamos definir todos aquellos parámetros de entrada necesarios para arrancar nuestras aplicaciones como por ejemplo el puerto o algún directorio donde vayamos a contener la configuración inicial.

Otras dos funcionalidades importantes dentro del módulo process son las que nos permiten interactuar con *Standard Input* y *Standard Output* para manejar la entrada de datos y la salida de datos de nuestras aplicaciones.

Podemos escribir cualquier tipo de texto dentro de la salida estándar de nuestra aplicación (por defecto sería el terminal) mediante el siguiente código:

```
process.stdout.write('Escribiendo ');
process.stdout.write('en el terminal\n\n');
```

No funciona igual que el método **console.log** ya que cualquier salto de línea que necesitemos incluir lo debemos hacer de manera implícita dentro de la ejecución.

Un aspecto importante que podemos tratar al trabajar con la **entrada estándar (por teclado)** es el manejo que hace de los datos. Si queremos interactuar con los datos que introduce el usuario, podemos hacerlo a través de **eventos**.

Esta es una práctica muy común que veremos en varios de los módulos propios de

NodeJS, así como en librerías externas.

En el siguiente código vamos lanzando una serie de preguntas a partir de un array y mediante los diferentes eventos, tanto de process como de la entrada estándar recogemos la información y vamos almacenando las respuestas en otro array totalmente diferente para mostrarlas cuando termine la ejecución.

```
const preguntas = [
  "¿Cual es tu nombre?",
  "¿Cual es tu edad?",
  "¿Cual es tu lenguaje de programación preferido?"
];
const respuestas = [];
function hacerPregunta(i) {
  process.stdout.write(`\n\n\n${preguntas[i]}`);
  process.stdout.write(" > ");
}
process.stdin.on('data', function (data) {
  respuestas.push(data.toString().trim());
  if (respuestas.length < preguntas.length) {
    hacerPregunta(respuestas.length);
  } else {
    process.exit();
  }
});
// Evento que se lanza cuando terminamos nuestra app
process.on('exit', function () {
  process.stdout.write('\n\n\n');
  process.stdout.write(`Nombre: ${respuestas[0]} Edad: ${respuestas[1]}.
  Lenguaje: ${respuestas[2]}`);
  process.stdout.write('\n\n\n');
})
hacerPregunta(0);
```

A través del método **on** nos podemos **suscribir** a los diferentes eventos disponibles para cada uno de los objetos con los que estemos trabajando. En este caso estamos trabajando con el evento **data** para la entrada estándar y así poder recuperar la información introducida por teclado, y con el evento **exit** sobre el objeto process para poder determinar cuándo se termina nuestra aplicación (al ejecutar **process.exit()**).

Esto es una aproximación muy artesanal a lo que sería una aplicación gestionada a

Tema 2. Node Core

través de la consola. Existen librerías más avanzadas que resuelven esta misma funcionalidad de manera más cómoda para el desarrollador, dentro de la sección «A fondo» destacamos Vorpal como una de ellas.

2.4. Funciones de timing

Dentro de un espacio global disponemos de una serie de funciones, con prácticamente la misma funcionalidad que en Javascript, que nos permiten trabajar de manera asíncrona con diferentes aspectos de nuestras aplicaciones.

```
const tiempo = 3000;
console.log('Esperando...');
setTimeout(() => {
  console.log('Termina');
}, tiempo);
```

Durante los 3 segundos de espera el resto de la función se mantendría en ejecución. Una vez termine ese tiempo volvería el control de la aplicación al callback de la función **setTimeout**.

Con la función **setInterval** lanzamos una ejecución cada cierto tiempo. Y asociada a ésta, la función **clearInterval** detiene un intervalo abierto:

```
let tiempoActual = 0;
const tiempoEspera = 500;
const interval = setInterval(() => {
  console.log(`Se muestra cada medio segundo (${tiempoActual})`);
  tiempoActual += tiempoEspera;
  if (tiempoActual > 5000) {
    clearInterval(interval);
  }
}, tiempoEspera);
```

2.5. Usos del módulo readline

Mediante este módulo podemos crear una estructura por encima de la entrada estándar y de la salida estándar para poder acceder con métodos más cómodos y a través de otra serie de eventos.

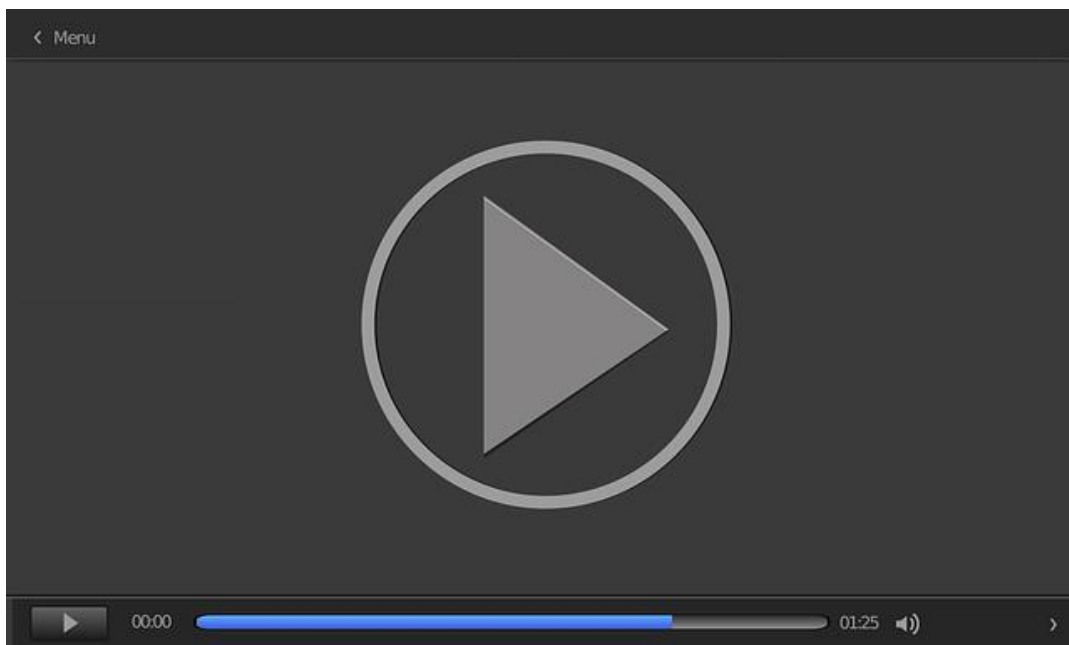
El siguiente código muestra un ejemplo de cómo podríamos lanzar preguntas al usuario e ir almacenándolas en una estructura de array:

```
const readline = require('readline');
// Creamos la interfaz sobre la entrada y la salida estándar
const rl = readline.createInterface(process.stdin, process.stdout);
const persona = {
  nombre: '',
  frases: []
};
// Mediante el método question lanzamos una pregunta a la salida estándar
// y esperamos su respuesta en la entrada
rl.question("¿Cuál es tu nombre? ", function (answer) {
  persona.nombre = answer;
  // setPrompt y prompt nos permiten preparar lo que vamos a escribir
  // y escribirlo, respectivamente
  rl.setPrompt(`¿Qué dice ${persona.nombre}? `);
  rl.prompt();
  // El evento line es aquel que nos avisa cuando se ha escrito algo
  // en la entrada estándar
  rl.on('line', function (saying) {
    if (saying.toLowerCase().trim() === 'exit') {
      // Si se escribe la palabra exit, se cierra el interfaz
      rl.close();
    } else {
      persona.frases.push(saying.trim());
      rl.setPrompt(`¿Qué más dice ${persona.nombre}? ('exit' para salir) `);
      rl.prompt();
    }
  });
});
// El evento close es aquel que se lanza cuando el interfaz es cerrado
rl.on('close', function () {
  console.log(`${persona.nombre} dice ${persona.frases}`);
  process.exit();
});
```

Tema 2. Node Core

Este tipo de acciones sencillas nos permiten conocer cómo se manejan muchas de las funciones que nos vamos a encontrar durante nuestro camino a la hora de crear servidores mucho más complejos. El manejo de eventos nos permite controlar acciones asíncronas y así no bloquear las diferentes ejecuciones dentro de nuestra aplicación.

Vamos a explorar la creación y ejecución de este pequeño ejemplo mediante la ***Creación de una interfaz basada en las librerías básicas de NodeJS.***



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=7f9221fe-2672-4279-8f0c-ae2e012e1e67>

2.6. Gestión de eventos con EventEmitter

EventEmitter es una de esas herramientas poderosas que tenemos para crear interacciones entre los diferentes ficheros de nuestra aplicación. Mediante este tipo de objetos podemos generar eventos para poder lanzarlos en cualquier momento.

Podríamos generar nuestros propios eventos y capturarlos de la siguiente manera:

```
const events = require('events');
const emitter = new events.EventEmitter();
// Escuchamos sobre un evento propio
emitter.on('customEvent', (mensaje, estado) => {
  console.log(`${estado}: ${mensaje}`);
})
// Emitimos un evento
emitter.emit('customEvent', 'Este es el mensaje', 200);
```

En la mayoría de los objetos con los que vamos a trabajar veremos que ya contienen sus propios eventos y podemos interactuar con ellos. Pero no debemos perder de vista la posibilidad de crear los nuestros propios en caso de que necesitemos definir una interacción como esta.

Podríamos incluso complicarlo haciendo que un objeto propio herede todas las características de EventEmitter y se convierta en un emisor de eventos.

```
const events = require('events');
class Persona extends events.EventEmitter {
  constructor(nombre, apellidos) {
    super();
    this.nombre = nombre;
    this.apellidos = apellidos;
  }
  habla(msg) {
    this.emit('speak', msg);
  }
}
const pers = new Persona('Laura', 'García');
pers.on('speak', (msg) => {
  console.log(`${pers.nombre} ${pers.apellidos} dice: ${msg}`);
});
```

Tema 2. Node Core

```
});  
pers.habla('Hola a todos');
```

2.7. Ejecución de procesos externos

Dentro de nuestras aplicaciones de NodeJS podemos ejecutar procesos externos. Es decir, podemos lanzar cualquier tipo de comando que ejecutaríamos de manera habitual dentro de nuestro terminal.

De esta manera, podemos combinar los diferentes algoritmos dentro de nuestras aplicaciones NodeJS con otra serie de ejecuciones de aplicaciones totalmente diferentes escritas incluso en otros lenguajes.

Podemos hacer un ejemplo muy sencillo lanzando el comando que nos liste todos los ficheros y directorios de una carpeta en concreto de nuestros ordenadores.

```
const exec = require('child_process').exec;
// Para usuarios windows sería el comando dir
exec('ls', (err, stdout) => {
  if (err) {
    throw err;
  }
  console.log(stdout);
});
```

En caso de que el comando externo ejecutase cualquier tipo de aplicación que implique mucho más tiempo de ejecución podríamos utilizar la función **spawn** y mediante eventos gestionaríamos la respuesta.

Para poder probarlo sobre un ejemplo creamos el fichero **cuentaNumeros.js** con el siguiente código:

```
let num = 0;
while (num < 10000) {
  console.log(`Número actual: ${num}`);
  num++;
}
```

A partir de aquí podemos lanzar la ejecución de este fichero desde otro script

NodeJS:

```
const spawn = require("child_process").spawn;
// creamos la ejecución que vamos a realizar
// Primero el comando a lanzar y después los parámetros
const cp = spawn("node", ["cuentaNumeros"]);
cp.stdout.on("data", function (data) {
  console.log(`STDOUT: ${data.toString()}`);
});
cp.on('close', function () {
  console.log("Se termina el Child Process");
  process.exit();
});
```

3.1. Introducción y objetivos

Si venimos acostumbrados al desarrollo con Javascript nativo hay muchas cosas que se separan cuando hablamos de la creación de aplicaciones con NodeJS, una de ellas es el trabajo que llevamos a cabo con el módulo **File System**.

Uno de los focos que debemos tener siempre presentes es que todo el trabajo que hagamos en nuestras aplicaciones sobre NodeJS se estará realizando dentro de un servidor y por tanto tenemos a nuestro alcance muchas características que no podemos alcanzar dentro del navegador, ya sea por permisos o por limitaciones el mismo.

Es el caso de la librería con la que vamos a trabajar en este tema, File System. Nos proporciona una serie de métodos que nos permiten interactuar con todo el sistema de directorios y ficheros del servidor donde vamos a trabajar.

El objetivo principal de este tema por tanto es **conocer cómo manejar las diferentes herramientas** que nos permitan crear, editar, borrar... tanto directorios como ficheros desde nuestros scripts de NodeJS.

3.2. Lectura de directorios

Como veremos a partir de ahora casi todos los métodos que vamos a conocer de File System trabajan de la misma manera y disponen de 3 alternativas diferentes para poder ser ejecutados: de forma síncrona, de forma asíncrona o a través de promesas.

Aprovecharemos este primer método para analizar en detalle cada una de estas sintaxis y así ser capaces a partir de aquí de utilizar cualquiera de los métodos del módulo.

Lectura de directorios de forma síncrona

```
const fs = require('fs');  
const files = fs.readdirSync('.');  
console.log(files);
```

El primer paso es importar la librería **fs**. En este caso, como forma parte de la instalación de NodeJS no debemos instalarla, se utiliza directamente

El método que estamos ejecutando es **readdirSync** el cual recibe por parámetro la ruta relativa al directorio con el cual vamos a trabajar. De manera síncrona (detiene la ejecución de la aplicación) nos devuelve un array con las rutas a los diferentes ficheros y directorios contenidos dentro de la carpeta donde estamos trabajando.

Esta forma de trabajar es la óptima siempre y cuando necesitemos que las acciones que están escritas después de la llamada al método se ejecuten con total seguridad después de la lectura de directorios.

Lectura de directorios de forma asíncrona

```
const fs = require('fs');
fs.readdir('.', (err, files) => {
  if (err) {
    return console.log(err);
  }
  console.log(files);
})
```

La ejecución del método **readdir** cuenta con los mismos parámetros que el método anterior y la única diferencia la vemos en el último parámetro del método. Como último parámetro debemos pasar una función que actuará como **callback**. Es un método que se ejecutará automáticamente cuando se recojan todos ficheros y directorios.

Esta forma de ejecutar el método es mucho más eficiente si necesitamos una ejecución que no bloquee el resto de las sentencias que tengamos en nuestro código.

Lectura de directorios con promesas

```
const fs = require('fs/promises');
fs.readdir('.')
  .then(files => console.log(files))
  .catch(err => console.log(err));
```

Como en las ejecuciones anteriores los parámetros son los mismos. La única diferencia con esta opción es que el método devuelve **un objeto de tipo Promise**, por lo que podemos acceder al resultado a través de la llamada a los métodos **then**, **catch** o a través del uso de **async-await**.

Para el resto de los métodos que veamos dentro de este tema no especificaremos tanto detalle ya que los métodos de ejecución son los mismos que hemos visto para **readdir**.

Tema 3. File System

3.3. Lectura y escritura de ficheros

Podemos leer el contenido de un fichero a través del método **readFile**. Se le pasa como primer parámetro la ruta donde se encuentra el fichero y como segundo parámetro la codificación en la que se encuentran los datos dentro del propio fichero (normalmente utf-8).

Para todas las pruebas sobre esta serie de métodos se ha creado un directorio llamado **ficheros** y dentro de dicho directorio hemos colocado diferentes ficheros con texto para poder realizar las pruebas.

```
const fs = require('fs');
const fsPromises = require('fs/promises');
const content = fs.readFileSync('./ficheros/ficheroConTexto.md', 'utf-8');
console.log(content);
fs.readFile('./ficheros/ficheroConTexto.md', 'utf-8', (err, content) => {
  if (err) return console.log(err);
  console.log(content)
});
fsPromises.readFile('./ficheros/ficheroConTexto.md', 'utf-8')
  .then(content => console.log(content))
  .catch(err => console.log(err));
```

El resultado obtenido a través de las diferentes fórmulas de ejecución es siempre el mismo. En este caso obtenemos un error en caso de que no podamos leer el fichero y el contenido de este en caso contrario.

Para la ejecución síncrona o para la versión de promesas con **async-await** **podemos recuperar el error incluyendo la llamada al método dentro de un bloque try-catch.**

Del mismo modo disponemos del método **appendFile** para poder agregar contenido dentro de un fichero. Los parámetros que debemos incluir son, la ruta del fichero a modificar y el contenido que necesitemos incluir.

Si el fichero donde vamos a hacer la inserción de información no existe, la primera acción que se llevará a cabo es la creación de este.

```
const fs = require('fs');
const fsPromises = require('fs/promises');
fs.appendFileSync('./ficheros/pruebaEscritura.txt', 'Primera línea\n');
fs.appendFile('./ficheros/pruebaEscritura.txt', 'Segunda línea\n', () => {
  console.log('Fichero escrito');
});
fsPromises.appendFile('./ficheros/pruebaEscritura.txt', 'Tercera línea\n')
  .then(() => console.log('Fichero escrito'))
  .catch(err => console.log(err));
```

En todas las ejecuciones que realicemos es muy importante tener en cuenta los errores ya que provocan que nuestras aplicaciones acaben de manera abrupta si no los controlamos.

3.4. Operaciones más comunes sobre ficheros

Vamos a intentar desgranar alguno de los métodos más interesantes para el manejo de directorios y ficheros dentro de nuestras aplicaciones NodeJS.

Existen muchos más métodos de los que vamos a tratar en este tema, para ampliar conocimiento o simplemente tener una documentación donde poder revisar código, se puede consultar el siguiente enlace donde se encuentran todos los métodos de File System.

Creación de directorios

Disponemos de la función **mkdir** para poder generar carpetas en el punto del servidor donde estemos trabajando.

```
const fs = require('fs');
fs.mkdir('bin', (err) => {
  if (err) return console.log('Directorio no creado');
  console.log('Directorio creado')
});
```

Esta función tiene las mismas características que las otras, es decir, dispone de versión síncrona, asíncrona y mediante el uso de promesas.

El método **mkdir** nos devuelve error si es imposible generar el directorio y en caso de que el directorio ya se encuentre creado.

Comprobación de la existencia de ficheros o directorios

Para solucionar el problema anterior disponemos del método **existsSync**. En este caso no disponemos de versión asíncrona ya que este método está diseñado para poder ser usado dentro de un condicional, donde no podemos ejecutar acciones

asíncronas.

```
const fs = require('fs');
if (fs.existsSync('./bin')) {
  console.log('El directorio ya existe');
} else {
  fs.mkdirSync('./bin');
}
```

Estas comprobaciones son importantes para evitar errores en el momento de la creación de nuevas estructuras dentro de nuestros directorios. Siempre es una muy buena práctica intentar evitar errores de este estilo para que el rendimiento de nuestra aplicación no se vea afectado.

Mover/Renombrar ficheros

Una de las prácticas más habituales cuando estamos trabajando con los ficheros del sistema es cómo los cambiamos de directorio o cómo modificamos su nombre en función de aspectos que sucedan dentro de nuestras aplicaciones.

Para llevar a cabo estas dos acciones lo podemos hacer a través del método **rename**.

Vemos un ejemplo de cómo podríamos modificar el nombre de un fichero concreto:

```
const fs = require('fs/promises');
(async () => {
  try {
    await fs.rename('./ficheros/pruebaEscritura.txt', './ficheros/config.txt');
  } catch (err) {
    console.log(err.message);
  }
})();
```

En este ejemplo se ha usado la función a partir de su versión con promesas. Para ello hemos definido una función autoejecutable dentro de la cual hemos lanzado la

función. Esto lo hacemos para poder disponer de un ámbito donde poder colocar el decorador **async** y así no depender de una función extra.

La función **rename** recibe dos parámetros, el primero es la ruta del fichero original y el segundo es la ruta donde vamos a colocar el fichero después de renombrarlo.

De esta manera si en vez de modificar únicamente el nombre del fichero, modificamos también la ruta en donde lo vamos a colocar, en vez de modificar únicamente el nombre estaríamos moviendo ese fichero de una ruta a otra diferente.

Lo vemos en el siguiente ejemplo:

```
const fs = require('fs/promises');
fs.rename('./ficheros/config.txt', './bin/config.txt')
.then(() => console.log('El fichero se ha movido'))
.catch(err => console.log(err.message));
```

(la sintaxis a utilizar es independiente del resultado obtenido. En los diferentes ejemplos estamos intercalando cada uno de los métodos para poder practicar con todas las posibilidades).

En esta acción el fichero se moverá al nuevo directorio siempre y cuando el fichero exista, tenga permisos para poder trasladarse y exista el directorio de destino. En caso contrario, la llamada al método nos devolverá error.

Estas acciones las podemos aplicar de igual manera para el renombrado de directorios.

```
const fs = require('fs');
try {
  fs.renameSync('./ficheros', './files');
} catch (err) {
  console.log(err.message);
}
```

Borrado de ficheros/directorios

Para llevar a cabo el borrado de cualquier fichero podemos utilizar el método **unlink** en sus diferentes versiones.

```
const fs = require('fs');
try {
  fs.unlinkSync('./bin/config.txt');
} catch (err) {
  console.log(err.message);
}
```

En caso de que el fichero que vayamos a borrar no exista dentro de la ruta específica el método nos devuelve error. Es por eso por lo que resulta muy importante capturar esa serie de errores para poder actuar en consecuencia.

Para poder borrar directorios completos usamos el método **rmdir**.

```
const fs = require('fs');
fs.rmdir('./files', (err) => {
  if (err) return console.log(err.message);
  console.log('Directorio borrado');
})
```

Para llevar a cabo esta acción debemos primero borrar todos los ficheros contenidos dentro del directorio que estamos intentando borrar. Para ello lo podemos hacer de la siguiente manera, mezclando alguno de los métodos anteriores:

```
const fs = require('fs');
fs.readdirSync('./files').forEach(file => {
  fs.unlinkSync(`./files/${file}`);
});
```

3.5. Manejo de Streams

Mediante los métodos que tenemos disponibles para streams vamos a poder controlar el flujo de información tanto de lectura como de escritura mediante una transmisión continua.

Este tipo de acciones vienen a solucionar los problemas que podríamos encontrar en caso de que necesitemos hacer lecturas sobre ficheros extremadamente grandes o escrituras de datos continuadas en el tiempo.

Podemos trabajar con datos binarios (en caso de archivos de imagen o audio) o con texto codificado en UTF-8 para guardar o rescatar cualquier tipo de información.

El trabajo con streams además, nos permite gestionar toda la interacción a través de eventos y por lo tanto no bloquear el resto de acciones que estemos ejecutando en nuestra aplicación.

Vemos un ejemplo de lectura de fichero a través de un stream.

```
const fs = require('fs');
// Este tipo de objetos se recomiendan siempre y cuando el fichero
// sea de un gran tamaño
const stream = fs.createReadStream('./files/ficheroGigante.txt', 'utf-8');
let data = '';
// Esta función solo se ejecuta la primera vez
// que se recibe el evento data
stream.once('data', () => {
  console.log('Comienza la lectura del fichero');
});
// Este evento se ejecuta siempre que recibamos el evento data
// incluso la primera vez
stream.on('data', (chunk) => {
  console.log('Recibido bloque: ' + chunk.length);
  data += chunk;
});
// Cuando se termina la lectura del fichero se lanza el evento end
stream.on('end', () => {
  console.log('Lectura finalizada: ' + data.length);
});
```

```
})
```

De esta manera no debemos esperar a que se termine de recuperar toda la información y podemos empezar a trabajar con ella mientras la vamos recibiendo.

Aparte de los streams de lectura tenemos también los de escritura, los cuales nos permiten tener un canal abierto para escribir de manera constante dentro de un fichero.

Del mismo modo que los anteriores, no bloquean el resto de las acciones y además nos evitan tener que estar constantemente haciendo referencia al fichero sobre el cual vamos a escribir.

Mediante el módulo **readline** de temas anteriores, podemos crear una pequeña interfaz donde todo lo que vayamos escribiendo se almacene dentro de un fichero.

```
const readline = require('readline');
const rl = readline.createInterface(process.stdin, process.stdout);
const fs = require("fs");
const stream = fs.createWriteStream('./files/frases.txt');
rl.question("Dime una frase ", function (answer) {
  stream.write(`${answer}\n`);
  rl.setPrompt(`Dime otra frase (exit para salir) `);
  rl.prompt();
  rl.on('line', function (saying) {
    if (saying.toLowerCase().trim() === 'exit') {
      stream.close();
      rl.close();
    } else {
      stream.write(`${saying.trim()}\n`);
      rl.prompt();
    }
  });
});
rl.on('close', function () {
  console.log(`Revisa el fichero para recuperar las frases`);
  process.exit();
});
```

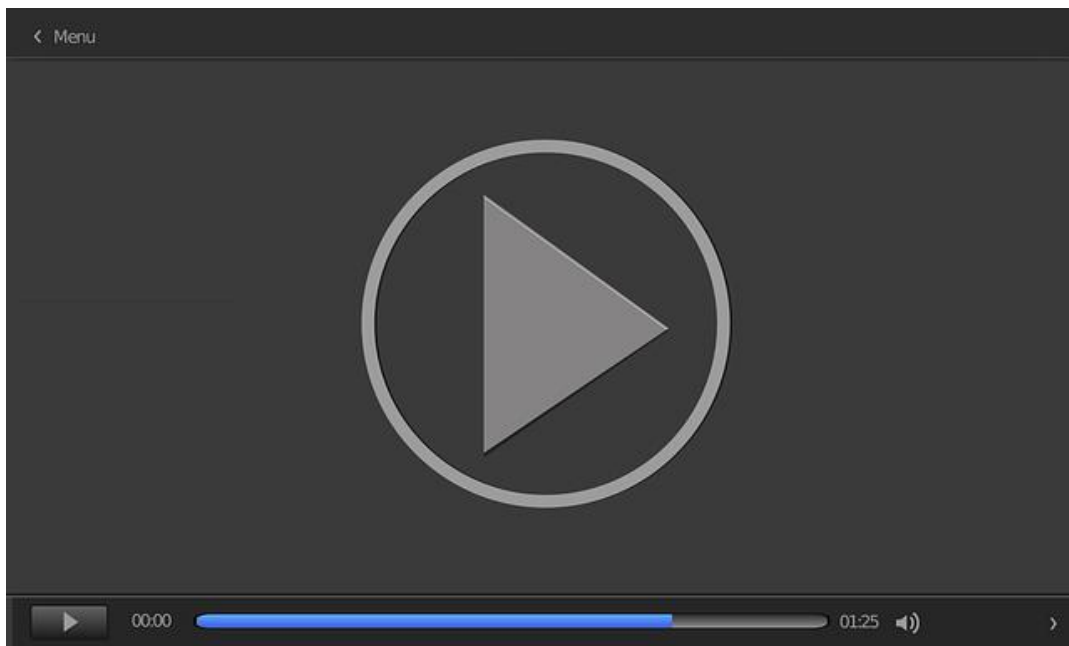
Como vemos en el ejemplo anterior, una vez generamos el stream de escritura, a

Tema 3. File System

partir de ese momento, siempre que utilicemos el método **write** estaremos escribiendo contenido nuevo dentro del fichero.

Es muy importante que **cerremos el stream de escritura** a través del método `close` para no obtener resultados inesperados cuando queramos acceder al fichero mediante cualquier otra técnica.

En el vídeo **Lectura de grandes ficheros** se muestra cómo en ocasiones nos enfrentamos a la lectura y gestión de ficheros de gran tamaño. Las funciones normales de NodeJS nos pueden devolver algunos problemas en este tipo de tareas, es por eso por lo que tenemos a nuestra disposición el uso de los **streams de datos**.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=893179ae-5650-4035-975a-ae2f010ad2ff>

4.1. Introducción y objetivos

Para los desarrolladores web, uno de los puntos clave es la comunicación que podemos llevar a cabo entre diferentes máquinas o servidores sirviéndonos de ciertos protocolos a través de internet.

Uno de los conceptos importantes que vamos a tratar es cómo, a partir de herramientas que nos proporciona NodeJS podemos crear aplicaciones web completas que sean capaces de **recibir peticiones y generar una respuesta coherente a dicha petición.**

Todo ello lo haremos apoyándonos en el protocolo HTTP que es uno de los más utilizados cuando navegamos por internet. A todos nos suenan estas 4 letras ya que las solemos colocar delante de la página web a la que necesitamos navegar cuando nos enfrentamos a cualquier navegador.

En este tema analizamos los conceptos básicos de conexión mediante dicho protocolo y las herramientas más sencillas para generar nuestros propios servidores. El trabajo que realizaremos aquí será «muy artesanal» pero nos servirá para poner en valor librerías externas como ExpressJS que facilitan mucho la consecución de objetivos en este aspecto.

El módulo HTTP se divide en dos diferentes, http y https, dependiendo si en nuestra comunicación vamos a encriptar los datos a través de un certificado o no. En la mayoría de los ejemplos utilizaremos la versión **http** para evitar problemas a la hora de encriptar los datos con los que vamos a trabajar.

4.2. Creación básica de peticiones

Hay gran cantidad de datos usados en nuestras aplicaciones que se encuentran fuera de nuestros servidores o fuera del ordenador donde estamos alojando nuestras diferentes aplicaciones. En todo momento necesitamos herramientas que nos permitan acceder a ellos para poder ser consumidos en el espacio donde los necesitemos.

Para realizar esta labor NodeJS cuenta con el método **request** el cual es capaz de lanzar cualquier tipo de petición a través de internet y recuperar la respuesta utilizando diferentes eventos para así no bloquear la ejecución del resto de acciones de nuestra aplicación.

En el ejemplo siguiente vamos a realizar una descarga de datos desde la Wikipedia para poder ver cómo se utilizan los diferentes eventos y cómo podemos acceder a información que en principio no tenemos a nuestro alcance directo.

```
const https = require('https');
const fs = require('fs');
// Detalles para la petición
const options = {
  hostname: "es.wikipedia.org",
  port: 443,
  path: "/wiki/Steve_Jobs",
  method: "GET"
};
// Realizamos la petición
// Obtenemos en el callback de la función la respuesta, que ya implementa
un stream
const req = https.request(options, function (res) {
  let responseBody = "";
  console.log("Empieza la respuesta del servidor");
  console.log(`Server Status: ${res.statusCode}`);
  console.log("Response Headers: %j", res.headers);
  // Modificamos el encoding para que los datos binarios descargados estén
  correctos
  res.setEncoding("UTF-8");
  // La primera vez que recibimos datos, los mostramos
```

```
res.once("data", function (chunk) {
  console.log(chunk);
});
// Todas las veces que recibimos datos, los agregamos al total.
res.on("data", function (chunk) {
  console.log(`-- chunk -- ${chunk.length}`);
  responseBody += chunk;
});
// Cuando termina la descarga, escribimos lo descargado al fichero.
res.on("end", function () {
  fs.writeFile("Steve_Jobs.html", responseBody, function (err) {
    if (err) {
      throw err;
    }
    console.log("Se ha descargado la web");
  });
});
req.on("error", function (err) {
  console.log(`Problema en la petición: ${err.message}`);
});
req.end();
```

Como podemos observar en el código, se definen una serie de opciones (variable **options**) que son las que van a dar forma a la petición que vamos a realizar contra el servidor de la Wikipedia. Se especifica la **url**, el **puerto**, la **ruta que vamos a descargar** y sobre todo el método con el que se va a realizar dicha petición, en este caso **GET**.

El método **request** es el encargado de crear la petición y a través de la llamada a **res.end()** al final del script lanzamos dicha petición.

El manejo de los datos según los vamos recibiendo es muy parecido a otras técnicas que ya hemos visto anteriormente ya que a través de eventos podemos ir recuperando toda la información y, de la misma manera, cuando termine la descarga de datos podemos actuar en consecuencia. En este ejemplo estamos generando un fichero con extensión **html** donde estamos colocando todos los datos descargados de la Wikipedia.

Si abrimos ese documento generado dentro de cualquier navegador deberíamos observar todos los datos de la página requerida sin contar con las hojas de estilo o los scripts asociados.

Esta es la forma más básica de extraer información a través de internet. Existen una serie de librerías que mejoran mucho este trabajo y que se suelen usar cuando ya tenemos un manejo mucho más avanzado, como pueden ser **axios** o **got**.

4.3. Creación de servidores básicos

Cuando hablamos de servidores estamos hablando de **aplicaciones desarrolladas con diferentes tecnologías que reciben cualquier tipo de petición web desde un cliente y se encargan de gestionar la respuesta que volverá a dicho cliente.**

Resumiendo, nuestro trabajo cuando desarrollemos este tipo de servidores será el de elegir un puerto de la máquina en la que estemos trabajando, donde estaremos escuchando las diferentes peticiones entrantes. Una vez analizada la petición, creamos la respuesta y **siempre estamos «obligados» a enviarla de vuelta al cliente.**

En el siguiente ejemplo vamos a ver cómo podemos generar este tipo de servidores y sobre todo a definir las limitaciones que nos marca el uso único de NodeJS frente a las diferentes librerías de creación de aplicaciones como podrían ser Express o Koa.

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.end('Hola Servidor');
});
server.listen(3000);
```

La función **createServer** recibe una función por parámetro que será la que se ejecute cuando recibamos algún tipo de petición sobre el puerto **3000** que es el que hemos especificado en la llamada a la función **listen**.

Dentro de la función manejadora, recibimos dos parámetros muy importantes:

- **req:** hace referencia a la petición que entra dentro de nuestro servidor. Dentro de esta variable disponemos información sobre las cabeceras (headers), el método con el cual estamos realizando la petición (method), la url sobre la cual se está realizando dicha petición (url) e incluso los posibles datos que acompañen la petición entrante (body).

- **res:** es el objeto mediante el cual vamos a poder generar la respuesta que enviaremos de vuelta al cliente. Es imprescindible que para cada petición que llegue a nuestro servidor siempre respondamos con datos. En el ejemplo anterior estamos enviando un texto sencillo en la ejecución del método **end** y dando por tanto por cerrada la comunicación con el cliente.

Si sobre un servidor básico queremos devolver cualquier tipo de HTML podemos hacerlo modificando la cabecera **Content-Type** de la respuesta:

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end(`
<html>
<head>
<title>Página de ejemplo</title>
</head>
<body>
<h1>Servidor sencillo</h1>
</body>
</html>
`);
});
server.listen(3000);
```

(después de cada modificación que hagamos sobre nuestros servidores tenemos que parar la aplicación y volver a arrancarla).

4.4. Respondiendo a peticiones POST

Las peticiones de tipo GET son las más habituales cuando trabajamos intercambiando datos a través de internet. Pero, como veremos cuando nos enfrentemos a la creación de un API Rest, no son las únicas.

Por ejemplo, los formularios HTML pueden enviar dentro de una petición POST todos los datos incluidos dentro de dicho formulario. Vamos a ver en un ejemplo cómo lo podemos gestionar a través de un servidor simple de NodeJS.

```
const http = require('http');
const fs = require('fs');
const server = http.createServer((req, res) => {
  // Decidimos si la petición es de tipo GET o POST
  if (req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    // Leemos un fichero que contendrá un formulario HTML
    // El stream de datos lo "enganchamos" a la respuesta
    fs.createReadStream('./public/formulario.html', 'utf-8').pipe(res);
  } else if (req.method === 'POST') {
    // recogemos todos los datos del formulario
    let body = '';
    req.on('data', (chunk) => {
      body += chunk;
    });
    req.on('end', () => {
      // recogemos los datos y los transformamos a string
      res.end(JSON.stringify(body));
    })
  }
});
server.listen(3000);
```

El fichero **formulario.html** que hemos utilizado para este ejemplo podría ser algo parecido al siguiente código:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
```



```
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
<form action="/" method="POST">
<div>
<label>Usuario</label>
<input type="text" name="usuario">
</div>
<div>
<label>Password</label>
<input type="password" name="password">
</div>
<input type="submit" value="Enviar">
</form>
</body>
</html>
```

De esta manera, en función del método utilizado, estamos devolviendo diferentes datos. De igual manera que los ejemplos anteriores, esta forma de trabajo es bastante poco eficiente de cara a la creación de aplicaciones más grandes, pero sí nos sirve para poder analizar los conceptos más básicos de la comunicación con servidores.

4.5. Respondiendo con JSON

Cuando sigamos avanzando en nuestro camino como desarrolladores veremos que el trabajo con datos en formato JSON se vuelve vital para nosotros ya que es el medio habitual de comunicación entre nuestras aplicaciones del Front y del Back.

En estos servidores sencillos también podemos ver cómo responder a través de datos en formato JSON:

```
const http = require('http');
const data = require('./data/productos.json');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(data));
});
server.listen(3000);
```

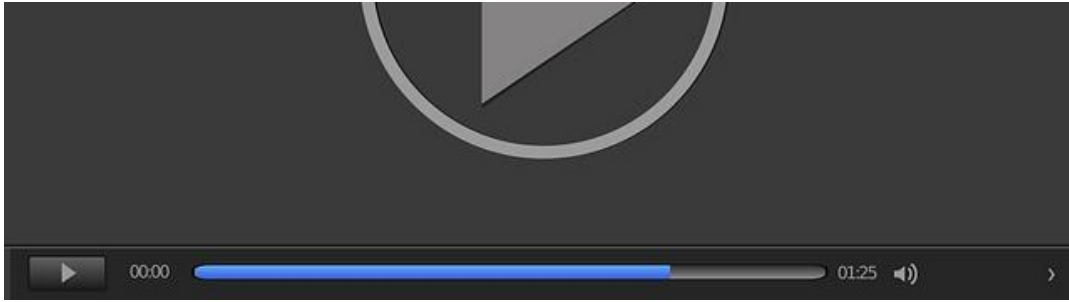
El primer paso es informar al navegador que el tipo de datos que va a recibir son json a través de la cabecera **Content-Type**. A partir de ahí respondemos transformando en formato string los datos recuperados del fichero con extensión .json.

Este será nuestro punto de partida para aprender cómo mejorar estas estructuras mediante el uso de ExpressJS.

En el vídeo **Manejo del servidor creado para JSON** se explica cómo, utilizando las herramientas básicas de NodeJS, podemos crear servidores un poco más complejos que impliquen la creación y recuperación de recursos.



Tema 4. Http



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=c54d8c68-c563-4eae-960c-ae32010bf77d>

5.1. Introducción y objetivos

En temas anteriores hemos tratado la importancia que tiene la escalabilidad y la flexibilidad en los diferentes desarrollos que hacemos sobre nuestros servidores. Con las herramientas nativas que nos proporciona NodeJS es muy complicado alcanzar objetivos profesionales y a largo plazo y es por eso por lo que utilizamos librerías que nos ofrecen las acciones necesarias para poder superar estos retos.

ExpressJS es una librería de terceros que nos permite crear aplicaciones web complejas. Tiene una curva de aprendizaje bastante rápida y es por eso por lo que la encontramos en multitud de desarrollos modernos.

Todos los conceptos del tema anterior vamos a desarrollarlos en este mismo tema, pero utilizando la metodología de Express y viendo cómo nos puede ayudar a la hora de agilizar y estructurar nuestros desarrollos.

5.2. Características de Express

Vamos a intentar desgranar por qué ExpressJS es una librería que mejora mucho el trabajo de desarrollador cuando necesitamos crear aplicaciones web:

- ▶ Disponemos de **numerosas herramientas asociadas a Express que nos permiten generar una estructura de ficheros y directorios** que nos ayudan a organizar el código de nuestras aplicaciones. Esto es especialmente importante cuando nuestras aplicaciones alcanzan un tamaño considerable y cada vez se vuelve más complicado realizar modificaciones.
- ▶ Se encarga de **gestionar por nosotros todas las peticiones entrantes al servidor** evitándonos el código más básico y dejando al desarrollador las tareas más orientadas a negocio. Disponemos de multitud de herramientas para la creación de rutas dentro de un servidor.
- ▶ Dispone de soporte para poder generar páginas web a través de los principales sistemas de gestión de plantillas. En nuestro caso analizaremos dos de los más importantes, **pug** y **hbs** (handlebars)
- ▶ Nos permite **interactuar con cualquier tipo de base de datos**. Tenemos multitud de drivers y librerías para poder conectar con las bases de datos más utilizadas del mercado. Además, existen numerosas herramientas que nos facilitan esta interacción sin necesidad de estar pensando en profundidad en cómo funcionan las diferentes bases de datos.
- ▶ Creando aplicaciones con Express también contamos con la posibilidad de generar de un modo muy sencillo pruebas unitarias que nos permitan asegurar la fiabilidad a lo largo del tiempo de todo el código que escribimos.

5.3. Arquitectura de aplicación Express

Una de las características más importantes a la hora de crear aplicaciones web con ExpressJS es la flexibilidad que nos ofrece el framework. A diferencia de otro tipo de herramientas, con Express no necesitamos una serie de directorios pre-establecidos o una configuración de ficheros concreta, podemos adecuar la situación dependiendo de nuestras necesidades.

Esto tiene su parte positiva en cuanto a la libertad que nos ofrece de cara a organizar nuestros proyectos, pero también podríamos encontrarnos con la parte negativa en la que encontraríamos proyectos muy mal organizados y poco escalables.

Es por ello por lo que, para establecer una serie de normas a seguir, vamos a usar una herramienta que nos proporciona una arquitectura básica a partir de la cual nosotros vamos a poder hacer crecer nuestras aplicaciones.

Instalación de Express Generator

Mediante esta librería que instalaremos a nivel global dentro de nuestro ordenador vamos a poder generar siempre la misma estructura de aplicación como punto de partida. Esto nos permite centrarnos en qué va a hacer nuestra aplicación y menos en cómo lo vamos a poder llevar a cabo o cuál es la estructura básica.

Para poder instalar la librería, desde el terminal y desde cualquier directorio de nuestro ordenador lanzamos el siguiente comando:

npm install -g express-generator

De esta manera instalamos la herramienta de Express para poder ser utilizada desde cualquier punto de nuestro ordenador. Si necesitamos comprobar si se ha instalado correctamente, podemos lanzar el siguiente comando:

express --help

Si nos devuelve toda la información sobre la ayuda querrá decir que hemos completado la instalación de manera satisfactoria.

A partir de este momento ya podemos empezar a crear aplicaciones a través de este comando, lanzando desde el terminal sentencias como la siguiente:

express --view=pug --git miapp

El resultado de la ejecución debería ser el siguiente:

Change directory:

```
$ cd miapp
Install dependencies:
$ npm install
Run the app:
$ DEBUG=miapp:* npm start
```

Nos indica que se ha generado una estructura de ficheros y directorios para nuestra aplicación (posteriormente analizaremos los más importantes) y los pasos que debemos llevar a cabo. El primero indica que debemos entrar dentro del nuevo directorio creado y el segundo nos sirve para poder instalar todas las dependencias que necesita el nuevo proyecto para funcionar.

Además, con el flags **--view=pug** estamos dejando configurado qué motor de plantillas vamos a utilizar y con **--git** estamos forzando la generación del fichero **.gitignore** para poder trabajar de manera más cómoda con nuestro repositorio local.

Estructura del proyecto

Antes de ponernos a modificar cualquiera de los proyectos generados es interesante conocer cómo se organizan de cara a ser más eficientes a la hora de realizar

modificaciones sobre los mismos. Necesitamos saber qué papel desempeña cada bloque de código en esta estructura.

Vamos a analizar en detalle cada uno de los ficheros que conforman la aplicación que hemos creado con anterioridad para saber cómo manejarlos a futuro.

package.json

Se trata del fichero donde encontramos toda la configuración inicial de nuestro proyecto, así como las dependencias externas sobre librerías de terceros necesarias para el buen funcionamiento de la aplicación.

También podemos encontrar el apartado de **scripts** donde podemos generar nuestros propios comandos de ejecución y así controlar de una manera más personalizada nuestras aplicaciones.

/bin/www

Es el fichero con la configuración más básica del servidor. Es el punto donde lanzamos la creación del servidor y lo ponemos a escuchar, usando además uno de los métodos que ya hemos visto en temas anteriores:

```
var server = http.createServer(app);
```

En esta línea de código estamos delegando toda la gestión de rutas de nuestra aplicación al fichero **app.js** que analizaremos a continuación.

Quizá **www** no es el fichero con el que más vamos a interactuar en nuestro desarrollo, pero sí merece la pena saber dónde está y cuáles son sus funciones por si en algún momento se nos plantea la necesidad de modificar nuestro servidor al más bajo nivel.

El resto del código de este fichero hace referencia a eventos globales de nuestra aplicación, sobre todo errores.

app.js

Se trata del fichero principal dentro de nuestra aplicación y donde vamos a encontrar la configuración básica del proyecto, así como la gestión de las rutas que vamos a ir generando para nuestra aplicación.

Vamos a desgranar las partes más importantes de este fichero para ver qué implicación tiene en la creación de nuestra aplicación:

```
var createError = require('http-errors');
var express = require('express');
var path = require('path');
var cookieParser = require('cookie-parser');
var logger = require('morgan');
```

En primera instancia requerimos todas las librerías necesarias para el trabajo dentro de nuestro proyecto, con especial atención a **express** que es la que nos va a proporcionar todas las herramientas para poder hacer frente a las peticiones entrantes, generando respuestas coherentes para cada una de ellas.

El resto son librerías secundarias que nos ayudan a completar nuestro desarrollo manejando errores, el log o simplemente como apoyo para realizar tareas un poco más complejas.

```
var indexRouter = require('./routes/index');
var usersRouter = require('./routes/users');
```

Estas dos importaciones hacen referencia a dos ficheros contenidos dentro del directorio **routes**. Su uso no es obligatorio y se generan como ejemplo de organización para la posterior gestión de las rutas dentro de la aplicación. Más adelante veremos que una buena forma de organizar nuestros proyectos es que dicha carpeta routes se encargue de contener todos los ficheros que van a manejar rutas dentro de la aplicación.

A partir de aquí encontramos todas las sentencias de configuración de la aplicación:

```
var app = express();
```

Esta primera sentencia define la aplicación a partir de la cual vamos a trabajar. Dispondrá para nosotros todas las herramientas que nos van a facilitar la captura de las diferentes rutas. Es imprescindible para el trabajo que vamos a realizar a continuación.

```
// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'pug');
```

Dentro de estas dos líneas definimos dónde se encuentra el directorio con las vistas de nuestra aplicación (views) y el motor de plantillas que estamos utilizando, respectivamente. Más adelante en este curso entraremos en profundidad en cómo podemos generar vistas a partir de nuestras aplicaciones de Express.

```
app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));
```

Aquí estamos configurando opciones como el Sistema de *logging*, la posibilidad de enviar y recibir datos en formato JSON, la configuración de las cookies y la localización del directorio **public** donde vamos a almacenar todos los archivos estáticos de la aplicación.

El siguiente bloque **es uno de los más importantes** para el trabajo que vamos a llevar a cabo dentro de nuestras aplicaciones:

```
app.use('/', indexRouter);
app.use('/users', usersRouter);
```

Mediante estas dos líneas estamos determinando cómo se va a poder acceder a nuestra aplicación.

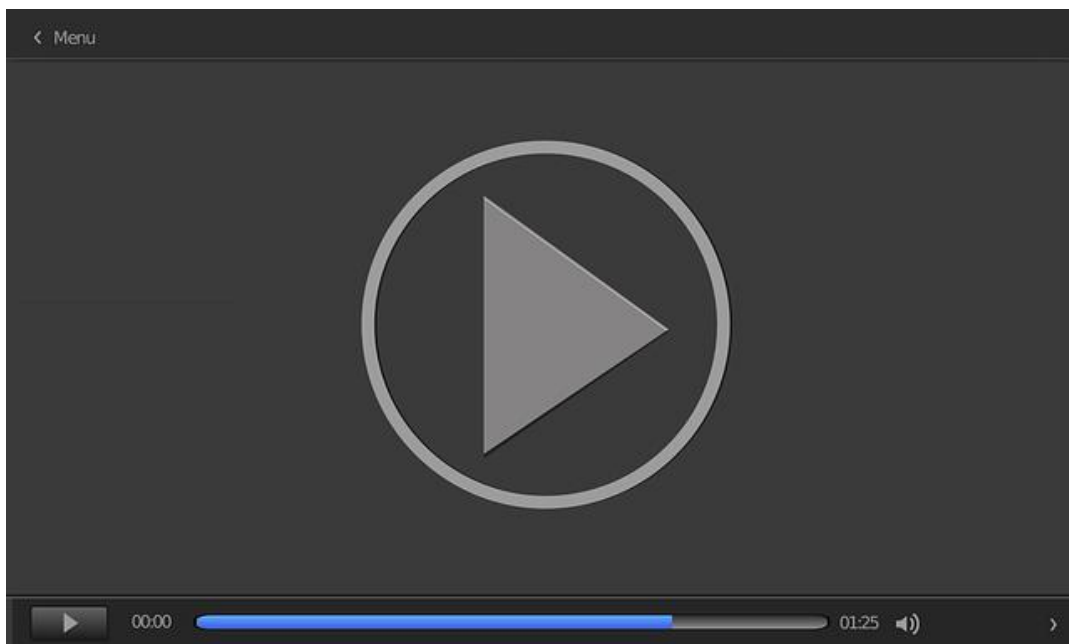
Tema 5. ExpressJS

Mediante el uso del método **use** estamos delegando cualquier tipo de petición que comience por la cadena colocada como primer parámetro a cada uno de los ficheros definidos con anterioridad dentro de **app.js**.

Es decir, si a mi servidor llega una petición con la url/**users**, independientemente del método con el que se haya lanzado (GET, POST, PUT...), la gestión de dicha petición se delega al fichero users.js representado por la variable **usersRouter**.

De esta manera y, para lograr una mejor organización dentro de nuestro código, no estamos centrando todos nuestros esfuerzos en un único fichero, si no que estamos delegando ciertas acciones y funcionalidades a otros ficheros donde poder trabajar de manera más cómoda.

Analizamos en el siguiente vídeo la **Estructura completa de una aplicación de Express** así como el uso de alguna librería imprescindible para nuestro trabajo.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=af0bf9bd-ef43-41cf-938f-ae350145b09e>

5.4. Gestión de rutas

Una de las partes más importantes de cara a gestionar nuestras aplicaciones con Express es cómo respondemos a las diferentes peticiones que llegan a nuestros servidores. Necesitamos **establecer una serie de patrones de trabajo para que podamos alcanzar objetivos de la manera más sencilla posible**.

Vamos a tomar como ejemplo el fichero **users.js** que se encuentra dentro de la carpeta **routes** de nuestro proyecto:

```
var express = require('express');
var router = express.Router();
/* GET users listing. */
router.get('/', function(req, res, next) {
  res.send('respond with a resource');
});
module.exports = router;
```

Para poder gestionar las rutas de nuestra aplicación sin disponer del objeto **app** es necesario recuperar el **router de express** y a partir del mismo **empezar a definir los métodos manejadores** que se van a ejecutar para definir una respuesta dependiendo del método utilizado y de la url.

El primer paso dentro de este tipo de ficheros es la recuperación de **router**.

```
var express = require('express');
var router = express.Router();
```

A partir de ahí ya podemos empezar a definir cualquier tipo de ruta.

No debemos olvidar exportarlo al final del fichero para que el resto de manejadores de rutas puedan incorporar sus propios manejadores y de esta manera completemos nuestra aplicación.

```
module.exports = router;
```

Entre la recuperación del router y su exportación definimos todas las posibles peticiones que vamos a poder recoger dentro de la aplicación.

En este punto vamos a dar un pequeño paso atrás importante que nos va a permitir fijar un concepto que nos servirá en un futuro para gestionar de manera óptima las rutas dentro de nuestros servidores.

Como vimos anteriormente, dentro de **app.js** teníamos estas dos líneas de código:

```
var usersRouter = require('./routes/users');
app.use('/', indexRouter);
```

Mediante estas sentencias de código estamos indicando que, cualquier petición que entre a nuestro servidor con la url/users va a ser gestionada dentro del fichero users.js.

Por esto, todos los manejadores de rutas que definamos dentro de dicho directorio ya cuentan con el prefijo/users y no hay que repetirlo en el momento de establecer el primer parámetro de la llamada a la función **get**.

Lo vemos en el ejemplo:

```
router.get('/', function (req, res, next) {
  res.send('respond with a resource');
});
```

La función que pasamos como segundo parámetro se ejecuta **siempre y cuando a mi aplicación llegue una petición con el método GET y la url/users**. Hay que tener mucho cuidado con esto porque nos puede llevar de manera muy fácil a cometer errores a la hora de definir ciertas rutas dentro de la aplicación.

Dentro del manejador de rutas, gracias a la ejecución del método **send** estamos generando la respuesta que volverá al cliente. Entraremos más en detalle sobre los tipos de respuestas que tenemos disponibles más adelante.

Una vez tengamos clara esta gestión de ficheros, podemos arrancar nuestra aplicación con el comando `npm start` o `node ./bin/www`. Esto pone en funcionamiento nuestro servidor y nos permitirá desde el navegador acceder a <http://localhost:3000/users>

Ponemos otro ejemplo:

- ▶ Queremos generar una ruta POST sobre la url <http://localhost:3000/users/create>

Los pasos que podríamos tener en cuenta para analizar dónde y cómo podemos implementar este punto de entrada a nuestra aplicación son los siguientes:

- ▶ Nos deberíamos hacer la pregunta: **¿tenemos algún fichero que se encargue de gestionar las rutas sobre la url /users?**
- ▶ En este caso la respuesta es **sí**, ya hemos visto que se trata del fichero `/routes/users.js`. En caso de no disponer de dicho fichero habría que crearlo.
- ▶ Una vez determinemos dónde se va a desarrollar este manejador de rutas, procedemos a desarrollarlo.

```
router.post('/create', (req, res) => {  
  res.send('Respondemos a la petición POST');  
})
```

En esta ocasión utilizamos el método **post**, pero el número y tipo de los parámetros es el mismo. Como primer parámetro especificamos la url con la cual vamos a interactuar y como segundo parámetro, la función manejadora que se va a ejecutar cuando a nuestro servidor llegue una petición con el método POST sobre la url `/users/create`.

Probando la petición POST

Cuando estamos trabajando con el navegador solo podemos lanzar peticiones

de tipo **GET**, si necesitamos probar cualquier tipo de petición con un método diferente debemos utilizar algún tipo de herramienta externa que nos permita llevar a cabo este trabajo.

Existen muchas herramientas destinadas a cumplir con esta labor, pero para este caso concreto vamos a usar una muy sencilla de utilizar y que además se puede integrar directamente con **Visual Studio**.

El plugin se llama **REST Client** <https://marketplace.visualstudio.com/items?itemName=humao.rest-client>

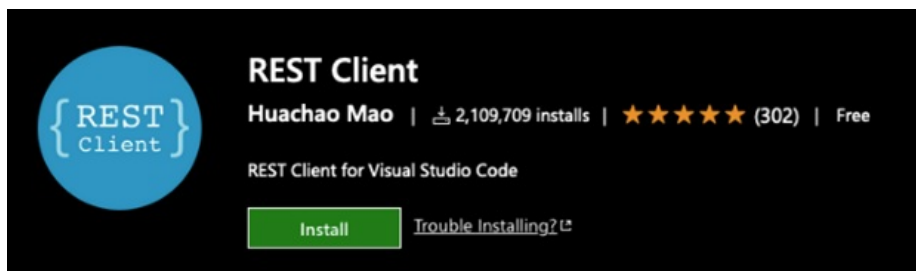


Figura 5. Rest Client VSCODE Extensión. Fuente: elaboración propia

Una vez instalado el plugin dentro de Visual Studio, lo único que tenemos que hacer es **generar un fichero con extensión .rest**. En nuestro caso vamos a generar el fichero **peticiones.rest**.

Dentro de dicho fichero establecemos las diferentes peticiones que vamos a probar, como marca la siguiente imagen:

```
GET <a href="http://localhost:3000/users">http://localhost:3000/users
</a><a href="http://localhost:3000/users" target="_blank"></a>###
Send Request
POST <a href="http://localhost:3000/users/create"
target="_blank">http://localhost:3000/users/create</a>
<a href="http://localhost:3000/users/create" target="_blank"></a>
```

Con la aplicación arrancada y pulsando el botón que pone **Send Request** podemos

lanzar cualquier tipo de petición para probar el flujo que estamos creando dentro de nuestro servidor. La separación entre cada petición la debemos hacer con los caracteres **###**.

5.5. Creación de middlewares

Una vez hemos establecido las normas básicas para la gestión de rutas dentro de nuestras aplicaciones es momento de conocer otras herramientas que nos ofrece Express para facilitar el trabajo sobre nuestro código.

Puede ser que antes de responder al cliente, durante el flujo que sigue la petición entrante al servidor hasta que llega al manejador final, necesitemos realizar una serie de acciones y que además esas acciones se repitan para un gran número de peticiones. Pongamos un ejemplo:

Me llega una petición de tipo GET sobre la url /info. Antes de responder, se debe generar un registro en la base de datos y además enviar un email al administrador de la aplicación. Una vez se realizan esta serie de tareas procedemos a responder a la petición.

Si, además, como decíamos antes, esta forma de trabajo se repite en múltiples peticiones debemos usar un código común para todas. Es por ello por lo que **Express nos permite el uso de Middlewares.**

Características de los middlewares

Estas son algunas de las características que deben cumplir los middlewares que generamos dentro de nuestras aplicaciones de Express:

- ▶ Son funciones manejadoras que tienen acceso a la petición entrante (req), al objeto para generar la respuesta (res) y a una función que llamaremos **next** la cual nos permite delegar la petición al siguiente manejador.
- ▶ Dentro de un middleware podemos colocar todo el código que necesitemos, sin limitaciones.

- ▶ Internamente se pueden hacer **comprobaciones y modificaciones** sobre la petición entrante al servidor. Es una acción bastante común dentro de estas estructuras agregar ciertos datos dentro del objeto req para poder usarlos más adelante en la aplicación.
- ▶ Dependiendo de las comprobaciones que realicemos dentro del propio middleware podemos terminar incluso con la petición en curso. Esta acción es muy útil cuando necesitamos limitar el acceso a nuestro servidor si no se cumplen ciertas condiciones.
- ▶ Podemos provocar que la petición siga hacia el siguiente manejador disponible una vez se hayan realizado todas las tareas para las cuales se había diseñado el middleware.

Ejemplo de middleware LOG

Dentro de nuestro fichero **app.js** podemos incluir la siguiente ejecución:

```
app.use((req, res, next) => {  
  console.log('Mensaje dentro del middleware');  
  next();  
})  
app.use('/', indexRouter);  
app.use('/users', usersRouter);
```

Justo antes de la definición de nuestras rutas colocamos una llamada al método use.

El primer parámetro de esta llamada es nuestra función middleware.

Lo que estamos haciendo con este bloque de código es asegurarnos que, antes de llegar al manejador final y, para todas las peticiones entrantes a nuestro servidor, primero pasaremos por esta función.

Internamente, después de lanzar un log informativo, ejecutamos la función **next** la cual permitirá que la petición siga su camino hasta que se cruce con el manejador

que se encargue de responder al cliente.

Si dentro del propio middleware omitimos la ejecución del método `next`, el servidor no determinaría ninguna respuesta y por lo tanto el cliente quedaría a la espera obteniendo casi con total seguridad algún tipo de error de timeout.

Ejemplo de middleware aleatorio

Definimos otro middleware en el que de forma aleatoria damos por terminada la petición o, por el contrario, dejamos que siga con el flujo normal hacia el siguiente manejador.

```
app.use((req, res, next) => {
  const randomNum = Math.random();
  console.log(randomNum);
  if (randomNum > 0.5) {
    res.send('Número mayor de 0.5');
  } else {
    next();
  }
});
```

Para este pequeño ejemplo primero recuperamos un número aleatorio entre 0 y 1 y en función del resultado, respondemos al cliente, dando por concluida la petición o dejamos que la petición siga el curso natural hasta el siguiente manejador.

Este middleware es un ejemplo creado para mostrar las acciones que podemos llevar a cabo a través de este tipo de bloques de software. El contenido que incluyamos dentro de cada uno de ellos depende en gran parte de aquello que necesitemos para nuestras determinadas aplicaciones.

Modificando la petición en el middleware

El objeto **req** con el que trabajamos dentro de estos middlewares **es el mismo**

objeto al que luego tendremos acceso dentro del manejador pertinente.

Es por ello por lo que, contando con este concepto, podemos modificarlo a nuestro gusto para que cumpla alguna funcionalidad extra con respecto a las que ya tiene predefinidas.

Lo vemos en el siguiente ejemplo de middleware:

```
app.use((req, res, next) => {  
  req.currentDate = new Date();  
  next();  
})
```

Dentro de la petición entrante estamos generando una nueva propiedad llamada **currentDate** (el nombre depende del valor que guarde). Como el paso posterior es la llamada al método **next**, la propiedad creada estará disponible desde cualquier manejador de ruta que se ejecute justo detrás de este middleware.

De esta manera, valores que queramos compartir a lo largo de todo el flujo de nuestra aplicación podemos hacerlo a través de la petición entrante de una manera muy cómoda y elegante.

Encadenando Middlewares

ExpressJS nos ofrece una gran flexibilidad a la hora de establecer los diferentes manejadores a través de los cuales pasará la petición entrante antes de generar la respuesta final al cliente. Aparte de esto, nos permite la reutilización de gran parte de nuestro código para conseguir los objetivos planteados.

De esta manera, casi todas las funciones para el manejo de rutas (get, post, put...) permiten recibir como parámetro un número indeterminado de funciones manejadoras, que nos permitirán realizar ciertas acciones específicas antes de devolver la respuesta final.

Lo vemos a través de un ejemplo:

```
app.get('/ejemplo/a', (req, res, next) => {  
  console.log('Pasa por el middleware');  
  next();  
}, (req, res) => {  
  res.send('Respondemos a la petición');  
})
```

Recibimos dos funciones como parámetros dentro de la llamada y cada una de ellas podría encargarse de resolver problemas diferentes.

Siempre tenemos que tener en cuenta que **la última función manejadora de la cadena es la encargada de responder**. Si esto no sucede el cliente se queda a la espera de respuesta y puede ocasionar numerosos errores.

Las funciones que definamos como middlewares intermedios pueden, o responder directamente como ya hemos visto anteriormente, o llamar a la función next y por lo tanto pasar la responsabilidad de la petición al siguiente manejador.

Es muy importante que **solamente respondamos una única vez por cada una de las peticiones de nuestro servidor**. Si por error usamos los métodos de respuesta asociados al objeto res en más de una ocasión, recibiremos un error a través del terminal en el que se especificará que la respuesta ya ha sido enviada y por lo tanto no se puede volver a reenviar.

Podemos llevar el ejemplo anterior un pasito más allá separando las funciones middleware y de respuesta que vamos a ejecutar. Lo vemos en el siguiente ejemplo:

```
const handler1 = (req, res, next) => {  
  console.log('Pasa por el manejador 1');  
  next();  
}  
const handler2 = (req, res, next) => {  
  console.log('Pasa por el manejador 2');  
  next();  
}
```

```
const handler3 = (req, res) => {  
  res.send('Respuesta final');  
}  
app.get('/ejemplo/b', [handler1, handler2, handler3]);
```

Las tres funciones se definen de manera independiente y posteriormente pasamos dichas funciones al manejador final de ruta a través de un array.

En este caso **es importantísimo el orden de ejecución** ya que handler3 es la que realmente responde al cliente. Si handler3 estuviese situada en otra posición dentro del array pasado por parámetro, las funciones definidas posteriormente a esta no se ejecutarían.

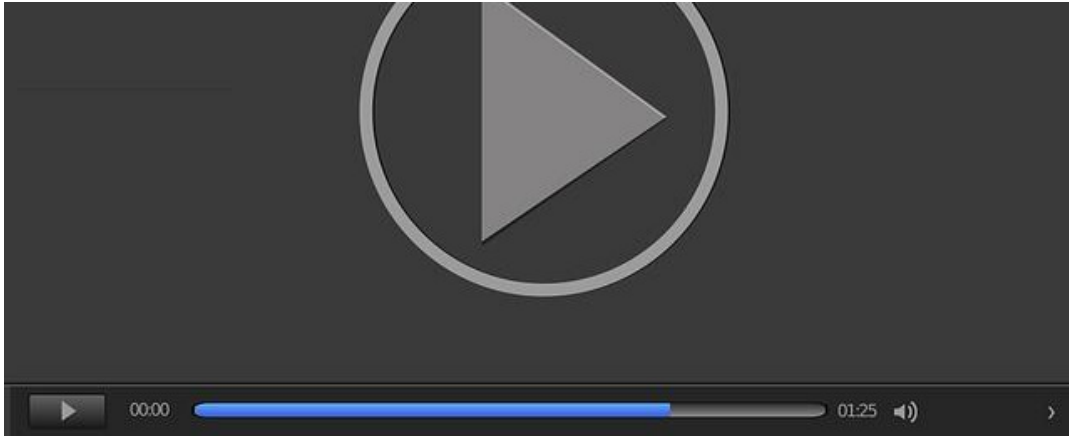
Podríamos incluso hacer una combinación de las versiones que hemos visto anteriormente:

```
const handler1 = (req, res, next) => {  
  console.log('Pasa por el manejador 1');  
  next();  
}  
const handler2 = (req, res, next) => {  
  console.log('Pasa por el manejador 2');  
  next();  
}  
app.get('/ejemplo/c', [handler1, handler2], (req, res, next) => {  
  console.log('Pasa por el manejador 3');  
  next();  
}, (req, res) => {  
  res.send('Respuesta final');  
});
```

Podemos crear gran cantidad de middlewares para resolver diferentes acciones dentro de nuestros proyectos. En el siguiente ejemplo generamos un ***Middleware para la gestión de acciones en un LOG.***



Tema 5. ExpressJS



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=e464a33a-274a-4cda-b4c2-ae360119ffc2>

5.6. Respuestas disponibles

Para las pruebas que hemos ido lanzando hemos usado el método **send** asociado al objeto **res** para poder generar la respuesta que viajará de vuelta al cliente. Express nos ofrece una serie de métodos que podemos utilizar en este caso dependiendo de lo que queramos transmitir de vuelta hacia el cliente.

Alguno de ellos ya lo veremos más ampliado cuando nos enfrentemos a la creación de un **API Rest** o cuando tengamos que devolver código HTML para que se visualice dentro del navegador.

Los métodos que tenemos disponibles son los siguientes:

```
res.download();  
// Fuerza la descarga de un fichero  
res.end();  
// Termina el proceso de la respuesta  
res.json();  
// Devuelve una estructura en formato JSON  
res.redirect();  
// Redirige la petición  
res.render();  
// A través de un motor de plantillas genera una vista en HTML  
res.send();  
// Devuelve una respuesta genérica
```

Tema 6. Integración de bases de datos MySQL

6.1. Introducción y objetivos

Ha llegado un punto en el que ya tenemos herramientas suficientes como para enfrentarnos a la creación de aplicaciones complejas con NodeJS. Debemos ser capaces de gestionar las peticiones entrantes al servidor y generar respuestas adecuadas en cada caso.

Con una buena arquitectura de aplicación y con la posibilidad de escalar nuestro código de la manera óptima es el momento de plantearse cómo extraemos los datos con los que vamos a trabajar y hacer que nuestras respuestas cumplan con los objetivos planteados.

Vamos a tratar de integrar todos los conocimientos que traemos sobre bases de datos MySQL dentro de nuestras aplicaciones Express siguiendo unas buenas prácticas y una organización que nos permita interactuar con los datos de la manera óptima.

Nuestro objetivo final en este tema es que asimilemos el concepto de **API Rest** y seamos capaces de implementar una aplicación que cumpla con ese estándar extrayendo los datos necesarios de una base de datos MySQL.

Tema 6. Integración de bases de datos MySQL

6.2. Conexión a la base de datos

Antes de comenzar con el código de nuestra aplicación debemos generar una base de datos que nos servirá para poder trabajar de aquí en adelante. La base de datos con la que trabajaremos tendrá las siguientes características:

Nombre: empresa

Tablas:

- ▶ employees (id (PK AI), name, surname, address, email, phone, notes, project_id (FK)).
- ▶ projects (id(PK, AI), name, start_date, end_date, budget).

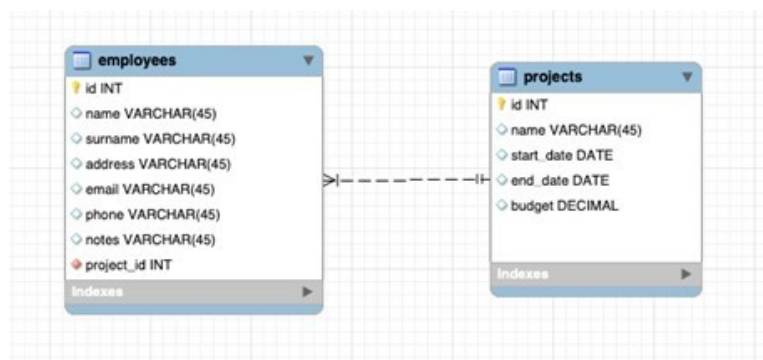


Figura 6. Modelo de datos para las tablas employees y projects . Fuente: elaboración propia

Además de la creación previa de esta base de datos, vamos a generar de igual manera una aplicación con Express que nos permita trabajar sobre ella en los próximos temas. Para ello utilizamos el comando **express** que vimos en el tema anterior:

express --view=pug --git appEmpresa

Una vez hecha la creación, seguimos las instrucciones para la instalación de las

Tema 6. Integración de bases de datos MySQL

dependencias y ya tendríamos todo preparado para empezar a trabajar.

Instalación de la librería mysql

Desde Express no podemos conectarnos de manera directa con la base de datos y necesitamos alguna librería que nos ofrezca todas las herramientas necesarias para realizar dicha conexión y aparte, que nos permita lanzar todas las acciones que vamos a llevar a cabo.

Existe una librería nativa que podemos instalar sobre nuestro proyecto llamada **mysql2**. La instalamos desde el terminal, en el directorio de nuestra aplicación a través del siguiente comando:

npm install mysql2

A partir de aquí debemos disponer el código que nos permita realizar la conexión contra nuestra base de datos pasándole todos los parámetros de conexión de la misma.

Para ello, vamos a generar un fichero **dbConfig.js** en la raíz del proyecto:

```
const mysql = require('mysql2');
const pool = mysql.createPool({
  host: '127.0.0.1',
  user: 'root',
  password: 'mysqlpassword',
  database: 'empresas',
  port: 3306
});
global.db = pool.promise();
```

Dentro de este pequeño script estamos generando un **pool de conexiones**. Se trata de una herramienta que se encarga de abrir la conexión, lanzar la query que necesitamos ejecutar y volver a cerrar la conexión.

En la creación del pool **necesitamos especificar los datos concretos de nuestra**

Tema 6. Integración de bases de datos MySQL

base de datos, ya que pueden cambiar dependiendo del sistema operativo.

La última línea del script la utilizamos para generar una variable global **db** la cual almacenará el valor del pool pudiendo acceder a él desde cualquier parte de nuestra aplicación. Al llamar al método **promise** sobre el pool de conexiones estamos recuperando un objeto sobre el cual vamos a poder trabajar con promesas de manera más cómoda.

El paso siguiente será ejecutar este código. Para ello podemos requerir el fichero desde **app.js** y así se ejecutará cuando arranquemos la aplicación.

Antes de la creación de la app de Express:

```
// Conectamos con la base de datos
require('./dbConfig');
var app = express();
```

(Podemos arrancar la aplicación para visualizar si aparece algún tipo de error heredado de esta conexión).

Tema 6. Integración de bases de datos MySQL

6.3. Creación de modelos e integración de queries

Como hemos venido indicando hasta ahora en temas anteriores, la organización en este tipo de proyectos es fundamental y determina un alto porcentaje de éxito a la hora de modificar o escalar nuestras aplicaciones.

Para poder trabajar de manera más cómoda sobre los datos vamos a **generar ficheros independientes donde vamos a disponer todos los métodos de acceso a base de datos.**

Dentro de la librería **mysql** tenemos un método llamado **query** que nos permite ejecutar una sentencia SQL y nos devuelve el resultado de dicha sentencia. Si es una sentencia de **SELECT**, recuperamos todos los registros que estemos requiriendo, mientras que, si es una sentencia de **INSERT**, **UPDATE** o **DELETE** recuperamos el resultado de la acción que estemos llevando a cabo.

Para poder probar que todo está correcto y ver cómo ejecutar nuestras primeras queries vamos a generar el fichero **employee.js** dentro del directorio **models** donde encontraremos todas las interacciones sobre la tabla **employees**.

```
const getAll = () => {  
  return db.query('select * from employees');  
}  
module.exports = {  
  getAll  
}
```

En primera instancia se trata de un fichero muy simple en el que vamos a ir colocando diferentes métodos para tener encapsuladas en diferentes métodos las acciones que llevaremos a cabo sobre la base de datos.

Este primer método ejecuta la sentencia para la recuperación de todos los empleados de la base de datos y nos devuelve una promesa, la cual retornamos con

Tema 6. Integración de bases de datos MySQL

el método para poder consumirla de manera externa en el punto exacto donde necesitemos los datos.

Como prueba podemos colocar dentro de app.js, justo debajo de la importación de la configuración de la base de datos el siguiente código que nos permita asegurarnos de que todo va según lo esperado.

```
// Conectamos con la base de datos
require('./dbConfig');
// PRUEBA
(async () => {
  const employeeModel = require('./models/employee');
  const [rows] = await employeeModel.getAll();
  console.log(rows);
})();
```

Si nos fijamos en el ejemplo, **el primer paso es requerir el modelo para poder utilizarlo**. Seguidamente **llamamos al método getAll para que nos devuelva la promesa**. Mediante async-await recuperamos el valor devuelto por el método.

Como podemos observar, la ejecución de la promesa nos devuelve un array con diferentes propiedades. Una de esas propiedades es **rows** donde encontraremos todos los registros de la tabla con la que estemos interactuando. El resultado nos debería mostrar a través del terminal todos los empleados de la tabla **employees**.

Si la query que necesitamos generar es diferente a una selección, podemos crear un método dentro del modelo de empleados de la siguiente manera:

```
const getAll = () => {
  return db.query('select * from employees');
}
const create = ({ name, surname, address, email, phone, notes }) => {
  return db.query(
    'insert into employees (name, surname, address, email, phone, notes) values',
    (?, ?, ?, ?, ?, ?)',
    [name, surname, address, email, phone, notes]
  );
}
module.exports = {
```

Tema 6. Integración de bases de datos MySQL

```
getAll, create  
}
```

En este caso la query es más compleja ya que debemos insertar ciertos datos. Para solucionarlo de una manera elegante podemos **pasarle al método por parámetro un objeto con cada una de las claves con las que vamos a generar el nuevo empleado.**

Dentro de la query **debemos colocar el carácter de interrogación final (?) en cada uno de los espacios donde vayamos a colocar un valor variable.**

Como segundo parámetro del método **pasamos un array con todos aquellos valores que se van a sustituir por las interrogaciones, en el orden que necesitemos.**

Podemos probar esta inserción del mismo modo que hemos probado la recuperación de datos:

```
// PRUEBA  
(async () => {  
  const employeeModel = require('./models/employee');  
  const newEmployee = {  
    name: 'Luis', surname: 'Sánchez', address: 'C Mayor 23', email:  
    'luis@mail.com', phone: '99891299', notes: 'NONE'  
  }  
  const [result] = await employeeModel.create(newEmployee)  
  console.log(result);  
  const [rows] = await employeeModel.getAll();  
  console.log(rows);  
})();
```

En este caso el resultado de la ejecución del método es totalmente diferente al anterior ya que, en vez de recibir una serie de registros, recibimos el resultado de la inserción dentro de la base de datos.

Tema 6. Integración de bases de datos MySQL

```
ResultSetHeader {  
  fieldCount: 0,  
  affectedRows: 1,  
  insertId: 6,  
  info: '',  
  serverStatus: 2,  
  warningStatus: 0  
}
```

Figura 7. Resultado obtenido por consola a partir de la inserción. Fuente: elaboración propia

Dentro de este objeto observamos la propiedad **insertId** la cual nos indica que el registro se ha creado y además cuál es su identificador único dentro de la base de datos.

Con estas pequeñas pautas ya podemos empezar a crear una estructura de rutas que nos permitan realizar todas las acciones pertinentes sobre la tabla employees.

Tema 6. Integración de bases de datos MySQL

6.4. CRUD sobre MySQL para la creación de una API

Antes de comenzar a diseñar y desarrollar las urls dentro de nuestra aplicación es importante que tengamos claro el concepto de **REST Api**.

Se trata de una serie de normas que debemos seguir para el desarrollo de nuestro servidor para que cualquier cliente que quiera interactuar con nosotros lo haga de la misma manera y por lo tanto no tengamos problemas en la comunicación.

Para la creación de este tipo de estructuras debemos tener en cuenta estos aspectos:

- ▶ Es importante que nuestro sistema de comunicación **no establezca ningún estado**. Toda la información para completar la petición debe ir encapsulada en la misma petición. La respuesta, de igual manera, debe devolver todos los datos requeridos, **no se pueden establecer una serie de pasos para lograr un objetivo concreto**.
- ▶ Debemos establecer un patrón de operaciones bien definidas a partir de los diferentes verbos que tenemos disponibles en el protocolo HTTP. Cada uno de ellos hace referencia a una acción concreta (GET, POST, PUT, DELETE...).
- ▶ Debemos mantener una **sintaxis universal** para nuestras urls y que así sean reconocibles por cualquier sistema externo que necesite acceder a ellas.
- ▶ Es muy útil crear relaciones entre recursos que estén relacionados entre sí para así facilitar su consumo. Por ejemplo, si estamos paginando un resultado podemos devolver la url siguiente y la url anterior para así poder acceder a los diferentes datos relacionados.

Si nos basamos en el estándar que establece el protocolo HTTP, el significado de los diferentes verbos sería el siguiente:

Tema 6. Integración de bases de datos MySQL

- ▶ **GET** – recuperamos datos de un recurso dentro de nuestra aplicación.
- ▶ **POST** – generamos un nuevo recurso. Normalmente encontraremos los datos del recurso a crear dentro de la propia petición.
- ▶ **PUT / PATCH** – lanzamos acciones de edición sobre un recurso en concreto.
- ▶ **DELETE** – borramos un registro existente en nuestra base de datos.

Estableciendo esta pequeña guía podemos concretar qué urls vamos a implementar para poder interactuar sobre la tabla **employees**.

GET <http://localhost:3000/api/employees>

Recupera todos los empleados de la base de datos y nos devuelve un JSON con el resultado. Se podría establecer algún tipo de paginación o filtro en el momento de su recuperación.

POST <http://localhost:3000/api/employees>

Recibe todos los datos necesarios para la creación de un nuevo empleado dentro de la base de datos. En este caso deberíamos realizar algún tipo de validación previa a la inserción.

PUT <http://localhost:3000/api/employees/23>

Recibe los datos correspondientes para la edición de un empleado, especificando su identificador dentro de la url.

DELETE <http://localhost:3000/api/employees/23>

Tema 6. Integración de bases de datos MySQL

Borramos el empleado cuyo id recibimos a través de la URL.

Una vez tenemos claras las urls que vamos a desarrollar debemos establecer qué ficheros deben ser los encargados de alojar dichas urls.

Vamos a plantear varias situaciones y en función de puntos a favor y puntos en contra elegiremos la más acertada:

Opción 1

Creamos el código dentro del fichero **app.js**. Colocamos dentro de dicho fichero todos los manejadores necesarios para hacer frente a las diferentes urls que hemos diseñado anteriormente.

A favor: nada.

En contra: Si todos los manejadores de nuestra aplicación se encuentran definidos dentro de `app.js`, éste se terminará convirtiendo en un fichero muy grande, con demasiadas líneas de código y de difícil manejo.

DESCARTADA

Opción 2

Dentro de la carpeta **routes** generamos un fichero **employees.js** que se encargue de alojar todos los manejadores para hacer frente a las peticiones anteriores. Desde **app.js** delegamos todas las peticiones sobre la url **/api/employees** a ese fichero y así quedarían enganchados.

A favor: Dentro de ese fichero quedarían desarrolladas todas las acciones sobre la tabla `employees`. Tendríamos controlado dónde y cómo modificarlas sin entrar en conflicto con ficheros diferentes.

Tema 6. Integración de bases de datos MySQL

En contra: Si tenemos que gestionar distintos tipos de recursos (projects, tasks...) el fichero **app.js** seguiría quedando con numerosas líneas de código y bastante complicado de manejar al tener que tratar con todas las delegaciones a los ficheros de rutas.

DESCARTADA

Opción 3

Dentro de la carpeta routes generamos un fichero **api.js** sobre el cual delegaremos el desarrollo de todas las rutas que comiencen con **/api**.

A partir de este fichero volveríamos a delegar las rutas sobre ficheros más específicos. Por ejemplo, todas las rutas sobre **/api/employees** podrían estar manejadas desde un fichero situado en **/routes/api/employees.js**.

A favor: Los ficheros manejadores de rutas se especializan bastante y se centran en manejar una serie de acciones concretas.

En contra: Debemos prestar mucha más atención a dónde están situados los ficheros y qué labor cumplen cada uno de ellos.

ACEPTADA

Desarrollo del API Rest

Una vez hemos establecido nuestro punto de partida y el diseño previo de urls, podemos pasar al desarrollo de las mismas.

routes/api/employees.js

```
const express = require('express');
```

Tema 6. Integración de bases de datos MySQL

```
const router = express.Router();
// GET <a
href="http://localhost:3000/api/employees">http://localhost:3000/api/employees

</a>router.get('/', (req, res) => {
});
// POST <a
href="http://localhost:3000/api/employees">http://localhost:3000/api/employees

</a>router.post('/', (req, res) => {
});
// PUT <a
href="http://localhost:3000/api/employees/23">http://localhost:3000/api/employees/23

</a>router.put('/:employeeId', (req, res) => {
});
// DELETE <a
href="http://localhost:3000/api/employees/23">http://localhost:3000/api/employees/23

</a>router.delete('/:employeeId', (req, res) => {
});
module.exports = router;
```

Una vez enlacemos todos los ficheros y partiendo del esqueleto anterior desarrollaremos todas las rutas.

routes/api.js

Este fichero nos servirá como punto de unión para todas las peticiones de nuestro api. Tendrá la labor de delegar cada ruta a su fichero gestor correspondiente.

```
const express = require('express');
const router = express.Router();
// Recuperamos el fichero con los manejadores para la api de empleados
const apiEmployeesRouter = require('./api/employees');
// Delegamos todas las peticiones con el prefijo /api/employees al fichero
anterior
router.use('/employees', apiEmployeesRouter);
module.exports = router;
```

app.js

El último paso para completar este diseño es enlazar todas aquellas peticiones que

Tema 6. Integración de bases de datos MySQL

lleguen con la url/**api** con el fichero api.js que hemos desarrollado anteriormente.

```
const apiRouter = require('./routes/api');  
app.use('/api', apiRouter);
```

(Es recomendable, una vez hemos desarrollado todos los ficheros, probar si las urls funcionan antes de entrar en otro tipo de desarrollos).

Vamos a ir generando cada una de las partes del **CRUD (Create, read, update, delete)** para interactuar con la tabla de employees.

READ – Recuperación de datos

Para la lectura de datos sobre la tabla ya tenemos generado sobre el modelo el método que nos permite recuperar todos los registros, por lo tanto, el siguiente paso sería, dentro del fichero **routes/api/employees.js** definir qué hace el manejador de ruta para este caso concreto.

Tengamos en cuenta que estamos desarrollando un API y todas las respuestas deberían ser en formato JSON para que sean comprensibles por el cliente. Es por esto que debemos cuidar siempre cualquier tipo de respuesta para que el cliente siempre quede informado del resultado.

routes/api/employees.js

```
const { getAll } = require('../../models/employee');  
// GET <a href="http://localhost:3000/api/employees">http://localhost:3000/api/employees  
  
</a>router.get('/', async (req, res) => {  
  try {  
    const [rows] = await getAll();  
    res.json(rows);  
  } catch (err) {  
    res.json({ error: err.message });  
  }  
});
```

Tema 6. Integración de bases de datos MySQL

Dentro del manejador de la ruta incluimos la llamada al método **getAll**, previamente importado, y recuperamos todos los registros que nos devuelva la query. Si no hay ningún tipo de error que nos obligue a pasar por el catch, se devuelve un JSON con el array de registros recuperados.

El error podemos transmitirlo como necesitemos. En este caso hemos elegido responder con un JSON con la clave «error» y el valor del mensaje del error que aparezca.

CREATE – Creación de un empleado

Como en el caso anterior, el método en el modelo ya está definido por lo que nos centraremos en la creación del manejador para la ruta.

```
const { getAll, create } = require('../models/employee');
// POST <a
href="http://localhost:3000/api/employees">http://localhost:3000/api/employees

</a>router.post('/', (req, res) => {
  create(req.body)
  .then([result]) => res.json(result))
  .catch(err => res.json({ error: err.message }));
});
```

Para este método debemos analizar un par de aspectos importantes:

Para poder recuperar todos los datos que vengan asociados a esta petición, dentro del método manejador usamos la propiedad **req.body**. De esta manera le estamos haciendo llegar al método un objeto con todos los valores a insertar.

Si necesitamos probar esta url, podemos generar un fichero con extensión **.rest** como hemos hecho en otras ocasiones e incluir el siguiente contenido:

Tema 6. Integración de bases de datos MySQL

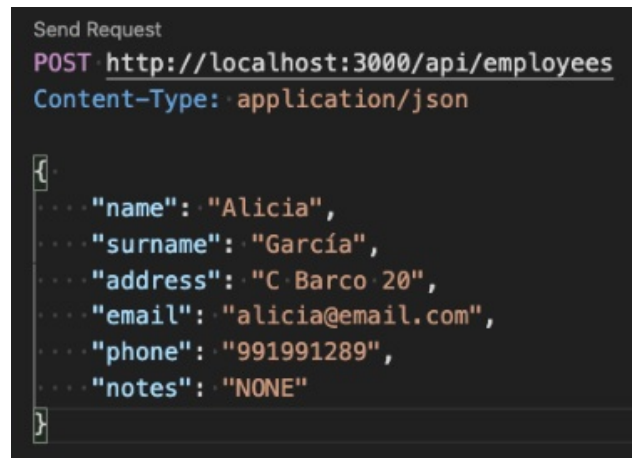


Figura 8. Prueba para la creación de empleados. Fuente: elaboración propia

Además, para tener presentes las diferentes formas de trabajar, en este caso hemos consumido la promesa a través de los métodos **then y catch**. La respuesta generada es exactamente igual que si lo hiciésemos con `async-await`.

UPDATE – actualización de empleado

Para esta acción seguimos los mismos pasos que para la inserción ya que necesitaremos una serie de datos que llegarán asociados a la petición (`req.body`).

El primer paso es definir el método de actualización dentro del modelo de empleado:

models/employee.js

```
const update = (employeeId, { name, surname, address, email, phone, notes
}) => {
  return db.query(
    'update employees set name = ?, surname = ?, address = ?, email = ?, phone
    = ?, notes = ? where id = ?',
    [name, surname, address, email, phone, notes, employeeId]
  );
}
module.exports = {
  getAll, create, update
}
```

Tema 6. Integración de bases de datos MySQL

Como primer parámetro de la llamada a la función query pasamos la sentencia SQL que nos permitirá hacer la actualización. Como en el caso de la inserción, en los puntos donde necesitemos incluir algún tipo de variable, lo sustituimos por una interrogación de cierre.

Como segundo parámetro pasamos un array con los valores que se van a sustituir en la query en el mismo orden en el que estén definidos dentro de la query. En este caso hay que prestar especial atención al **employeeId**, el cual recibimos como primer parámetro del método **update** pero luego lo colocamos en la última posición del array porque coincide con esa posición dentro de la query.

Una vez definido este método dentro del modelo ya podemos utilizarlo dentro de la ruta que gestiona la petición de actualización, prestando especial atención a cómo recuperamos el id del empleado incluido dentro de la url.

routes/api/employees.js

```
const { getAll, create, update } = require('../../models/employee');
// PUT <a href="http://localhost:3000/api/employees/23">http://localhost:3000/api/employees/23
</a>router.put('/:employeeId', async (req, res) => {
  try {
    const [result] = await update(req.params.employeeId, req.body);
    res.json(result);
  } catch (err) {
    res.json({ error: err.message });
  }
});
```

En este manejador destacamos la obtención del id del empleado de la url. Para poder hacerlo, primero hemos definido la url como primer parámetro del método **put** con **:employeeId**.

De esta manera estamos indicando que en ese espacio vamos a poder colocar cualquier valor y que la clave para recuperarlo será **employeeId**.

Tema 6. Integración de bases de datos MySQL

Dentro del método recuperamos el valor incluido dentro de la url con **req.params.employeeId**, dato necesario para posteriormente hacer la actualización. Además recogemos todos los datos recibidos con la petición a través de **req.body** y se lo pasamos de igual manera al método **update**.

Si necesitamos probar esta url podemos hacerlo a través de la siguiente estructura en nuestro fichero de peticiones.

```
PUT http://localhost:3000/api/employees/23
Content-Type: application/json

{
  "name": "Alicia",
  "surname": "García",
  "address": "C Barco 20",
  "email": "alicia@email.com",
  "phone": "991991289",
  "notes": "dpto: contabilidad"
}
```

Figura 9. Prueba para la actualización de un empleado concreto. Fuente: elaboración propia

DELETE – Borrado de un empleado

Es una de las acciones más sencillas. Únicamente tenemos que prestar atención al identificador del empleado que necesitamos borrar.

Dentro del modelo de **employee** podemos definir el método para borrar

models/employee.js

```
const deleteById = (employeeId) => {
  return db.query(
    'delete employees where id = ?',
    [employeeId]
  );
}

module.exports = {
  getAll, create, update, deleteById
}
```

Tema 6. Integración de bases de datos MySQL

```
}
```

A partir de aquí podemos utilizar este método para crear la ruta

routes/api/employees.js

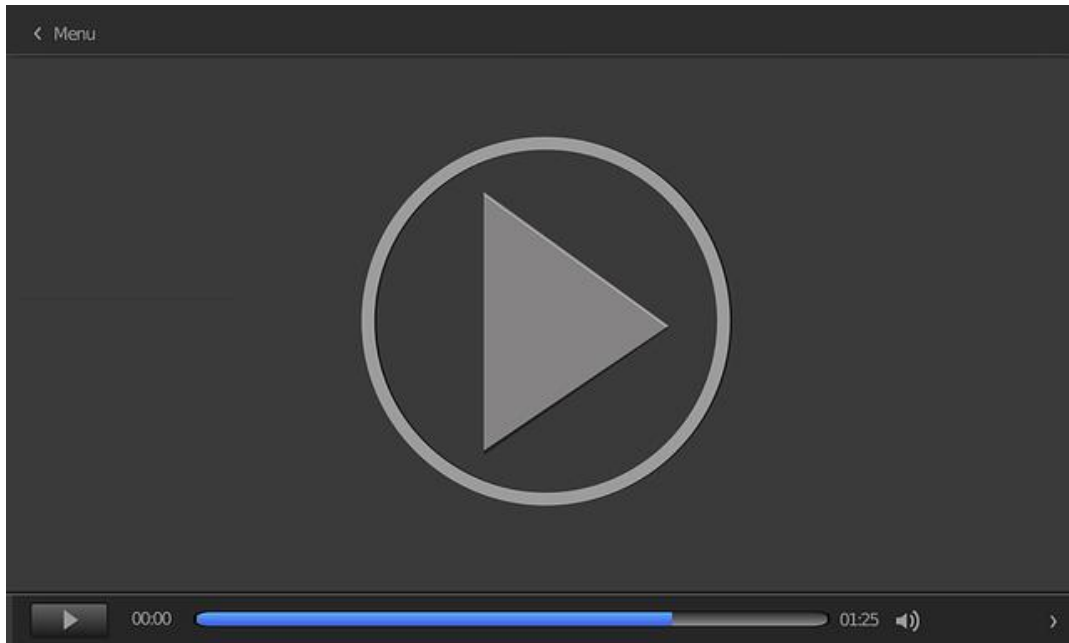
```
const { getAll, create, update, deleteById } =
require('../models/employee');
// DELETE <a
href="http://localhost:3000/api/employees/23">http://localhost:3000/api/employees/23

</a>router.delete('/:employeeId', async (req, res) => {
  try {
    const [result] = await deleteById(req.params.employeeId);
    res.json(result);
  } catch (err) {
    res.json({ error: err.message })
  }
});
```

De la misma manera que hemos recuperado el id para el caso de la actualización lo hacemos en este otro caso. En la definición de la url colocamos precedido de dos puntos el nombre que le vamos a asignar a ese valor y luego lo recuperamos a través del objeto **req.params**.

Tema 6. Integración de bases de datos MySQL

En el siguiente vídeo, *API Express y MySQL: Recuperación y creación*, se hace un repaso general a la configuración de una aplicación de Express que nos permitan recopilar datos a partir de una tabla MySQL e insertar de la misma manera.

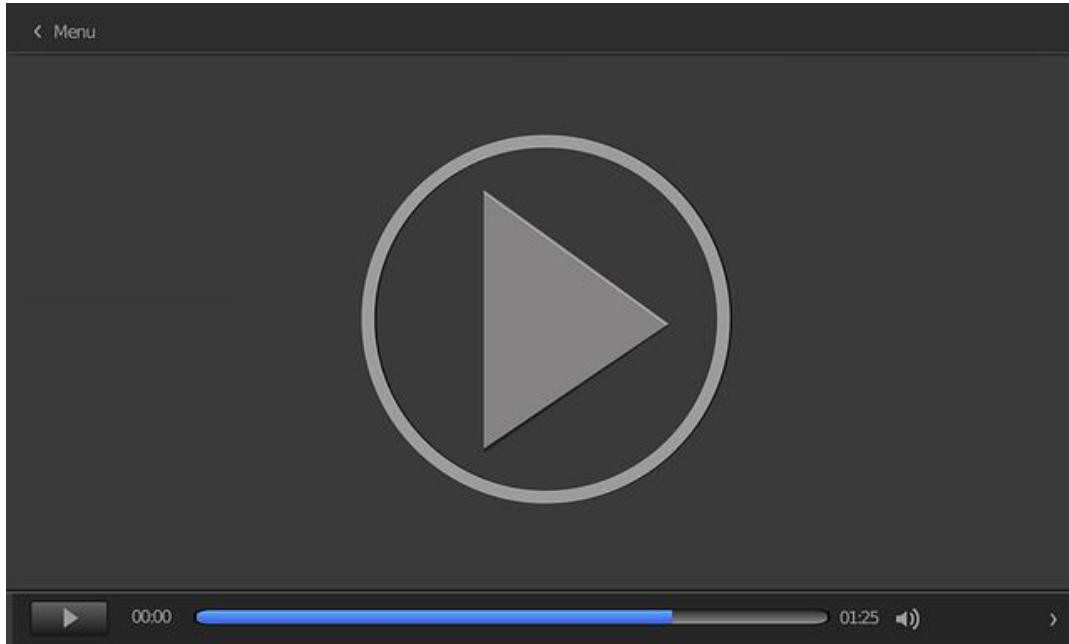


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=02d4d052-5aad-44cd-a8a2-ae36012223f0>

Tema 6. Integración de bases de datos MySQL

En el siguiente vídeo, *API Express y MySQL: Actualización y borrado*, completamos el ejemplo anterior realizando las acciones necesarias para poder actualizar nuestros recursos y posteriormente borrarlos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=5a23ccaa-eacb-4cca-b761-ae360128ee2a>

Tema 7. Integración de bases de datos Mongo DB

7.1. Introducción y objetivos

Uno de los stacks más demandados en el mercado es el stack MEAN que comprende como ya sabemos las siguientes tecnologías, MongoDB, Express, Angular y NodeJS.

Para completar ese stack, con los conocimientos que ya tenemos sobre las bases de datos MongoDB, nos falta conocer cómo podemos integrar las diferentes colecciones y documentos dentro de una estructura de Express y por lo tanto que estén accesibles a través de diferentes urls como ya hemos hecho en el caso de las bases de datos relacionales.

Para este tipo de bases de datos tenemos una ventaja muy grande y es que la tecnología que utilizamos para interactuar con la base de datos es muy parecida a la que utilizamos en el desarrollo de la propia aplicación. Es por eso por lo que, para nosotros, la interacción con una base de datos MongoDB se resumirá en el aprendizaje de qué métodos tenemos que utilizar para realizar una acción u otra.

Al final del tema seremos capaces de generar un API Rest conectando a una base de datos MongoDB con la misma estructura utilizada en el tema anterior.

Tema 7. Integración de bases de datos Mongo DB

7.2. Conexión a la base de datos con Mongoose

Para poder conectar con nuestra base de datos vamos a utilizar la librería **Mongoose** la cual, de una manera muy sencilla, nos abstrae de las tareas más complejas que conlleva esta interacción y nos ofrece una serie de métodos que nos permiten mantener un contacto continuo con la base de datos.

El primer paso será generar la aplicación de Express con la que vamos a trabajar a través del comando **express**.

express --view=pug --git tiendaOnline

En cuanto instalemos las dependencias del Proyecto y probemos que todo funciona según lo esperado podemos empezar con la integración

El primer paso, tal y como hicimos en la integración de la base de datos MySQL es la instalación de la librería que vamos a utilizar:

npm install mongoose

Esta acción nos da la posibilidad de utilizar todas las herramientas de Mongoose dentro del proyecto donde ha sido instalada.

Para poder realizar la conexión lo primero que debemos comprobar es si el servidor de MongoDB se encuentra activo en nuestro ordenador. Posteriormente, en la raíz del proyecto vamos a generar un fichero **dbConfig.js** que nos sirva para lanzar las tareas iniciales de conexión contra la base de datos.

```
const mongoose = require('mongoose');
const url = 'mongodb://127.0.0.1/tiendadb';
mongoose.connect(url);
const db = mongoose.connection;
db.on('error', console.error.bind(console, 'Error MongoDB'))
module.exports = db;
```


Tema 7. Integración de bases de datos Mongo DB

Analicemos en detalle este fichero:

- ▶ El primer paso es la definición de la url. Tomamos como **host** la url local de nuestro ordenador. El nombre **tiendadb** es el que le vamos a asignar a la base de datos (no es necesario crearla con antelación)
- ▶ Mediante el método **connect** pasándole la url que hemos definido realizamos la conexión.
- ▶ Además, rescatamos dicha conexión para posteriormente enlazar los posibles errores que existan con la salida estándar por consola.

Una vez definido el fichero de conexión pasamos a requerirlo desde **app.js** y que, de esta manera, se consiga conectar a la base de datos cuando arranque la aplicación:

```
require('./dbConfig');  
var app = express();
```

Podemos comprobar a arrancar la aplicación para ver si existe algún tipo de problema y tratar de solventarlo previo al desarrollo de los diferentes manejadores de rutas.

Tema 7. Integración de bases de datos Mongo DB

7.3 Creación de esquemas

Como ya analizamos cuando trabajamos sobre las propias bases de datos MongoDB, el trabajo que podemos realizar sobre las diferentes colecciones no se basa en ninguna estructura fija y podemos, de manera dinámica, ir modificando los campos y los valores que vamos a contener en cada uno de los documentos.

En el mundo del desarrollo web esta práctica es poco recomendable ya que nos puede llevar a mantener unas estructuras muy dinámicas y con poca posibilidad de utilizar de forma genérica. Es por ello por lo que con Mongoose disponemos de la posibilidad de generar **esquemas** que definan qué campos van a tener nuestros documentos y qué tipo de valores vamos a colocar en cada uno de ellos.

Vamos a trabajar con productos, por lo tanto, podemos generar, dentro de una carpeta models el archivo **product.js**

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;
const productSchema = new Schema({
  name: String,
  description: String,
  price: Number,
  department: String,
  available: Boolean,
  tags: [String]
});
module.exports = mongoose.model('product', productSchema);
```

Dentro de la variable **productSchema** estamos definiendo la serie de campos que van a tener cada uno de los documentos que insertemos dentro de esta colección. Para cada uno de los campos **definimos el nombre y el tipo que van a tener**.

Los tipos utilizados en este caso son los tipos propios que nos ofrece Mongoose y no están asociados a ninguno de los tipos vistos anteriormente. Podemos trabajar con los siguientes tipos:

Tema 7. Integración de bases de datos Mongo DB

String, Number, Date, Buffer, Boolean, Mixed, ObjectId, Array, Decimal128, Map, Schema.

Dependiendo de los datos que necesitemos almacenar, utilizamos un tipo u otro.

La última línea del fichero muy importante:

```
module.exports = mongoose.model('product', productSchema);
```

A través de esta sentencia, además de exportar el modelo para poder utilizarlo en otros ficheros también estamos asociando la colección en nuestra base de datos con el esquema que la representa en nuestro código. El primer parámetro del método **model** hace referencia al **nombre de la colección en la base de datos en singular**. El segundo parámetro es el objeto que representa el esquema con el que vamos a trabajar.

Es importante que prestemos atención al nombre que colocamos a la colección ya que automáticamente, si no existe, Mongoose se encarga de crearla en nuestra base de datos. Además, podemos tener problemas si definimos el nombre en castellano ya que no gestiona demasiado bien los plurales irregulares.

Una vez definido el esquema ya podemos utilizarlo. Para hacer una serie de pruebas, vamos a generar un fichero externo al proyecto que nos permita comprobar si la conexión y el esquema son correctos.

pruebasEsquemaProduct.js

```
(async () => {  
  const db = require('./dbConfig');  
  const Product = require('./models/product');  
  const prod = new Product();  
  prod.name = 'Test product II';  
  prod.description = 'Description test';  
  prod.price = 120;  
  prod.department = 'home';  
  prod.available = true;
```

Tema 7. Integración de bases de datos Mongo DB

```
prod.tags = ['test', 'uno', 'dos'];
await prod.save();
db.close();
})();
```

En este pequeño fichero requerimos la conexión sobre la base de datos y, a partir del modelo generado anteriormente generamos un objeto y modificamos sus propiedades. El último paso sería llamar al método **save** para persistir esta información dentro de la base de datos.

Podemos ejecutar este pequeño script a través de node

node pruebasEsquemaProduct.js

Y comprobar si se ha insertado correctamente el documento desde la consola de MongoDB. Existen incluso herramientas gráficas que nos permiten realizar esta interacción de manera más cómoda. Te ampliamos esta información en la sección **A fondo**.

El resultado de esta ejecución debería ser la creación de la base de datos, de la colección products y la inserción de un documento con los datos que hemos asignado.

Propiedades virtuales

Los esquemas que generamos no tienen por qué ser estáticos y depende únicamente de los campos definidos. Podemos establecer una serie de campos cuyos valores se obtengan de algún cálculo a partir de los campos que ya están insertados en la base de datos. Es lo que denominamos como **propiedades virtuales**.

En el caso de los productos vamos a definir una propiedad virtual que calcule el precio sumándole los impuestos correspondientes. Este valor quizá sí se usa mucho dentro de la aplicación, pero no es necesario que esté almacenado dentro de la base

Tema 7. Integración de bases de datos Mongo DB

de datos ya que es calculado a partir de otro valor.

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;
const productSchema = new Schema({
  name: String,
  description: String,
  price: Number,
  department: String,
  available: Boolean,
  tags: [String]
});
productSchema.virtual('price_w_taxes').get(function () {
  return this.price + (this.price * 0.21);
});
productSchema.virtual('price_w_taxes').set(function (newPrice) {
  this.price = newPrice / 1.21;
});
module.exports = mongoose.model('product', productSchema);
```

Dentro de nuestro esquema llamamos al método **virtual** que será el encargado de gestionar, tanto la acción de get (**recupera el valor**), como la acción de set (**actualiza el valor**).

Dentro de estas dos acciones situamos el cálculo que se va a realizar cuando queramos recuperar el valor o actualizarlo respectivamente.

Podemos modificar nuestro fichero de pruebas de la siguiente manera para observar cómo trabaja esta propiedad virtual:

```
(async () => {
  const db = require('./dbConfig');
  const Product = require('./models/product');
  const products = await Product.find();
  for (let product of products) {
    console.log(product.price_w_taxes);
  }
  db.close();
})();
```

Estamos usando el método **find directamente sobre el modelo** para recuperar

Tema 7. Integración de bases de datos Mongo DB

todos los documentos de la colección Products. El resultado de esta ejecución es un array el cual podemos recorrer para calcular el precio con impuestos de cada uno de los productos dentro de la base de datos.

Métodos de instancia

Además de la creación extra de este tipo de propiedades virtuales podemos definir asociados a nuestros esquemas una serie de métodos que nos ayuden a realizar actividades complejas sin tener la necesidad de repetir demasiado código.

Estos métodos se ejecutarán tomando como referencia un objeto del modelo con el que estemos trabajando. Este mismo concepto es el que aplicamos cuando generamos métodos dentro de las clases normales en javascript.

Por ejemplo, podríamos recuperar todos los productos que compartan departamento con el producto seleccionado.

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;
const productSchema = new Schema({
  name: String,
  description: String,
  price: Number,
  department: String,
  available: Boolean,
  tags: [String]
});
productSchema.virtual('price_w_taxes').get(function () {
  return this.price + (this.price * 0.21);
});
productSchema.virtual('price_w_taxes').set(function (newPrice) {
  this.price = newPrice / 1.21;
});
productSchema.methods.sameDepartment = function () {
  return this.model('product').find({ department: this.department });
}
module.exports = mongoose.model('product', productSchema);
```

Tema 7. Integración de bases de datos Mongo DB

Asociado al esquema que hemos generado podemos modificar la propiedad **methods** y asociarle claves nuevas. En este caso creamos un nuevo método llamado **sameDepartment** y como valor le pasamos la función que se va a ejecutar cuando la ejecutemos.

Dentro de la función no podemos acceder directamente al model y por eso utilizamos el método **model** para poder lanzar la función **find** que nos recupere todos los registros asociados.

En este caso llamamos a la función find **pasándole como parámetro un objeto con el filtro que queremos aplicar.**

Podemos probar este método de la siguiente manera dentro de nuestro fichero de prueba de la siguiente manera:

```
(async () => {
  const db = require('./dbConfig');
  const Product = require('./models/product');
  const prod = new Product();
  prod.department = 'home';
  const products = await prod.sameDepartment();
  console.log(products);
  db.close();
})();
```

Métodos estáticos

Del mismo modo podemos generar métodos que se ejecuten de manera genérica a partir del propio nombre del modelo.

Estos métodos nos sirven para resolver acciones repetitivas dentro de nuestras aplicaciones sin necesidad de estar constantemente repitiendo código.

En el esquema de product podemos incluir el siguiente método antes de exportar.

Tema 7. Integración de bases de datos Mongo DB

```
productSchema.statics.actives = function () {  
  return this.model('product').find({ available: true });  
}
```

Seguimos las mismas prácticas que para definir el método anterior. A partir de la propiedad **statics** generamos una nueva clave llamada **actives** donde incluimos una función. Dentro de esta función llamamos al método **find** para recuperar todos aquellos productos que cuadren con el filtro correspondiente.

Podemos probarlo de igual manera que el resto de los métodos.

Tema 7. Integración de bases de datos Mongo DB

7.4 Métodos CRUD de acceso a la base de datos

En este apartado vamos a definir una estructura de rutas sobre nuestra aplicación aprovechando el modelo que hemos generado y que nos permita conocer los métodos más comunes dentro de la librería Mongoose.

Para ello desarrollaremos las siguientes URLs:

GET <http://localhost:3000/api/products>

Recupera todos los productos de la base de datos.

POST <http://localhost:3000/api/products>

Recibe la información de un nuevo producto y lo almacena en la base de datos.

PUT <http://localhost:3000/api/products/IDPRODUCT>

Recibe toda la información del product a editar, además del id del mismo para poder completar el trabajo.

DELETE <http://localhost:3000/api/products/IDPRODUCT>

A través de la url recogemos el id del producto que vamos a borrar.

Para llevar esta tarea a cabo, generaremos una estructura muy parecida a la que hicimos en el tema anterior cuando generamos un API conectando a una base de datos MySQL.

Tema 7. Integración de bases de datos Mongo DB

routes/api.js

```
const router = require('express').Router();
const productsApiRouter = require('./api/products');
router.use('/products', productsApiRouter);
module.exports = router;
```

app.js

```
const apiRouter = require('./routes/api');
app.use('/api', apiRouter);
```

Con esta serie de ficheros bien enlazados podemos empezar a generar cada uno de los manejadores que se encargarán de responder a las peticiones entrantes.

routes/api/products.js

```
const router = require('express').Router();
const Product = require('../models/product');
router.get('/', async (req, res) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (err) {
    res.json({ error: err.message });
  }
})
module.exports = router;
```

Como podemos observar, en esta primera ruta hemos utilizado el método **find** para poder recuperar todos los productos asociados. El método **find** nos devuelve una promesa que consumimos a través de **async-await**.

A partir de la recuperación de estos productos respondemos al cliente con una serie de datos en formato JSON para que sean legibles en el destino.

Para poder generar un nuevo documento dentro de la colección **products** podemos usar varias técnicas. Ya hemos visto anteriormente como a través del método **save** se puede conseguir esto, pero dependemos de la creación de una instancia para ello.

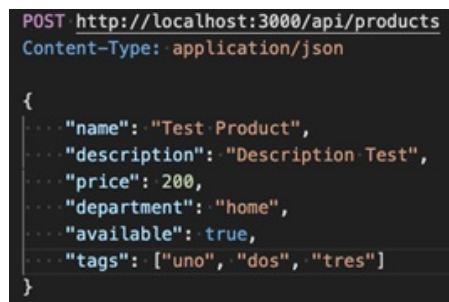
Tema 7. Integración de bases de datos Mongo DB

Podemos llegar al mismo resultado usando la función **create** la cual recibe como parámetro todos los datos del nuevo documento que vamos a generar.

```
router.post('/', async (req, res) => {
  try {
    const result = await Product.create(req.body);
    res.json(result);
  } catch (err) {
    res.json({ error: err.message });
  }
})
```

En este manejador utilizamos la función `create` pasándole todos los datos recibidos dentro de la petición a partir de **req.body**. De esta manera automáticamente estamos generando el nuevo documento dentro de la base de datos y recibiendo el resultado de dicha inserción.

Si probamos esta petición dentro de un fichero con extensión `.rest` podemos utilizar el siguiente esquema:



```
POST http://localhost:3000/api/products
Content-Type: application/json

{
  "name": "Test Product",
  "description": "Description Test",
  "price": 200,
  "department": "home",
  "available": true,
  "tags": ["uno", "dos", "tres"]
}
```

Figura 10. Esquema para la creación de productos. Fuente: elaboración propia.

El resultado de esta petición debería ser el nuevo objeto generado dentro de la base de datos con la que estamos trabajando.

En el caso de la actualización de datos debemos tener en cuenta qué producto es el que vamos a modificar y qué datos son los que conformarán parte a partir de ese momento del documento modificado.

Tema 7. Integración de bases de datos Mongo DB

El producto a modificar lo recibiremos a través de la url y los datos dentro del cuerpo de la petición entrante.

```
router.put('/:productId', async (req, res) => {
  try {
    const result = await Product.findByIdAndUpdate(
      req.params.productId,
      req.body,
      { new: true }
    );
    res.json(result);
  } catch (err) {
    res.json({ error: err.message });
  }
})
```

En este caso usamos la función **findByIdAndUpdate** que nos permite de una manera muy sencilla, pasando por parámetro el identificador del producto a modificar y los nuevos datos, realizar la actualización sobre el producto.

Como tercer parámetro debemos incluir el objeto **{ new: true }** para que el resultado devuelto por la ejecución de la función sea el objeto actualizado. Si no lo incluimos recibiremos el objeto antes de su actualización.

Podemos probar este manejador a partir de la siguiente estructura:

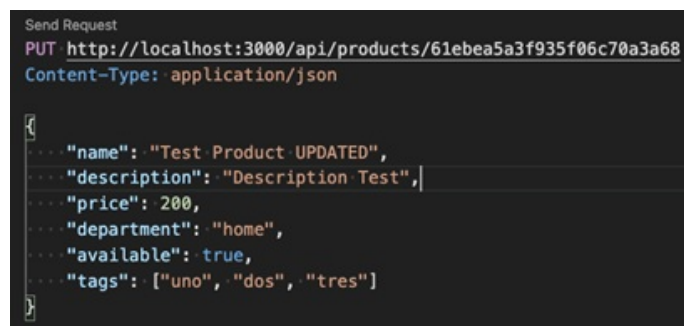


Figura 11. Prueba para la edición de datos. Fuente: elaboración propia.

Para realizar el borrado de productos, únicamente debemos tener en cuenta el

Tema 7. Integración de bases de datos Mongo DB

identificador del producto que vayamos a borrar.

```
router.delete('/:productId', async (req, res) => {
  try {
    const result = await Product.findByIdAndDelete(req.params.productId);
    res.json(result);
  } catch (err) {
    res.json({ error: err.message });
  }
});
```

El método **findByIdAndDelete** nos facilita mucho la acción de borrado ya que solo necesitamos pasar como parámetro el identificador del producto a borrar.

Existen numerosos métodos de ayuda dentro de la librería de Mongoose que nos ayudan en el momento de realizar las acciones más comunes sobre la base de datos.

Podemos probarlo a través de la siguiente estructura:

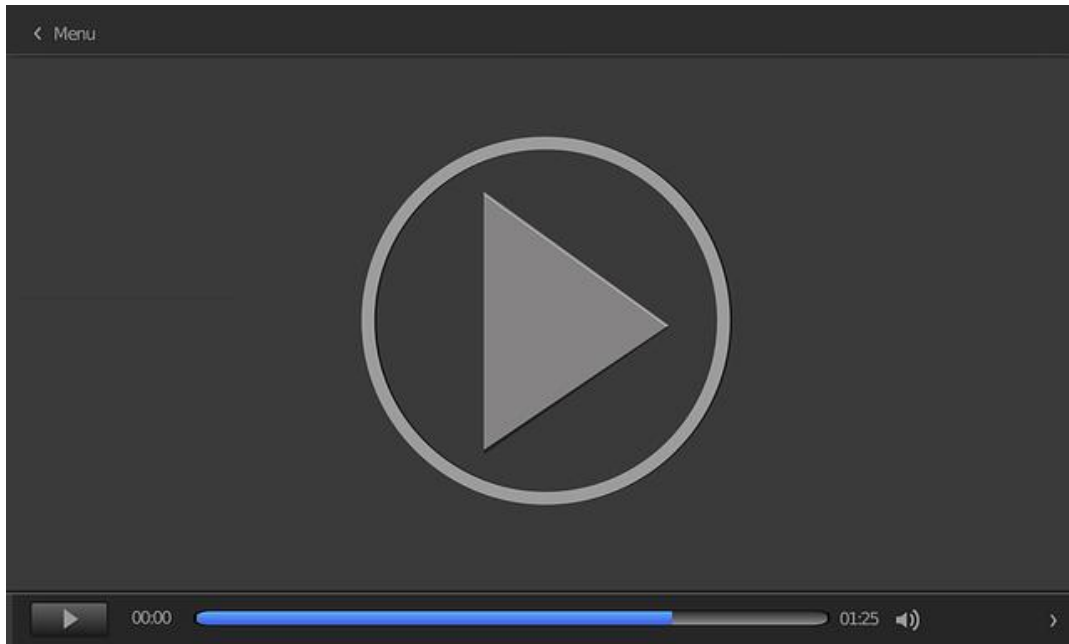


Figura 12. Prueba para el borrado de datos. Fuente: elaboración propia

De esta manera **quedarían definidos los métodos imprescindibles para la interacción con la tabla products**. Se podría ampliar mucho esta interfaz incluyendo manejadores que nos permitan recuperar todos los productos activos o incluso ordenados por departamentos.

Tema 7. Integración de bases de datos Mongo DB

En este vídeo, **API ExpressJS y MongoDB: CRUD**, a partir de una aplicación de ExpressJS generamos la estructura necesaria para responder a cualquier tipo de petición para la recopilación de datos, la creación de los mismos, la actualización y el borrado.

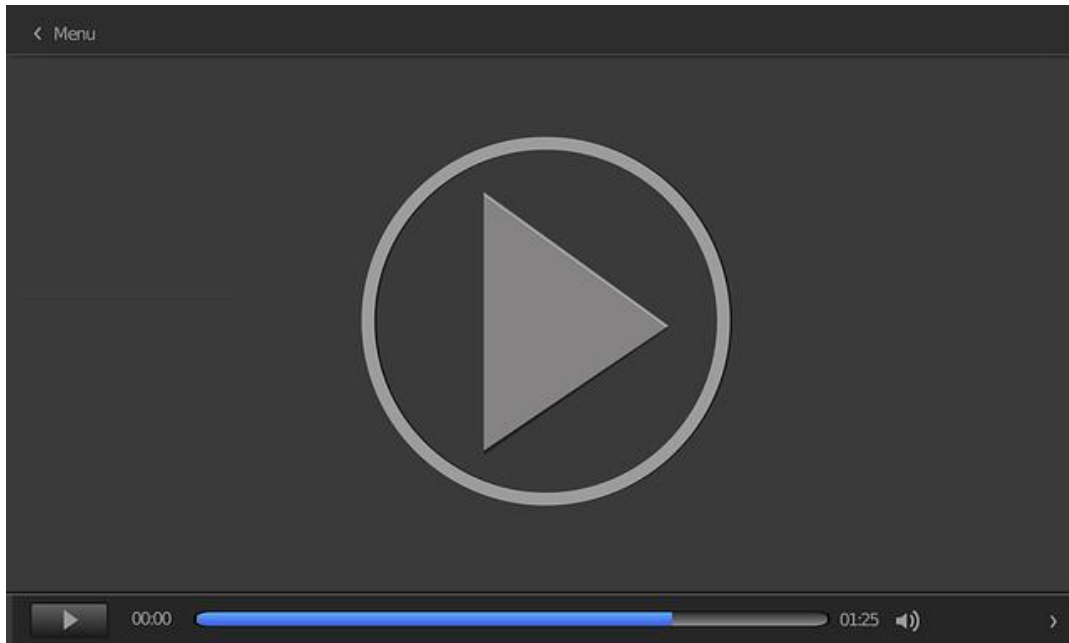


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=5df34355-01c8-4ca1-b4a5-ae3d012c5158>

Tema 7. Integración de bases de datos Mongo DB

En este vídeo, **API ExpressJS y MongoDB: relaciones**, completamos el ejemplo anterior generando una serie de relaciones entre los diferentes modelos de nuestra base de datos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=dd21456c-b598-44a1-84c7-ae3d01349d48>

Tema 8. Template Engines

8.1. Introducción y objetivos

Hemos completado en los temas anteriores una serie de aplicaciones basadas en Express que nos han permitido generar interfaces de acceso a datos a través del envío de ficheros de tipo JSON. Juntando todos los conocimientos adquiridos hasta ahora quizá es la forma de trabajo más habitual si necesitamos conectar varias aplicaciones entre sí.

Pero Express no es únicamente un framework para la creación de APIs. Con la ayuda de este potente framework podemos crear aplicaciones web completas, por lo que no tendríamos que depender de otras aplicaciones externas creadas en Angular, React, Vue...

Si usamos este otro tipo de librerías para poder generar la parte del front de nuestros desarrollos es porque nos queremos beneficiar de todas las características que nos ofrecen frente a la gestión tradicional con HTML y CSS.

En este tema vamos a tratar de diseñar aplicaciones con Express en las que, en vez de responder a través de un objeto con formato JSON, devolveremos una estructura HTML que será interpretada por el navegador tal y como estamos acostumbrados cuando navegamos de manera habitual por internet.

Tema 8. Template Engines

8.2. Gestión de plantillas en Express

Las primeras preguntas que nos debemos hacer son **¿qué es una plantilla y por qué utilizamos motores de plantillas para generarlas?**

Nuestro objetivo principal siempre ha sido el recibir peticiones en nuestra aplicación y generar respuestas acordes a la petición recibida. En este caso vamos a necesitar que nuestras respuestas envíen ciertos datos, pero en formato HTML. Es por ello que necesitamos una serie de **plantillas** que puedan ser rellenas en función de los datos que vamos a enviar al cliente.

Los **motores de plantillas** los usamos para que hagan el trabajo de coger los datos desde los manejadores de rutas, los coloquen en el espacio adecuado dentro del HTML y generen la vista que viajará al cliente para ser interpretada por un navegador.

Existen multitud de motores de plantillas disponibles y compatibles con Express, pero en este tema vamos a analizar dos, los que se encuentran más opuestos en su sintaxis, **PUG y HBS**.

El primer paso es **analizar cómo integramos la interacción del motor de plantillas dentro de una aplicación de Express**. Para ello vamos a generar una aplicación de ejemplo que nos sirva como base para el resto del tema.

`express --view=pug --git ejemplopug`

En la propia generación de la aplicación ya estamos especificando qué motor de plantillas es el que vamos a utilizar. Para este primer ejemplo nos decantamos por PUG.

Si analizamos en profundidad el fichero **app.js** encontramos estas dos líneas de código:

Tema 8. Template Engines

```
// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'pug');
```

En la primera de ellas estamos **modificando la propiedad views de nuestra aplicación de Express**. Le estamos pasando como valor una ruta dentro de nuestro proyecto. La librería **path** y en concreto el método **join** nos permiten concatenar la ruta del directorio de la aplicación con la palabra **views**. Estamos indicando que **dentro del directorio views de nuestro proyecto se van a encontrar todas las vistas que vamos a utilizar**.

(el nombre de la carpeta se puede modificar, pero es un nombre de directorio bastante estándar y comprensible por cualquiera que acceda detrás nuestro a la aplicación.)

En la segunda línea **modificamos la propiedad view engine de Express indicándole el valor del motor de plantillas que vamos a usar**. En este caso estamos usando **PUG**.

También podemos comprobar en el fichero **package.json** que tenemos como una de las dependencias del proyecto la propia librería **pug** para poder realizar todas las tareas que le encomendemos a la hora de transformar y renderizar las plantillas.

Si nos fijamos en el fichero **index.js** dentro de la carpeta **routes** encontramos un único manejador como el siguiente:

```
router.get('/', function (req, res, next) {
  res.render('index', { title: 'Express' });
});
```

Para responder a esta petición se está utilizando uno de los métodos que ya vimos en temas anteriores. El método **render** **toma una plantilla de la carpeta de views y gracias al motor de plantillas que estemos usando es capaz de transformarla en HTML y CSS que enviará de vuelta al cliente**.

Tema 8. Template Engines

El método por tanto recibe dos parámetros. El **primero** hace referencia al nombre de la vista que vamos a renderizar. En el ejemplo buscaremos la vista **index** dentro de la carpeta de vistas con la extensión del motor de plantillas que estemos utilizando (/views/index.pug).

El segundo parámetro es un objeto que pasamos desde el manejador de la ruta hacia la vista que vamos a renderizar. Este objeto es muy importante ya que a través del mismo le podremos hacer llegar a nuestras vistas todos los datos extraídos de las diferentes bases de datos con las que trabajemos.

Partiendo con esta base, vamos a ver qué ayudas nos aportan cada uno de los motores de plantillas que vamos a analizar en este tema.

Tema 8. Template Engines

8.3. Características básicas de PUG

PUG conforma una serie de herramientas para poder, con los datos recuperados dentro del manejador, generar las plantillas que vamos a visualizar dentro del navegador.

Su uso implica conocer una sintaxis un tanto diferente a la que estamos acostumbrados cuando estamos maquetando con HTML. A partir de ahora veremos cómo utilizando una serie de acciones concretas vamos a poder facilitarnos la vida a la hora de realizar construcciones complejas.

Etiquetas HTML

Dentro de la sintaxis de pug no necesitamos generar etiquetas de apertura y cierre para nuestra arquitectura HTML y simplemente colocando el nombre de la etiqueta la librería ya se encarga de completar el resto.

El tema de la jerarquía (es decir, qué etiquetas van dentro de otras), se soluciona mediante el uso de tabulaciones. Si una etiqueta o estructura está tabulada más a la derecha que la estructura anterior nos está indicando que está contenida dentro de la etiqueta con tabulación menor.

Por ejemplo, si necesitamos definir un div y dentro de ese div una lista no ordenada, lo podríamos hacer de la siguiente manera:

```
div
  ul
    li Elemento 1
    li Elemento 2
    li Elemento 3
```

Es muy importante definir con cuidado las tabulaciones para que todo cuadre

Tema 8. Template Engines

en el momento de la renderización.

Como podemos observar en los list-items, el contenido de cada uno de ellos se especifica dejando un espacio entre la etiqueta y dicho contenido.

Si dentro del manejador que renderiza esta plantilla pasamos algún tipo de objeto, podemos visualizarlo dentro de cualquiera de las etiquetas de dos maneras diferentes. Lo vemos a través del siguiente ejemplo:

```
router.get('/', function (req, res, next) {  
  res.render('index', {  
    nombre: 'Ramiro',  
    edad: 28  
  });  
});  
div  
ul  
li= nombre  
li La edad es #{edad}
```

Lo primero que debemos observar es que dentro del manejador estamos pasando como segundo parámetro del método render un objeto con las claves nombre y edad. Esas serán las claves que vamos a poder utilizar dentro de la plantilla.

Si el contenido de la etiqueta es exactamente igual al nombre de la variable que necesitamos renderizar se utiliza el símbolo igual pegado al nombre de la etiqueta y a su derecha especificamos qué variable estamos usando. En el ejemplo podemos verlo dentro del primer li.

Si además del valor de la variable queremos incluir algún otro texto dentro de la etiqueta seleccionada podemos utilizar la estructura `#{ }` para incrustar el valor de la variable en el espacio seleccionado. Lo hemos utilizado en el segundo li.

Atributos

Tema 8. Template Engines

Para nuestras etiquetas podemos seguir utilizando todos los atributos que ya conocemos de la maquetación normal con HTML. Lo vemos en el siguiente ejemplo:

```
a(href="http://www.google.es", class="button") Google.com
input(
  type="checkbox",
  name="acuerdo",
  checked
)
```

Podemos colocar los diferentes atributos que vayamos a utilizar entre paréntesis seguido del nombre de la etiqueta.

Un caso especial serían los atributos **id** y **class**, los cuales podemos especificarlos también de la siguiente manera.

```
a(id="main-link", class="contenido principal", href="http://google.es")
Google
a#main-link.contenido.principal(href="http://google.es") Google
```

Si utilizamos la línea de abajo podemos ahorrarnos gran parte de nuestra escritura y tendríamos resumidos al lado de la definición de la etiqueta los atributos de identificador y clases.

Es más, si la etiqueta que necesitamos representar es un **div** podemos omitir el tag que representa dicho div, quedando algo así.

```
div#primer.rojo.borde
#segundo.verde.borde
```

(es posible que, si en nuestro editor usemos **emmet** para autocompletar nuestro código, en ocasiones, nos haya borrado los div automáticamente, dejándolos como la segunda línea del ejemplo).

Comentarios

Tema 8. Template Engines

Al igual que la mayoría de los lenguajes de programación o gestión de estructuras, en Pug también contamos con comentarios para poder marcar diferentes puntos dentro de nuestro código.

```
// Comentario HTML - aparece en el navegador
//- Comentario PUG - no aparece en el navegador
```

De esta manera ya podemos poner notas dentro de nuestro código PUG en caso de que necesitemos dejar algún tipo de anotación.

Case

Se utiliza de la misma manera que el switch de Javascript. Por ejemplo, a través del manejador pasamos a nuestra plantilla un número de productos:

```
router.get('/', function (req, res, next) {
  res.render('index', {
    productos: 10
  });
});
```

Dentro de la plantilla, en función del valor de ese número podemos mostrar un bloque u otro.

case productos

```
when 0
  p Tienes cero productos
when 1
  p Tienes un producto
default
  p Tienes #{productos} productos
```

En este caso que necesitamos trabajar con plurales y mensajes diferentes dependiendo del número esta estructura aplica justo lo que necesitamos.

Tema 8. Template Engines

Código

Podemos incluir código Javascript dentro de cualquiera de las plantillas que vayamos a utilizar. Vamos a ver un ejemplo de cómo podríamos iterar un array recibido desde el manejador dentro de una lista no ordenada de elementos.

Esta misma acción la vamos a poder realizar un poco más adelante a través de los iteradores nativos de PUG.

```
router.get('/', function (req, res, next) {
  res.render('index', {
    animales: ['perro', 'gato', 'loro', 'elefante']
  });
});
ul
- for (let i = 0; i<animales.length; i++)
  li= animales[i]
```

De esta manera y manteniendo las normas básicas de creación de estructuras con pug podemos utilizar pequeños bloques de código en ciertas partes concretas de nuestra plantilla.

Condicionales

De la misma manera que definimos sentencias condicionales en la mayoría de los lenguajes de programación, podemos hacerlo dentro de nuestras estructuras de Pug.

```
router.get('/', function (req, res, next) {
  res.render('index', {
    usuario: { nombre: 'Laura', id: 13 },
    autorizacion: true
  });
});
#usuario
if(usuario.descripcion)
  h2 Descripción
```


Tema 8. Template Engines

```
p= usuario.descripcion
else if (autorizacion)
h2 Descripción
p El usuario no tiene descripción. Crea una
else
h2 Descripción
p El usuario no tiene descripción
```

Mediante el uso anidado de **if** – **ifelse** – **else** podemos ir creando estructuras condicionales dentro de nuestras plantillas a partir de los datos que recibamos.

Iteradores

Disponemos de dos iteradores básicos para poder repetir bloques de código dentro de nuestras plantillas: **each** y **while**.

Mediante el uso de **each** podemos iterar arrays u objetos complejos.

```
#animales
ul
each animal in animales
li= animal
```

La estructura que debemos utilizar es muy similar al **for-of** de javascript. Creamos un objeto (animal) donde se van a ir almacenando cada uno de los elementos del array que estemos iterando (animales) en cada una de las iteraciones.

También podemos acceder al índice de la iteración en el que nos encontremos:

```
#animales
ul
each animal, index in animales
li #{index} - #{animal}
```

Incluso podríamos iterar objetos más complejos definidos incluso dentro de la propia plantilla:

Tema 8. Template Engines

```
#animales
ul
each value, key in { title: 'First Post', description: 'La primera
entrada'}
li #{key} - #{value}
```

Una de las herramientas más potentes que tienen este tipo de iteradores es el uso de la cláusula **else** para poder mostrar un bloque de código en caso de que el array a iterar no tenga ningún elemento.

```
#animales
ul
each animal in animales
li= animal
else
li No hay animales
```

De igual manera podemos utilizar el iterador **while** si necesitamos repetir ciertos bloques de código. En este caso debemos tener muy presente la condición de ejecución del iterador para no obtener ningún tipo de bucle infinito.

```
- let n = 0
ul
while n < 10
li= n++
```

Extends

Gran parte de la estructura de nuestras páginas se repite. La barra de navegación o el menú sidebar es algo que vamos a necesitar en cada una de las páginas que vayamos a renderizar dentro de nuestras aplicaciones.

Para poder lograr esto sin necesidad de repetir demasiado código podemos usar la palabra reservada **extends** de pug para heredar de una plantilla padre.

Partimos del siguiente fichero **layout.pug**:

Tema 8. Template Engines

```
doctype html
html
head
  title= title
  link(rel='stylesheet', href='/stylesheets/style.css')
body
  block content
```

Definimos toda la estructura básica de la web y mediante la sentencia **block content** establecemos un punto donde vamos a poder agregar código dependiendo de la vista que vayamos a renderizar.

El nombre que le pongamos al bloque puede ser el que necesitemos, en función del espacio de la página que vayamos a rellenar. Por ejemplo, podríamos colocar algún bloque más con la siguiente fórmula.

```
doctype html
html
head
  title= title
  link(rel='stylesheet', href='/stylesheets/style.css')
body
  block content
  footer
  block footer
```

Cada vista que extienda de este layout debe ser responsable de pasar el código que vamos a situar en los dos bloques definidos. Lo vemos en el siguiente ejemplo:

```
extends layout.pug
block content
  h2 Contenido de la página
  p Aquí situamos las diferentes estructuras
block footer
  p Pie de página y copyright
```

En la primera línea especificamos de qué plantilla estamos heredando y posteriormente definimos el código que vamos a situar en cada uno de los bloques creados dentro de dicha plantilla.

Tema 8. Template Engines

De esta manera, compartiendo un esqueleto genérico podemos evitarnos repetir mucho código.

Includes

Mediante el uso de esta cláusula podemos incluir código pug de otro fichero totalmente diferente al que estamos renderizando. De esta manera extraemos bloques de código pug complejos y los podemos colocar en ficheros independientes.

Si generamos el fichero **includes/cabecera.pug** dentro del directorio de vistas

```
h1 Cabecera de la página
Lo podemos incluir dentro del layout:
doctype html
html
head
  title= title
  link(rel='stylesheet', href='/stylesheets/style.css')
body
  include includes/cabecera.pug
  block content
  footer
  block footer
```

De esta manera evitamos tener ficheros con muchísimas líneas de código delegando a otro tipo de ficheros ciertas funcionalidades o estructuras.

Mixins

Con los mixins podemos generar bloques de código reutilizables dentro de cualquiera de las plantillas de nuestra aplicación. Podríamos asemejarlos al uso de funciones en un lenguaje de desarrollo normal.

Los podemos definir de la siguiente manera:

Tema 8. Template Engines

```
mixin lista
ul
li uno
li dos
li tres
```

Posteriormente, en el punto donde queramos que aparezca el código del mixin lo usamos de la siguiente manera:

```
+lista
+lista
```

Al igual que las funciones, podemos usarlo tantas veces como necesitemos, obteniendo siempre el mismo resultado.

Es mucho más eficiente si le pasamos parámetros:

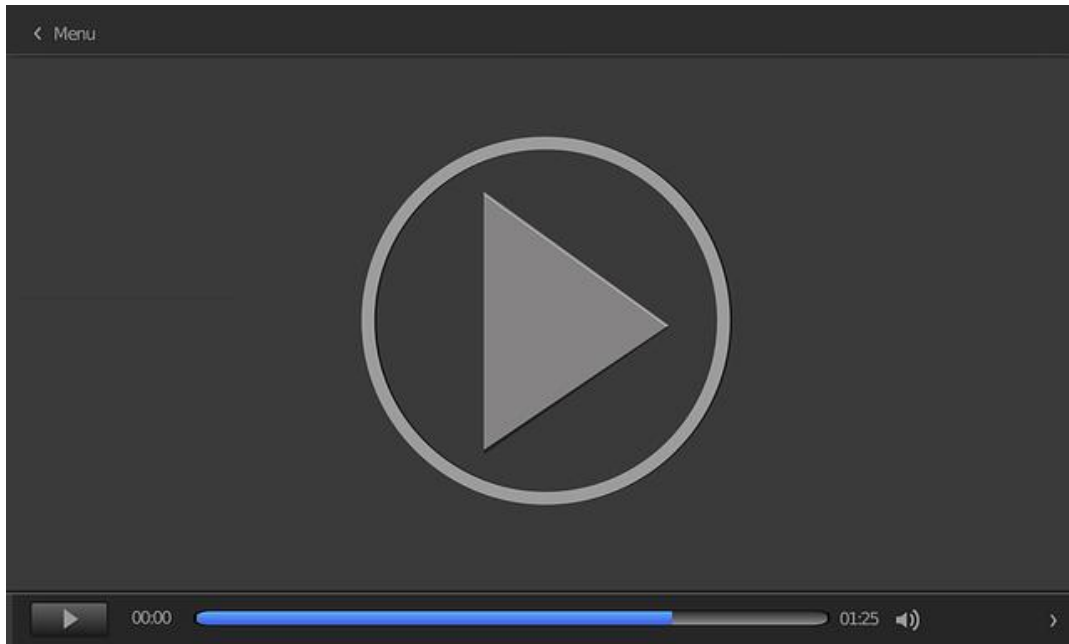
```
mixin lista (arr)
ul
each item in arr
li= item
else
li No hay elementos
```

De esta manera, podríamos pasarle un parámetro en la ejecución para iterarlo dentro del propio mixin. La ejecución sería la siguiente:

```
+lista(['perro', 'gato', 'loro'])
+lista(animales)
+lista([])
```

Tema 8. Template Engines

En este vídeo, ***Aplicación completa con PUG y MongoDB: lista de proyectos***, desarrollamos una aplicación completa que se conecta a una base de datos MongoDB y nos muestre la información a través de diferentes vistas hechas con PUG.

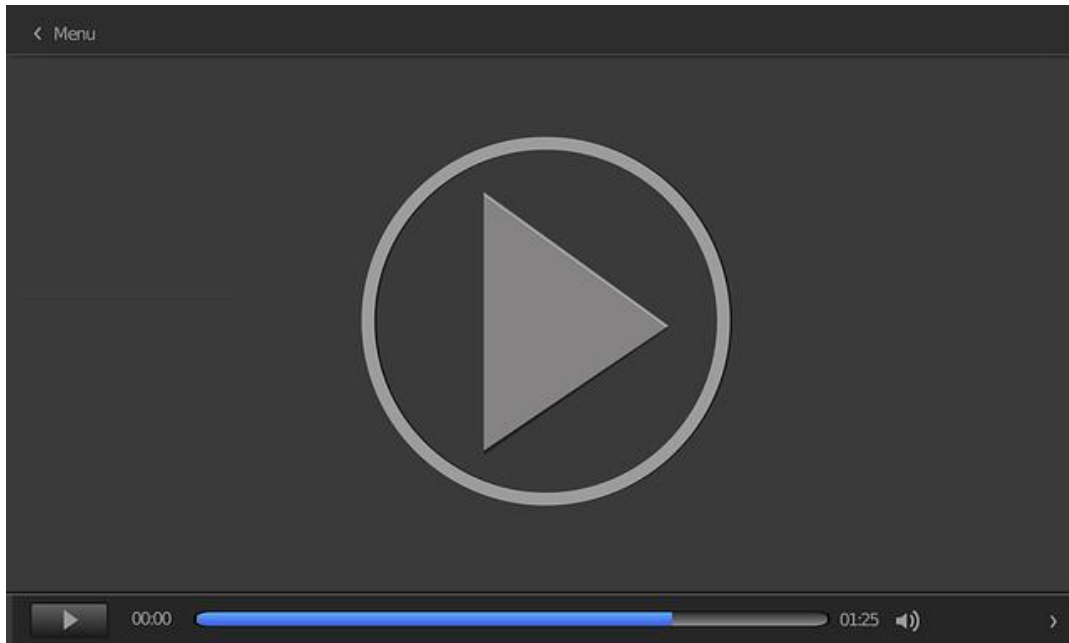


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=61750e2e-b479-4c38-b389-ae3e016848a5>

Tema 8. Template Engines

En este vídeo, **Aplicación completa con PUG y MongoDB: formulario de creación**, aprovechando el ejemplo anterior, incluimos el formulario y el código necesario para la creación de recursos.

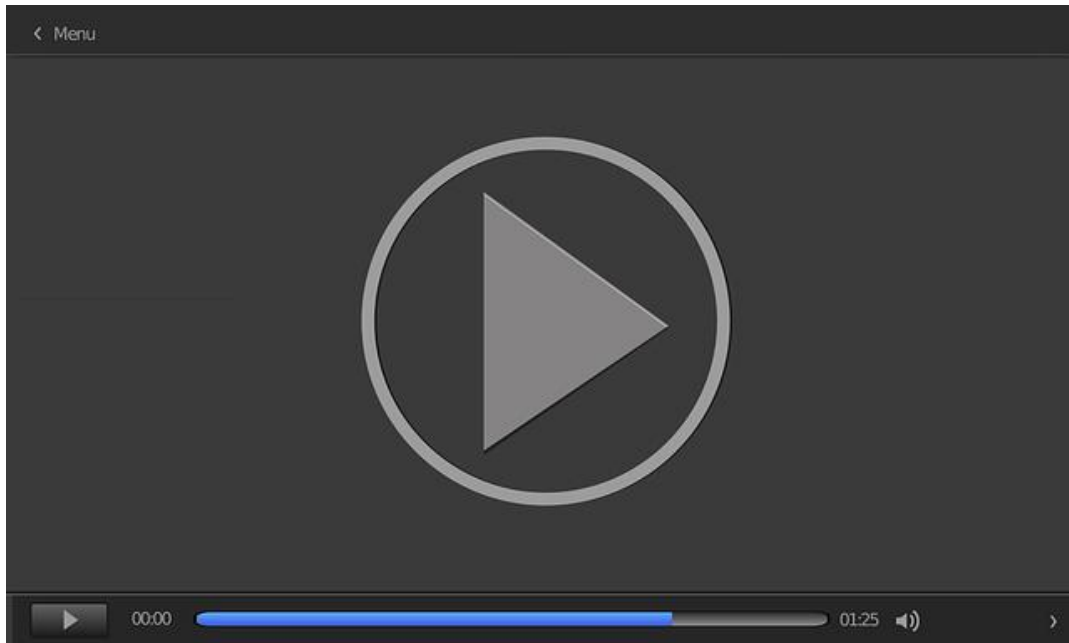


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=1f54533d-8d7b-463a-b1d1-ae3e016d4d93>

Tema 8. Template Engines

En este vídeo, **Aplicación completa con PUG y MongoDB: formulario de edición**, seguimos completando la aplicación anterior generando una funcionalidad que nos permita editar los diferentes recursos de nuestra base de datos.

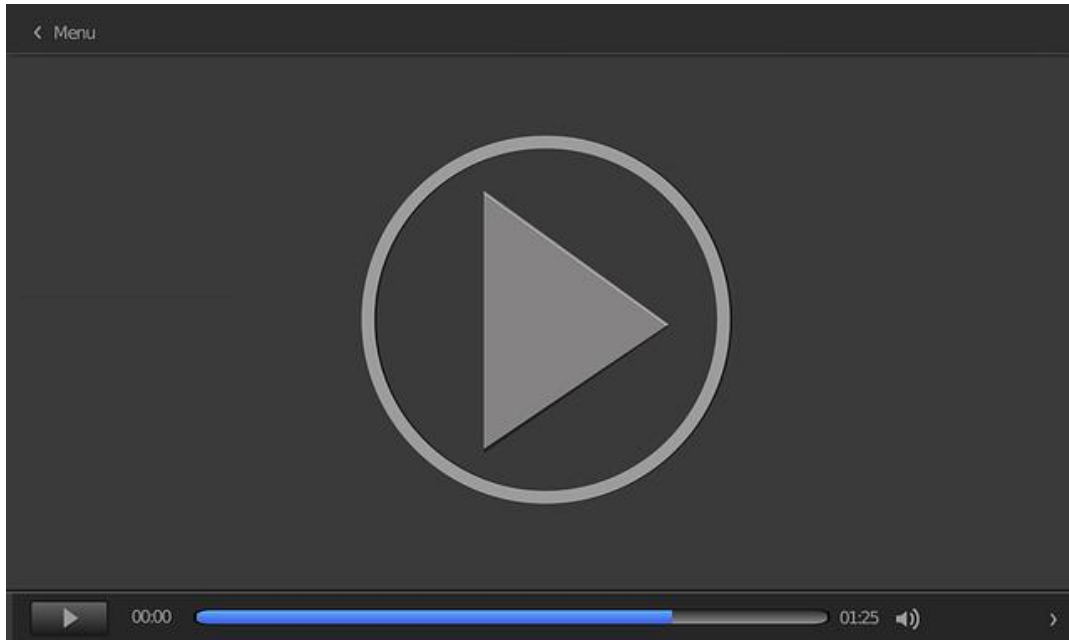


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=5fd7a140-94e2-4dc5-973f-ae3e016f8feb>

Tema 8. Template Engines

En este vídeo, **Aplicación completa con PUG y MongoDB: borrado**, cerramos el proyecto generando la acción de borrado de la base de datos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=27043589-e841-430d-b6c4-ae3e01726fd8>

Tema 8. Template Engines

8.4. Características básicas de HBS

Handlebars (HBS) es un motor de plantillas, como hemos visto con Pug, que se integra perfectamente dentro de una aplicación Express.

Una de sus características principales es que sí que utiliza las típicas etiquetas HTML con las que estamos acostumbrados a maquetar nuestros diseños.

El primer paso sería generar un proyecto para poder usarlo como referencia

express --view=hbs --git ejemplohbs

A partir de la estructura generada, dentro de la carpeta de views podemos encontrar el fichero **layout.hbs** con la siguiente jerarquía.

```
<!DOCTYPE html>
<html>
<head>
<title>{{title}}</title>
<link rel='stylesheet' href='/stylesheets/style.css' />
</head>
<body>
{{{body}}}
</body>
</html>
```

Es importante la definición dentro de nuestra plantilla de la variable **{{{ body }}}** ya que estamos indicando el espacio a sustituir por cada uno de los templates que rendericemos a partir de los manejadores de nuestra aplicación.

Dentro de una vista en concreto, si queremos visualizar variables enviadas desde el manejador a través del método render, podemos hacerlo de la siguiente manera.

```
<h1>{{titulo}}</h1>
<p>Mensaje: {{mensaje}}</p>
```

Como en el caso de PUG, podemos pasar a la vista todas las variables que

Tema 8. Template Engines

necesitemos.

Condicionales

Disponemos de la sentencia **if** para poder mostrar diferentes bloques dentro de nuestra estructura de manera condicional.

Hay que tener en cuenta que con HBS volvemos a la forma tradicional de especificar las etiquetas HTML, por lo que la sentencia condicional tendrá un punto de comienzo y un punto de final.

Dentro del manejador podemos renderizar una vista con las siguientes características.

```
router.get('/', function (req, res, next) {  
  res.render('index', {  
    mostrar: true,  
    nombre: "Antonio Peña"  
  });  
});
```

Dentro de la vista podemos lanzar el siguiente condicional.

```
{{#if mostrar}}  
<p>{{nombre}}</p>  
{{/if}}
```

Los condicionales de HBS son bastante limitados y no permiten el uso de comparadores. Si necesitamos más potencia en nuestras sentencias condicionales tendríamos que extender el motor de plantillas para que cumpla con nuestras expectativas.

La sentencia **if** puede ir acompañada por el **else** para marcar el sentido contrario de la condición.

Tema 8. Template Engines

```
{{#if mostrar}}  
<p>{{nombre}}</p>  
{{else}}  
<p>No se muestra el nombre</p>  
{{/if}}
```

Iteradores

En todos los motores de plantillas es imprescindible contar con una serie de herramientas que nos permitan iterar el contenido de las diferentes colecciones que extraigamos de la base de datos.

Para poder iterar un array o colección podemos utilizar la sentencia **each** de la siguiente manera.

```
<ul>  
  {{#each animales}}  
  <li>{{this}}</li>  
  {{/each}}  
</ul>
```

Dentro de la estructura determinada por el uso de **each** utilizamos la palabra reservada **this** para hacer referencia a cada uno de los elementos que formen parte de la iteración.

Podemos utilizar la cláusula **else** cuando queramos mostrar algún bloque de código en caso de que el array esté vacío.

```
<ul>  
  {{#each animales}}  
  <li>{{this}}</li>  
  {{else}}  
  <li>No hay animales</li>  
  {{/each}}  
</ul>
```

El uso únicamente de la palabra reservada **this** dentro del ámbito del iterador nos

Tema 8. Template Engines

puede llevar a confusiones. Es por ello que también podemos determinar una iteración mediante las siguientes prácticas.

```
<ul>
  {{#each animales as |animal|}}
  <li>{{animal}}</li>
  {{else}}
  <li>No hay animales</li>
  {{/each}}
</ul>
```

De esta manera queda mucho más comprensible y semántico.

Mediante el uso de **with** podemos mostrar o no ciertas partes de nuestro código en función de la existencia de un objeto dentro de la plantilla.

```
router.get('/', function (req, res, next) {
  res.render('index', {
    autor: { nombre: 'Miguel', apellidos: 'Delibes' }
  });
});
```

Dentro de la plantilla podríamos hacer lo siguiente:

```
<h1>Autor</h1>
{{#with autor}}
<h2>Escrito por {{nombre}} {{apellidos}}</h2>
{{/with}}
```

Para evitar confusiones como en el caso del each, podemos utilizar la siguiente sintaxis.

```
<h1>Autor</h1>
{{#with autor as |miAutor|}}
<h2>Escrito por {{miAutor.nombre}} {{miAutor.apellidos}}</h2>
{{/with}}
```

Una de las ventajas que tiene Handlebars es que es ampliable dependiendo de los objetivos que nos vayamos a plantear. Ampliamos dentro de la sección **A fondo** con una serie de herramientas que nos van a permitir ampliar el motor.

Tema 8. Template Engines

Tema 9. Sesiones y autenticación

9.1. Introducción y objetivos

Como ya hemos visto en temas anteriores, a través de Express podemos generar aplicaciones web completas que reciben peticiones desde cualquier tipo de cliente y generan respuestas acordes a dichas peticiones.

Una característica importante dentro de una aplicación web compleja es cómo gestionamos todos los datos y mantenemos una sesión abierta y una transmisión de datos constante entre el cliente y el servidor. Es importante saber quién está haciendo las peticiones y, en algunos casos modificar quién puede acceder o no a los recursos que estamos ofreciendo.

Además de analizar alguna de las herramientas que tenemos dentro de Express para gestionar sesiones y cookies, vamos a establecer una serie de normas para poder crear un acceso y autenticación para las peticiones que recibamos dentro de nuestras APIs.

Uno de los métodos más habituales para la autenticación de usuarios dentro de un sistema Full Stack es a través del uso de un token. Veremos en este tema qué posibles técnicas tenemos de cara a implementar este tipo de desarrollos dentro de nuestras propias aplicaciones.

Tema 9. Sesiones y autenticación

9.2. Configuración y manejo de sesiones con Express

Habitualmente denominamos con el nombre sesión al tiempo que estamos utilizando una aplicación web dentro de un navegador concreto. Durante ese tiempo, entre el navegador y el servidor se genera una especie de conexión en la que podemos almacenar ciertos datos para poder ser usados siempre que sea necesario.

Por ejemplo, el usuario en el navegador elige que el color con el que quiere ver nuestra aplicación es el oscuro. Todo el tiempo que dure la interacción con dicha aplicación el servidor sabe que debe utilizar el tono oscuro. Cuando el usuario cierre la aplicación, el navegador o cambie a otro diferente, esos datos que se han ido almacenando en sesión se pierden.

Esta transferencia de datos la podemos realizar de manera muy sencilla a través de Express.

El primer paso sería instalar la librería con la que vamos a trabajar

npm install express-session

Dentro de **app.js** (sobre un proyecto creado) configuramos la sesión para poder trabajar con valores a partir de ahí.

```
const session = require('express-session');
app.use(session({
  secret: 'frase secreta',
  resave: true,
  saveUninitialized: true
}));
```

Debemos definir la sesión a través de un middleware con los parámetros que observamos en el código anterior. El parámetro **secret** hace referencia a la frase con la cual se va a firmar el espacio dentro del navegador donde vamos a guardar los

Tema 9. Sesiones y autenticación

datos que utilicemos.

Mediante **resave** le estamos indicando que fuerce el guardado de la sesión dentro del espacio definido para ello dentro del navegador, aunque la sesión no se haya incluso inicializado.

El parámetro **saveUninitialized** fuerza a una sesión que no ha sido inicializada a guardarse dentro del navegador.

Desde el momento de la configuración podemos utilizar el objeto **req.session** dentro de cualquiera de nuestros manejadores para poder guardar ciertos datos y reutilizarlos más adelante según sea el uso que especifiquemos dentro de nuestra aplicación.

Lo vemos a partir de estos dos manejadores:

```
const router = require('express').Router();
router.get('/first', (req, res) => {
  req.session.test = 'Prueba de sesión';
  res.render('index');
});
router.get('/second', (req, res) => {
  console.log(req.session.test);
  res.render('prueba');
})
module.exports = router;
```

Dentro del primer manejador estamos agregando la propiedad **test** dentro del objeto que determina la sesión de nuestra aplicación. Le asignamos un valor que posteriormente vamos a querer recuperar. Si accedemos a la url <http://localhost:3000/first> ese valor quedará guardado dentro de la sesión.

Dentro del segundo manejador estamos intentando acceder al valor que hemos definido en el anterior bloque de código. Si accedemos a la url <http://localhost:3000/second> veremos como a través del terminal podemos observar el valor recibido.

Tema 9. Sesiones y autenticación

Para probar el verdadero uso de las sesiones podemos intentar realizar la misma acción cerrando la pestaña del navegador, cerrando el navegador por completo o cambiando a otro navegador totalmente diferente para ver cómo reaccionan los valores.

Tema 9. Sesiones y autenticación

9.3. Configuración y manejo de cookies con Express

Las cookies son pequeños bloques de información enviados desde el servidor que quedan almacenados dentro del navegador del cliente. Cuando desde dicho navegador se vuelva a acceder a la aplicación se podrán recuperar los datos almacenados viendo así cómo ha sido la interacción previa que ha tenido el usuario.

Se pueden almacenar datos de todos los tipos que marquen cómo el usuario ha ido configurando la aplicación a su gusto (filtros, colores, configuración de perfil...).

Si generamos la aplicación con el comando `express` ya tenemos, dentro de `app.js`, configurado el uso de cookies.

```
app.use(express.urlencoded({ extended: false }));
const cookieParser = require('cookie-parser');
```

Para establecer el valor de una cookie dentro del navegador podemos utilizar la siguiente sintaxis.

```
router.get('/third', (req, res) => {
  res.cookie('color', 'oscuro');
  res.render('index');
});
```

De esta manera estamos asociando a esta respuesta el envío de una clave llamada `color` con el valor `oscuro`. Si vamos al navegador y abrimos la consola, escribiendo **`document.cookie`** podríamos acceder a este valor.

Para poder recuperarlo en cualquier otro manejador procedemos del siguiente modo.

```
router.get('/fourth', (req, res) => {
  console.log(req.cookies.color);
  res.render('prueba');
});
```

Tema 9. Sesiones y autenticación

La gran diferencia de las cookies con respecto a las variables de sesión es que las primeras no se borran automáticamente cuando cerramos el navegador, se mantienen incluso entre varias sesiones. El usuario tiene la potestad de borrarlas cuando necesite, es por eso que no podemos confiar en que ese valor quede definido para siempre.

Disponemos del método **clearCookie** para eliminar cualquier tipo de cookie que hayamos almacenado en el navegador del cliente.

Tema 9. Sesiones y autenticación

9.4. Registro API Rest. Encriptación de password

A partir de este apartado vamos a plantear cómo podemos realizar un sistema dentro de nuestra aplicación de Express para poder gestionar el acceso de usuarios dentro de un API Rest.

Si el acceso tuviésemos que controlarlo desde las vistas generadas para ser visualizadas dentro de un navegador web sería más sencillo porque podríamos utilizar las dos herramientas vistas anteriormente. Mediante el uso de la sesión y el almacenamiento de datos en cookies podríamos controlar si un usuario está conectado o no.

En el caso de estar trabajando con un API Rest no podemos disponer de esta serie de herramientas porque no tenemos detrás un navegador que apoye nuestras acciones. Debemos utilizar otra técnica diferente.

Los pasos que vamos a seguir son los siguientes:

- ▶ Vamos a trabajar sobre una aplicación enganchada a una base de datos relacional de tipo MySQL. Dentro de esta base de datos generaremos la tabla **users** donde vamos a almacenar todos los datos de los usuarios que se registren en nuestra aplicación.
- ▶ Para poder acceder a los datos del API será obligatorio hacer *login* a partir de una ruta que generemos. La respuesta de este login incluirá un **token**. Esto no será más que un identificador que nos va a permitir indicarle al servidor qué usuario es el que está intentando acceder a los datos y si tiene concedidos esos permisos.
- ▶ En cada una de las peticiones que realicemos sobre la API incluiremos este token para que el servidor pueda determinar si tenemos permisos para acceder a los diferentes recursos existentes.

Tema 9. Sesiones y autenticación

El primer paso que vamos a realizar es la creación de la aplicación y la tabla dentro de la base de datos con la que vamos a trabajar.

express --no-view --git registrologin

En nuestra base de datos generamos la tabla **users** con los siguientes campos: **name, email, username, password**.

Column	Datatype	PK	NN	UQ	B...	UN	ZF	AI	G
id	INT	☒	☒	☐	☐	☐	☐	☒	☐
name	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐
username	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐
email	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐
password	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐

Figura 13. Características de la tabla users. Fuente: elaboración propia.

Dentro de la aplicación que hemos generado debemos seguir los pasos para conectar con la base de datos tal y como vimos en el tema de integración de bases de datos Mysql. Antes de realizar cualquier acción más compleja debemos estar seguros de que la base de datos funciona dentro de nuestra aplicación.

Registro

Para llevar a cabo la acción de registro necesitamos una url donde podamos recibir todos los datos del nuevo usuario. Además, dentro de esta serie de acciones vamos a tratar, dentro de nuestras posibilidades como desarrolladores web, incluir ciertos aspectos de seguridad encriptando la password del usuario antes de almacenarla en la base de datos.

El primer paso será generar la query que vamos a lanzar dentro del modelo de usuarios.

models/user.js

Tema 9. Sesiones y autenticación

```
const create = ({ name, username, email, password }) => {
  return db.query(
    'insert into users (name, username, email, password) values (?, ?, ?, ?)',
    [name, username, email, password]
  );
}
module.exports = {
  create
}
```

El siguiente paso será crear la url donde enviaremos todos los datos de registro del usuario y que además será la encargada de encriptar la password recibida antes de mandarla al método create.

Para poder realizar esta encriptación vamos a usar la librería **bcryptjs** la cual dispone de unos métodos muy sencillos para encriptar información y posteriormente hacer validaciones sobre estos datos.

npm install bcryptjs

El registro es la acción más sencilla ya que simplemente nos tenemos que preocupar de que los datos enviados por el cliente entren de manera correcta dentro de la base de datos.

routes/api/users.js

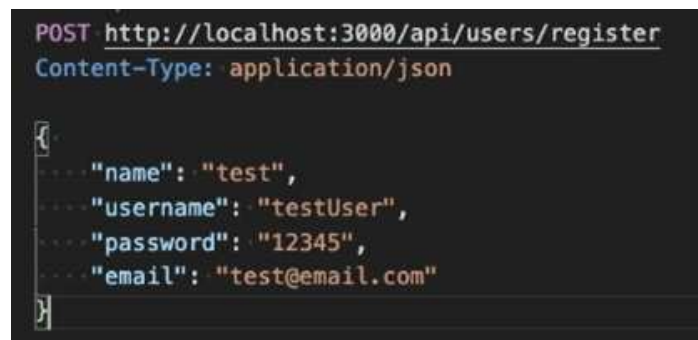
```
const router = require('express').Router();
const bcrypt = require('bcryptjs');
const UserModel = require('../models/user');
router.post('/register', async (req, res) => {
  // Los datos para el registro los recibimos en req.body
  // Encriptamos como primer paso la password
  req.body.password = bcrypt.hashSync(req.body.password, 10);
  // Procedemos a la creación del nuevo usuario
  try {
    const [result] = await UserModel.create(req.body);
    res.json(result);
  } catch (err) {
    res.json({ error: err.message });
  }
}
```

Tema 9. Sesiones y autenticación

```
});  
module.exports = router;
```

Mediante el método **hashSync** encriptamos la password antes de llamar al método del modelo que realiza la inserción en la base de datos.

Podemos probarlo a partir de la siguiente estructura.



```
POST http://localhost:3000/api/users/register  
Content-Type: application/json  
  
{  
  "name": "test",  
  "username": "testUser",  
  "password": "12345",  
  "email": "test@email.com"  
}
```

Figura 14. Prueba para el registro. Fuente: elaboración propia

Dentro de nuestra base de datos, si toda la operación se realiza correctamente podremos observar cómo la password se ha insertado de manera encriptada y no es posible volver al valor original con herramientas sencillas.

Tema 9. Sesiones y autenticación

9.5. Login API Rest

Una vez ya somos capaces de poder insertar nuevos usuarios dentro de nuestra base de datos, el siguiente paso sería generar una url que nos permita, mediante el envío del email y la clave, poder autenticarnos dentro de nuestra aplicación.

Para llevar a cabo el login de usuarios dentro de nuestra aplicación necesitamos comprobar los siguientes pasos:

- ▶ Comprobaremos si el usuario se encuentra registrado en nuestra base de datos. En caso contrario devolveremos el error **«email y/o password incorrecto»**.
- ▶ Si el usuario existe, comprobaremos la password recibida a través de la petición con la que tenemos almacenada en la base de datos. Esta pequeña acción la realizamos a través de **bcryptjs**. Si no coinciden devolvemos el error **«email y/o password incorrecto»**.
- ▶ Si todo ha ido correctamente devolveremos una respuesta satisfactoria adjuntando el token mediante el cual vamos a poder justificar quién es el usuario cada vez que quiera interactuar con los recursos de la base de datos.

Aunque la acción de comprobación de existencia del usuario y la comparación de las password son diferentes es recomendable mandar siempre el mismo mensaje de error para así evitar dar algún tipo de pista sobre si el email está siendo usado o no dentro de nuestra aplicación.

Primero generamos dentro del modelo de usuarios un método que nos permita recuperar el usuario a partir de su email.

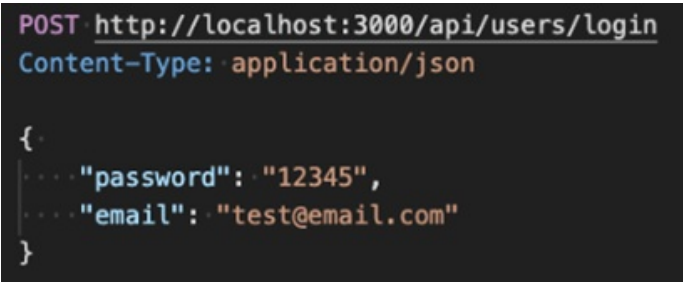
```
const getByEmail = (email) => {  
  return db.query('select * from users where email = ?', [email]);  
}
```

Tema 9. Sesiones y autenticación

Podemos definir el manejador de rutas, dentro del fichero **routes/api/users.js** de la siguiente manera:

```
router.post('/login', async (req, res) => {
  const [rows] = await UserModel.getByEmail(req.body.email);
  if (rows.length === 1) {
    // El usuario existe
    const user = rows[0];
    // Comprobamos si las password coinciden
    const iguales = bcrypt.compareSync(req.body.password, user.password);
    if (iguales) {
      res.json({
        success: 'Login correcto',
        token: createToken(user)
      })
    } else {
      res.json({ error: 'email y/o password incorrecto' });
    }
  } else {
    // El usuario no existe
    res.json({ error: 'email y/o password incorrecto' });
  }
});
```

Podemos probar esta url con la siguiente estructura:



```
POST http://localhost:3000/api/users/login
Content-Type: application/json

{
  "password": "12345",
  "email": "test@email.com"
}
```

Figura 15. Prueba para el login. Fuente: elaboración propia

En este desarrollo, la parte fundamental la encontramos en la respuesta correcta, en caso de que las passwords coincidan (automáticamente bcryptjs se encarga de comprobar esto por nosotros).

En la respuesta correcta estamos llamando al método **createToken** el cual nos va a

Tema 9. Sesiones y autenticación

permitir generar una cadena de caracteres **siguiendo el estándar JsonWebToken**.

Dentro de este token vamos a codificar información del usuario, así como la fecha en la que caduca este token.

Para poder crear este tipo de estructuras vamos a instalar la librería **jsonwebtoken**.

npm install jsonwebtoken

El método **createToken**, incluido dentro del mismo fichero anterior podría tener la siguiente estructura.

```
const jwt = require('jsonwebtoken');
const createToken = (user) => {
  const payload = {
    user_id: user.id,
    user_name: user.name
  };
  return jwt.sign(payload, 'frase muy secreta', { expiresIn: '1h' })
}
```

Como parámetros del método **sign** pasamos un objeto con los datos que queramos codificar. El segundo parámetro será la frase o grupo de caracteres que nos va a permitir firmar el token generado. Por último, agregamos una opción que nos va a permitir colocarle cierta caducidad a los token que generemos y así obligar al usuario a regenerarlo cada cierto tiempo.

Sería responsabilidad del cliente almacenar este token para poder enviarlo (dentro de las cabeceras) en cada una de las peticiones que realice sobre nuestra aplicación.

Tema 9. Sesiones y autenticación

9.6. Middleware de autenticación

Para poder disponer de ciertos recursos dentro de la aplicación que podamos limitar, vamos a generar el fichero **routes/api/products.js** con el siguiente contenido.

```
const router = require('express').Router();
router.get('/', (req, res) => {
  res.send('Recuperamos todos los productos');
});
router.post('/', (req, res) => {
  res.send('Generamos un nuevo producto');
});
router.put('/:productId', (req, res) => {
  res.send('Modificamos un producto');
});
router.get('/:productId', (req, res) => {
  res.send('Borramos un producto');
});
module.exports = router;
```

El fichero `routes/api.js` debería parecerse al siguiente:

```
const router = require('express').Router();
const apiUsersRouter = require('./api/users');
const apiProductsRouter = require('./api/products');
router.use('/users', apiUsersRouter);
router.use('/products', apiProductsRouter);
module.exports = router;
```

El objetivo de este apartado es, antes de poder interactuar con los diferentes recursos del api, comprobar si en la cabecera **Authorization** de la petición se ha incluido el token, éste es correcto y además no está caducado.

Podemos realizar esta comprobación con un middleware el cual, para organizarnos mejor, lo llevaremos a un fichero aparte. Dentro de este fichero además utilizaremos la librería **dayjs** para el manejo de fechas (npm install dayjs).

routes/middlewares.js

```
const jwt = require('jsonwebtoken');
const dayjs = require('dayjs');
```

Tema 9. Sesiones y autenticación

```
const checkToken = (req, res, next) => {
  // 1 - Comprobamos si la cabecera existe
  if (!req.headers['authorization']) {
    // Si la cabecera no existe, devolvemos el error
    return res.json({ error: 'Necesitas la cabecera de autorización' });
  }
  // 2 - Comprobamos si el token enviado es correcto
  const token = req.headers['authorization'];
  let payload;
  try {
    payload = jwt.verify(token, 'frase muy secreta');
  } catch (err) {
    return res.json({ error: 'El token es incorrecto' });
  }
  // 3 - Comprobamos la fecha de caducidad
  if (dayjs().unix() > payload.exp) {
    return res.json({ error: 'El token está caducado, pide otro' });
  }
  // Las comprobaciones pasan
  next();
}
module.exports = {
  checkToken
}
```

Hemos realizado 3 comprobaciones sobre el envío entrante, pero se podrían realizar todas las necesarias dependiendo de los requisitos de nuestro proyecto.

El siguiente paso, una vez tenemos este middleware de autenticación es aplicarlo sobre los recursos que queramos “esconder” a los usuarios no registrados. Podemos aplicarlo de manera global sobre todas las acciones relacionadas con los productos dentro del fichero **api.js**.

```
const router = require('express').Router();
const apiUsersRouter = require('./api/users');
const apiProductsRouter = require('./api/products');
const { checkToken } = require('./middlewares');
router.use('/users', apiUsersRouter);
router.use('/products', checkToken, apiProductsRouter);
module.exports = router;
```

Simplemente colocamos la definición del método **checkToken** que acabamos de

Tema 9. Sesiones y autenticación

desarrollar como segundo parámetro de la ejecución del método use para definir las diferentes rutas de los productos.

De esta manera, antes de lanzar cualquiera de los manejadores dentro del fichero de productos, la petición entrante pasará por el método checkToken, lanzando todas las comprobaciones que hemos desarrollado.

Podemos probar la recuperación de productos con la siguiente estructura.

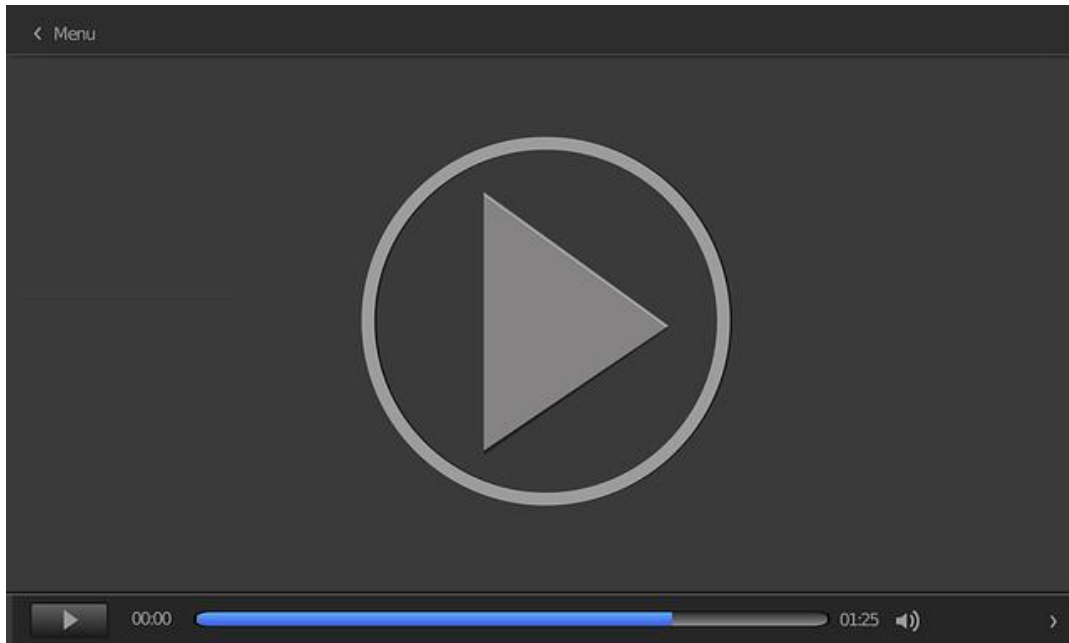
```
GET http://localhost:3000/api/products
Authorization: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VyX25hbWUiOiJ0ZXN0IiwiaWF0IjoxNjQzMDI3NzU5LCJleHAiOiJlE2NDMwMzEzMzI9.BLx2H2NaXmfJvNu5t8ZPGg9eFPqA9glbjNsTav2BZUw
```

Figura 16. Prueba para la comprobación del token. Fuente: elaboración propia.

El token incluido en esta prueba lo podemos recuperar dentro de la acción de login desarrollada anteriormente.

Tema 9. Sesiones y autenticación

En este vídeo, *Aplicación Registro-Login con MongoDB: registro*, repasamos todos los conceptos vistos hasta ahora e implementamos un proyecto en el que mediante ciertas rutas dentro de un API logramos generar un sistema de login y registro.

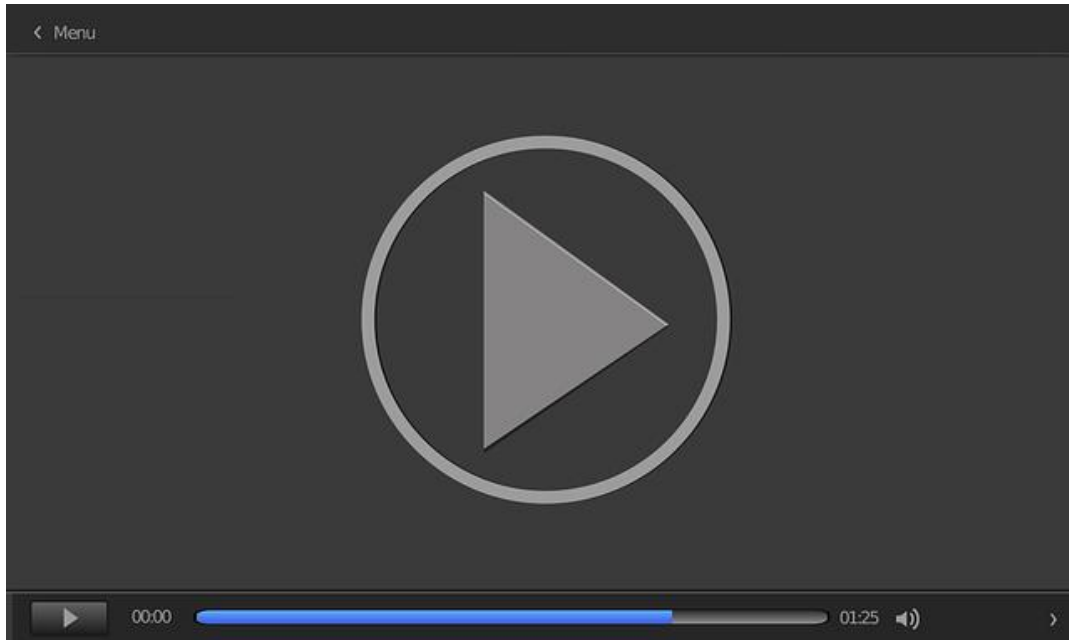


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=e93c5322-07e3-41b0-a2ba-ae3f0162f7c8>

Tema 9. Sesiones y autenticación

En este vídeo, *Aplicación Registro-Login con MongoDB: login*, aprovechamos el proyecto anterior y completamos la acción de login.

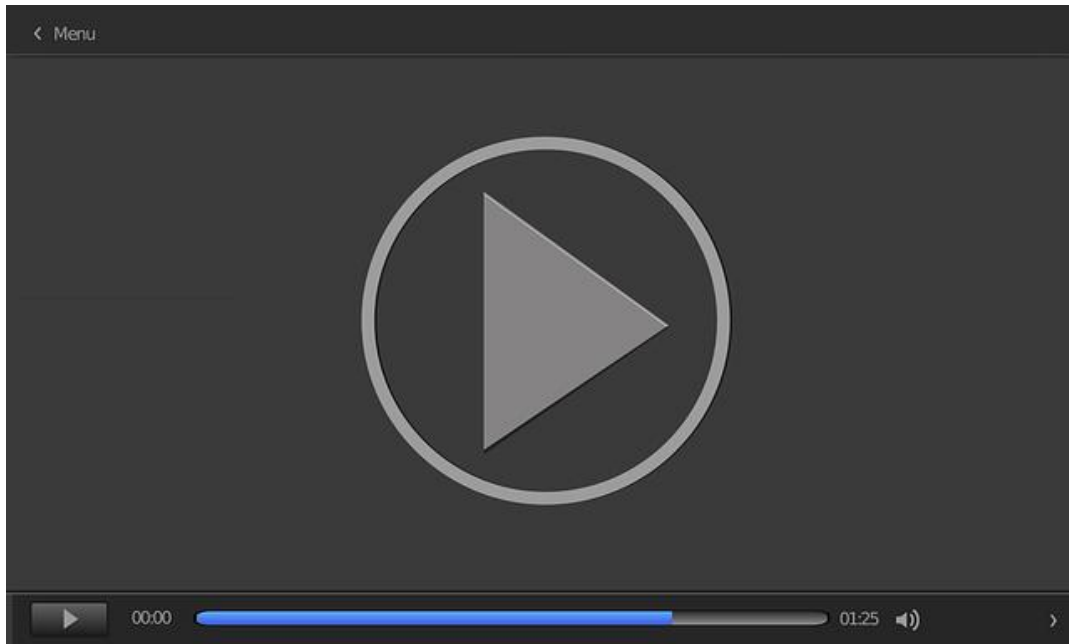


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=30a58ff0-af41-44d5-8e39-ae3f016f6309>

Tema 9. Sesiones y autenticación

En el siguiente vídeo, *Aplicación Registro-Login con MongoDB: autorización*, completamos el proyecto desarrollando un middleware que nos permita autenticar usuarios dentro de nuestra aplicación y por lo tanto nos permita de igual manera controlar el acceso a los recursos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=b3378672-58d8-4dae-98db-ae3f017441ba>

10.1. Introducción y objetivos

En el mundo de internet existen muchos modos de comunicación entre los diferentes personajes que encontramos dentro. Dependiendo de la tecnología que usemos y de las máquinas que vamos a poner en contacto utilizaremos una serie de herramientas u otras totalmente diferentes.

En los temas anteriores hemos tratado cómo se puede crear una conexión entre cliente y servidor a través del protocolo HTTP, que es el más común cuando trabajamos dentro de internet.

En este tema vamos a intentar tratar con un modo de comunicación que incluye otras características totalmente diferentes a las que venimos acostumbrados hasta ahora. Analizaremos las diferentes fórmulas para lograr una comunicación en tiempo real que nos permita el envío de mensajes y datos sin necesidad de realizar peticiones extra.

Este modo de comunicación nos dará pie a la creación de aplicaciones de «tipo chat», notificaciones push, y un sinfín de funcionalidades que se salen de la manera tradicional de creación de aplicaciones web.

10.2. ¿Qué son los websockets?

Para trabajar dentro de nuestras aplicaciones utilizaremos las diferentes herramientas que nos proporciona el API de Websockets. Este api y todos los métodos que incluye nos permitirán definir una comunicación bidireccional entre cliente y servidor.

Esta conexión creada entre cliente y servidor está constantemente abierta, es decir, que tanto desde el cliente como desde el servidor se pueden enviar constantemente datos. Con estas simples características obtenemos mucha potencia dentro de nuestras aplicaciones agregando comunicaciones en tiempo real.

Aunque es una tecnología relativamente joven, la podemos utilizar en la mayoría de los navegadores de escritorio y también en los navegadores móviles, por lo que no deberíamos tener demasiados problemas de compatibilidad. Al ser un API genérico de Javascript, todos los navegadores deberían recibir las actualizaciones que se produzcan sobre los diferentes objetos y métodos.

Además de ver una tecnología de trabajo nueva también nos va a servir para enfrentarnos a la interacción con nuestro código a través de eventos.

10.3. Ejemplo básico de websocket

Vamos a realizar un ejemplo muy sencillo para que veamos cómo podemos generar un servidor simple de websockets y poder establecer una comunicación continua entre un cliente y un servidor.

Nuestro servidor simplemente se limitará a repetir los mensajes que enviemos desde el cliente. Más adelante, con el apoyo de librerías más especializadas podremos empezar a crear interacciones mucho más complejas y sobre todo realizando una interacción vía web.

Para inicial el proyecto necesitamos lanzar los siguientes comandos en el terminal:

mkdir ejemplows

cd ejemplows

npm init -y

De esta manera ya tenemos generado el proyecto y creada la estructura del fichero **package.json** donde situaremos los diferentes scripts y las dependencias.

Para poder trabajar con websockets necesitamos la instalación de la **librería ws**.

npm install ws

A partir de esto, generamos un fichero **index.js** con el siguiente contenido.

```
const WebSocketServer = require("ws").Server;
const wss = new WebSocketServer({ port: 3000 });
// Recibimos los eventos de conexión sobre el WebSocketServer
wss.on("connection", (socket, req) => {
  socket.send("Bienvenido al WS");
  socket.send(`Esta es tu IP: ${req.socket.remoteAddress}`)
  // Nos enganchamos al evento message del servidor.
  socket.on("message", (message) => {
    message = message.toString();
```

```
if (message === 'exit') {
  socket.close();
} else {
  // Mandamos el mensaje broadcast a todos los clientes conectados
  wss.clients.forEach(function (client) {
    client.send(message);
  });
}
});
// Recibimos el aviso de que se ha cerrado la conexión
socket.on('close', () => {
  console.log('Termina la conexión');
})
});
```

Al principio del script estamos generando un servidor de websockets sobre el puerto 3000 y lo estamos poniendo a escuchar las diferentes peticiones que reciba.

A partir de aquí podemos trabajar con eventos. Es la manera más sencilla de, en tiempo real, saber qué está pasando dentro de nuestro servidor sin necesidad de lanzar bucles infinitos. NodeJS es capaz de avisarnos cuando los diferentes eventos sucedan dentro de nuestro servidor.

El primero evento que estamos gestionando es **connection**. Este nos avisa cuando se ha creado una nueva conexión contra nuestro servidor y es por ello que en el método que gestiona la nueva conexión estamos recibiendo la instancia del **socket**, es decir, **la conexión abierta y bidireccional entre el cliente y el servidor**.

A partir de recibir esta conexión ya podemos enlazar otros eventos como **message**, el cual nos avisa que ha entrado un mensaje a través de la conexión abierta o **close**, mediante el cual podemos ejecutar ciertas sentencias de código cuando el cliente se desconecta del servidor.

Además, disponemos del método **send** mediante el cual podemos mandar mensajes a todos los clientes conectados.

Para poder probar este ejemplo necesitamos un cliente de websockets. Dentro del store de **Google Chrome** tenemos uno bastante amigable. Se puede descargar desde el siguiente enlace: <https://chrome.google.com/webstore/detail/simple-websocket-client/gobngblklhkgmjhbpbdlkglbhhlafjnh/related?hl=es-419EditCookies>

Una vez instalado lo podemos configurar con los datos de la imagen:

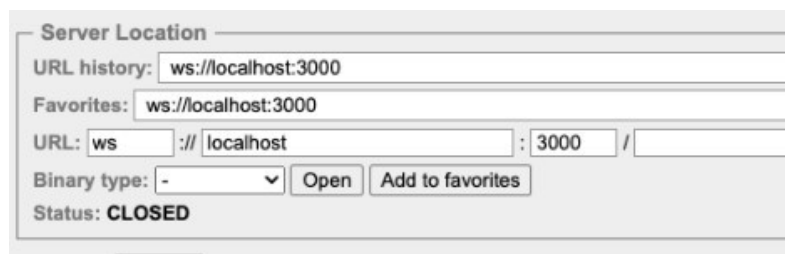


Figura 17. Configuración para la conexión con el Websocket. Fuente: elaboración propia.

Si pulsamos en el botón de **Open** el servidor quedará conectado con el cliente y podremos empezar a mandar mensajes que será respondidos del mismo modo.

A partir de este pequeño ejemplo podemos empezar a construir herramientas mucho más potentes.

10.4. Uso de la librería Socket.io

Socket.io es una librería que nos permite establecer una comunicación en tiempo real entre un servidor y multitud de clientes. Dispone de las herramientas necesarias para que esta conexión y el trabajo de comunicación entre todos los participantes sea muy fácil de implementar.

Además, dentro de las herramientas disponibles, tenemos las que se encuentran destinadas a la parte del servidor, y por lo tanto podemos usarlas asociadas a una aplicación de Express y las orientadas a la parte del cliente, que nos permiten que, desde cualquier página HTML podamos enviar mensajes en tiempo real.

Sigue una estructura de comunicación **basada en eventos**. Muchos de estos eventos ya están definidos dentro de la propia librería, pero tenemos total flexibilidad para crear nuestros eventos propios dependiendo del objetivo de nuestros proyectos.

Para poder probar las acciones más sencillas de conexión entre un cliente y un servidor, vamos a establecer un pequeño proyecto arrancándolo de la siguiente manera

```
mkdir ejemplosocketio
```

```
cd ejemplosocketio
```

```
npm init -y
```

Nuestro ejemplo va a ser un simple contador que, interactuando con el click de varios botones seamos capaces de modificarlo. Para ello necesitamos establecer el código para el **cliente y para el servidor**.

Servidor

El código dentro del servidor será el encargado de recibir todos los eventos lanzados desde los diferentes clientes conectados y transmitirlos si es necesario. Además, contará con la gestión del contador e informará de sus cambios a los diferentes clientes.

El primer paso para poder generar el servidor es instalar la librería.

npm install socket.io

A partir de esto podemos crear el fichero **server.js** con el siguiente contenido:

```
const io = require("socket.io")(3000);
let contador = 0;
io.on("connection", socket => {
  console.log('Nuevo cliente');
  socket.send("¡Bienvenido!");
  socket.on("decrementa", (data) => {
    contador--;
    io.emit('contador', {
      action: 'decrementa',
      value: contador
    });
  });
  socket.on("incrementa", (data) => {
    contador++;
    io.emit('contador', {
      action: 'incrementa',
      value: contador
    });
  });
});
```

Vamos a ir analizando cada uno de los pasos previos.

```
const io = require("socket.io")(3000);
let contador = 0;
```

El primer paso dentro del script implica la propia creación del servidor y la especificación del puerto donde lo estamos levantando. Sobre ese puerto dentro de nuestra máquina local se podrán lanzar peticiones con el protocolo ws que

posteriormente procederemos a responder adecuadamente.

Además, inicializamos la variable que nos servirá como contador. Al ser un script de prueba, este contador se reinicia cada vez que paremos el servidor. Si estuviésemos creando otro tipo de scripts tendríamos que introducir algún tipo de persistencia para que ese valor se mantenga.

```
io.on("connection", socket => {  
  });
```

Con este evento estamos indicando al servidor que queremos “ser avisados” cuando un nuevo cliente se conecte.

Además, en el método que se ejecuta cuando recibimos este evento, podemos recuperar la instancia del **socket**, es decir, la conexión abierta entre el cliente que se ha conectado y el servidor que estamos desarrollando. A partir de este socket, podemos empezar a enviar y recibir mensajes y datos.

```
console.log('Nuevo cliente');  
socket.send("¡Bienvenido!");
```

Nada más se conecte el nuevo cliente, dejamos una referencia en el log del servidor para comprobar dicha conexión y mediante la instancia del socket enviamos un mensaje de bienvenida que posteriormente tendremos que recuperar dentro del cliente.

```
socket.on("decrementa", (data) => {  
  contador--;  
  io.emit('contador', {  
    action: 'decrementa',  
    value: contador  
  });  
});  
socket.on("incrementa", (data) => {  
  contador++;  
  io.emit('contador', {  
    action: 'incrementa',  
    value: contador  
  });  
});
```

```
});  
});
```

Estas dos acciones son muy parecidas. Hemos definidos dos eventos propios (**incrementa y decrementa**) y a través del método **on** estamos generando dos funciones que se ejecutarán cuando recibamos cada uno de estos eventos.

De manera interna, los eventos modifican el contador dependiendo si es incremento o decremento y además emiten otro evento personalizado llamado **contador**. Como segundo parámetro de este evento pasamos un objeto con las propiedades **action y value** (podrían ser cualquier otra) que viajará a todos los clientes avisando de los cambios que se han realizado sobre el contador.

De esta manera, generando los eventos que necesitemos y enviando los datos asociados a los mismos, podemos crear nuestros propios canales de comunicación a través de sockets.

Cliente

Para poder llevar a cabo el cliente, primero debemos instalar una serie de herramientas en nuestra máquina. Estas herramientas las instalamos porque al trabajar en local tenemos ciertas restricciones que debemos intentar evitar.

Dentro de Visual Studio podemos instalar la siguiente extensión para trabajar sobre un servidor con nuestros ficheros

H T M L <https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>

En este caso concreto vamos a utilizar el navegador Google Chrome para poder llevar a cabo las pruebas. Debemos instalar la siguiente

Tema 10. Sockets

herramienta <https://chrome.google.com/webstore/detail/allow-cors-access-control/lhobafahddgcelffkeicbaginieeejlf?hl=es>

Una vez instalada, debemos activar el control de CORS para que las respuestas recibidas desde el servidor, aunque vengan desde una url local sean reconocidas.

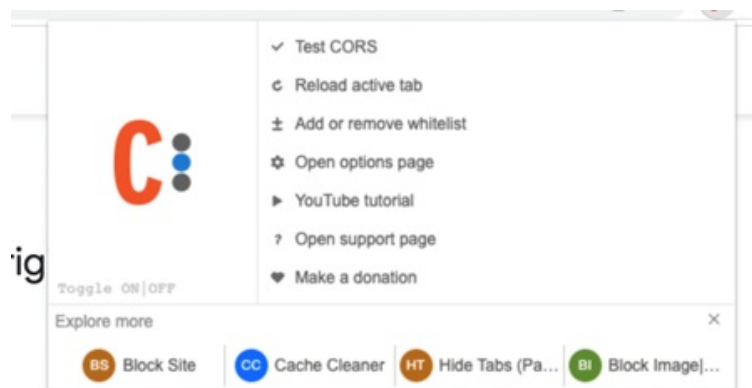


Figura 18. Extensión Google Chrome para manejo del Cors activa. Fuente: elaboración propia

A partir de aquí ya podemos empezar a crear la parte del cliente para nuestra aplicación dentro del fichero **index.html**.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script src="https://cdn.socket.io/4.4.1/socket.io.min.js"
integrity="sha384-
fKnu0iswBIqkxrxhQCTZ7qLLH0FEgNkRmK2va0/LbTZSXdJfAu6ewRBdwHPhBo/H"
crossorigin="anonymous"></script>
</head>
<body>
<p id="mensajes"></p>
<button id="btnDecrementar">DECREMENTAR</button>
<p id="contador"></p>
<button id="btnIncrementar">INCREMENTAR</button>
<script src="./client-socket.js"></script>
```

```
</body>  
</html>
```

Dentro de esta estructura encontramos cómo en la cabecera estamos incluyendo el script de Socket.io para poder trabajar con él. Lo incluimos a través de su CDN, pero podríamos utilizar cualquier otra técnica admitida.

Dentro del body de la página incluimos dos botones y dos párrafos de texto para poder interactuar con ellos dependiendo de las acciones que enviemos y recibamos por parte del servidor.

Pasamos por tanto a implementar el script **client-socket.js**.

```
const socket = io("ws://localhost:3000");  
const btnDecrementar = document.getElementById('btnDecrementar');  
const btnIncrementar = document.getElementById('btnIncrementar');  
const mensajes = document.getElementById('mensajes');  
const contador = document.getElementById('contador');  
btnDecrementar.addEventListener('click', (event) => {  
  socket.emit('decrementa');  
});  
btnIncrementar.addEventListener('click', (event) => {  
  socket.emit('incrementa');  
});  
socket.on("message", (data) => {  
  mensajes.innerText = data;  
});  
socket.on('contador', (data) => {  
  contador.innerText = data.value;  
  mensajes.innerText = data.action;  
});
```

Como en el caso anterior, analizamos cada uno de los pasos.

```
const socket = io("ws://localhost:3000");
```

El primer paso es **realizar la conexión con el servidor levantado**. De esta manera recibimos en el cliente la misma instancia del **socket** con la que estábamos trabajando en el servidor.

```
const btnDecrementar = document.getElementById('btnDecrementar');
const btnIncrementar = document.getElementById('btnIncrementar');
const mensajes = document.getElementById('mensajes');
const contador = document.getElementById('contador');
Con métodos propios de Javascript recuperamos todos aquellos elementos del
DOM con los que vamos a trabajar.
btnDecrementar.addEventListener('click', (event) => {
  socket.emit('decrementa');
});
btnIncrementar.addEventListener('click', (event) => {
  socket.emit('incrementa');
});
```

También intervenimos los eventos **click** de cada uno de los botones. En este caso, aprovechando la instancia del socket emitimos un evento por cada uno de ellos, dependiendo si es el botón de incremento o de decremento.

Recordemos que estos eventos (decrementa – incrementa) están siendo recogidos dentro del servidor.

```
socket.on("message", (data) => {
  mensajes.innerText = data;
});
socket.on('contador', (data) => {
  contador.innerText = data.value;
  mensajes.innerText = data.action;
});
```

A través de la instancia del socket también nos podemos subscribir a los diferentes eventos que envía el servidor para, dependiendo del evento recibido, realizar una acción concreta.

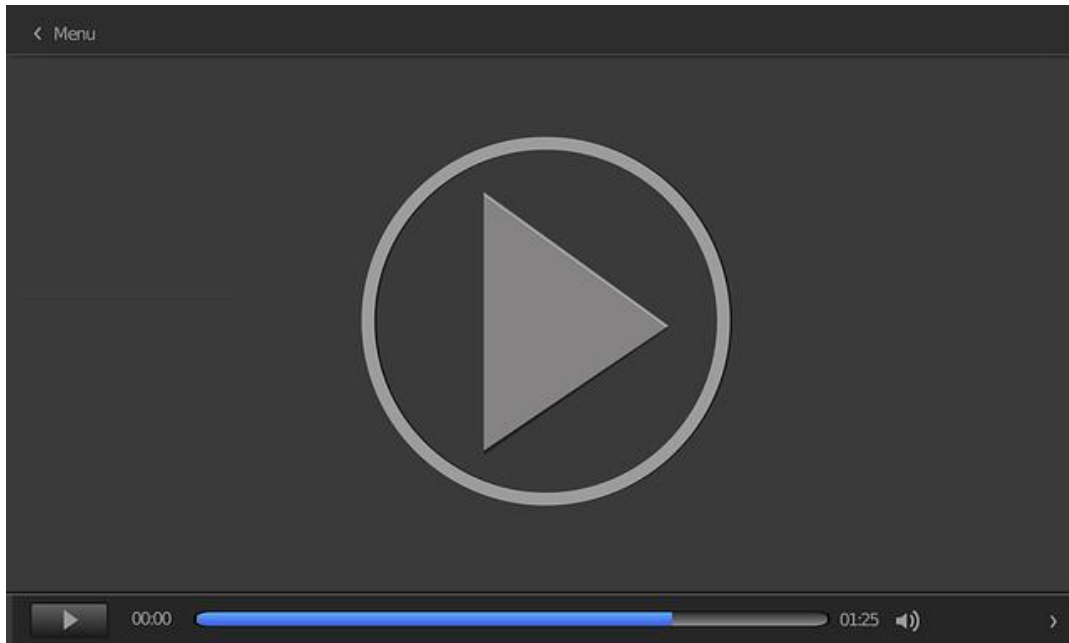
Para poder probar todo el sistema:

- ▶ Levantamos el servidor con **node server.js**
- ▶ A través del live server de Visual Studio abrimos el fichero **index.html**
- ▶ Entramos a Chrome y navegamos a la URL que nos ofrezca el Live Server. Podemos abrir tantas instancias como necesitemos del navegador y ver cómo interactúa el

cliente con el servidor levantado.

Tema 10. Sockets

En este vídeo, *Chat con Socket.io*, explica cómo, aprovechando la potencia que nos proporciona la librería Socket.io, desarrollamos una aplicación de chat para poder lograr una comunicación en tiempo real entre cliente y servidor.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=52b3969a-1b02-4545-bd32-ae3f0179da72>
